

【代码块可以用``"+回车】

注意，若有多组输入，每组样例输入完后会加EOFError，所以要写try+except弹出，防止阻塞。

N诺输出格式

1216 数字对其

输入输出样例

输入样例#:

复制

5

输出样例#:

复制

1	16	15	14	13
2	17	24	23	12
3	18	25	22	11
4	19	20	21	10
5	6	7	8	9

解决方法:

```
print("%-3s " % str(j), end='')#%-3s表示 'str(j)' 占用三个字符，向右对齐，多余字符用空格补充
```

1159 输出间隔问题

输入输出样例

输入样例#:

复制

3
1 90
2 87
3 92

输出样例#:

复制

2 87
1 90
3 92

解决方法:

```
#看清间隔大小，此处无需加空格
for i,s in res_order:
    print(i,s)
#若加空格，间隔明显偏大，输出格式为
'''
input:
3
1 90
2 87
3 92
output:
2 87
1 90
3 92
'''
```

global与nonlocal

global指引用全局变量，nonlocal只能用于嵌套函数中

```
a = 0
def change_a():
    global a
    a = 1
    print(a) #1
def change():
    b = 0
    def c_b():
        nonlocal b
        b = 1
        print(b) #1
    #global b
    #b = 1 # 报错:没有b这个全局变量
```

format自适应小数点

```
price = int(input())
discount = 1
if price >= 1000 and price < 2000:
    discount = 0.95
elif price >= 2000 and price < 3000:
    discount = 0.9
elif price >= 3000 and price < 5000:
    discount = 0.85
elif price >= 5000:
    discount = 0.8
pay = discount * price
print("discount={},pay={:g}".format(discount,pay))##{:g}自动去掉小数点后的0
```

format用法

1、基本用法

- (1) 不带编号，即“{}”
- (2) 带数字编号，可调换顺序，即“{1}”、“{2}”
- (3) 带关键字，即“{a}”、“{tom}”

```
>>> print('{} {}'.format('千锋','教育')) # 不带字段
千锋 教育

>>> print('{0} {1}'.format('千锋','教育')) # 带数字编号
千锋 教育

>>> print('{0} {1} {0}'.format('千锋','教育')) # 打乱顺序
千锋 教育 千锋
```

```
>>> print('{1} {1} {0}'.format('千锋','教育'))
教育 教育 千锋

>>> print('{a} {b} {a}'.format(a='千锋',b='教育')) # 带关键字
千锋 教育 千锋
```

2、进阶用法

(1) < (默认) 左对齐、> 右对齐、^ 中间对齐、= (只用于数字) 在小数点后进行补齐

(2) 取位数"{:4s}"、"{:.2f}"等

```
>>> print('{} and {}'.format('千锋','教育')) # 默认左对齐
千锋 and 教育

>>> print('{:10s} and {:>10s}'.format('千锋','教育')) # 取10位左对齐, 取10位右对齐
千锋          and          教育

>>> print('{:^10s} and {:^10s}'.format('千锋','教育')) # 取10位中间对齐
  千锋      and      教育

>>> print('{} is {:.2f}'.format(1.123,1.123)) # 取2位小数
1.123 is 1.12

>>> print('{0} is {0:>10.2f}'.format(1.123)) # 取2位小数, 右对齐, 取10位
1.123 is          1.12

3、多个格式化
'b' - 二进制。将数字以2为基数进行输出。
>>> print('{0:b}'.format(3))
11

'c' - 字符。在打印之前将整数转换成对应的Unicode字符串。
>>> print('{:c}'.format(20))
4

'd' - 十进制整数。将数字以10为基数进行输出。
>>> print('{:d}'.format(20))
20

'o' - 八进制。将数字以8为基数进行输出。
>>> print('{:o}'.format(20))
24

'x' - 十六进制。将数字以16为基数进行输出, 9以上的位数用小写字母。
>>> print('{:x}'.format(20))
14

'e' - 幂符号。用科学计数法打印数字。用'e'表示幂。
>>> print('{:e}'.format(20))
2.000000e+01

'g' - 一般格式。将数值以fixed-point格式输出。当数值特别大的时候, 用幂形式打印。
>>> print('{:g}'.format(20.1))
20.1
```

'n' - 数字。当值为整数时和'd'相同，值为浮点数时和'g'相同。不同的是它会根据区域设置插入数字分隔符。

```
>>> print('{:f}'.format(20))
20.000000
>>> print('{:n}'.format(20))
20
```

'%' - 百分数。将数值乘以100然后以fixed-point('f')格式打印，值后面会有一个百分号。

```
>>> print('{:%}'.format(20))
2000.000000%
```

print方法

```
a = "12"
b = "3"
#非指定end时，print默认end为换行符
print("%-3s" % str(a), end='!')#“-”为左对齐，“3”表示取3位，不够则空格补位，“%”后的
str(a)对应于“-3s”
print("%3s" % str(b), end='!')
#a: 12 !
#b:  3!
```

字符串输出

%s

%10s——右对齐，占位符10位

%-10s——左对齐，占位符10位

%.2s——截取2位字符串

%10.2s——10位占位符，截取两位字符串

```
print('%s' % 'hello world') # 字符串输出
#hello world
print('%20s' % 'hello world') # 右对齐，取20位，不够则补位
#hello world
print('%-20s' % 'hello world') # 左对齐，取20位，不够则补位
#hello world
print('%.2s' % 'hello world') # 取2位
#he
print('%10.2s' % 'hello world') # 右对齐，取2位
#he
print('%-10.2s' % 'hello world') # 左对齐，取2位
#he
```

字符类

ASCII码转换

字符转ascii

字符之间的减运算，那么在python中就可以借助ord()函数实现，ord()是python自带的函数，无需导入。

```
>>> print(ord('c')-ord('a'))  
2
```

借助ord()函数不仅可以进行字符减运算，也可以进行加运算，但是我们一般不会这么用。

```
>>> print(ord('c')+ord('a'))  
196
```

ascii转字符

python也有一个将ASCII码值转化成字符的函数——chr()。

```
>>> print(chr(99))  
c
```

字母大小写转换

upper(): 所有字母大写

lower(): 所有字母小写

capitalize(): 首字母大写，其他字母小写

title(): 每个单词首字母大写，其他小写

```
#encoding:UTF-8  
msg = 'www.BAIDU.com.123'  
print(msg.upper()) #upper() 函数，将所有字母都转换成大写  
print(msg.lower()) #lower() 函数，将所有字母都转换成小写  
print(msg.capitalize()) #capitalize() 函数，将首字母都转换成大写，其余小写  
print(msg.title()) #title() 函数，将每个单词的首字母都转换成大写，其余小写  
'''  
WWW.BAIDU.COM.123  
www.baidu.com.123  
www.baidu.com.123  
Www.Baidu.Com.123  
'''
```

isalpha()

isalpha() 方法检测字符串是否只由字母组成。如果字符串至少有一个字符并且所有字符都是字母则返回 True，否则返回 False。

```
#!/usr/bin/python  
# coding=utf-8  
  
str = "runoob";  
print str.isalpha();
```

```
str = "runoob菜鸟教程";
print str.isalpha();

str = "this is string example....wow!!!";
print str.isalpha();
'''
True
False
False
'''
```

去除字符串的空格

去除字符串开头或者结尾的空格

```
>>> a = " a b c "
>>> a.strip()
'a b c'
```

去除字符串开头的空格

```
>>> a = " a b c "
>>> a.lstrip()
'a b c '
```

去除字符串结尾的空格

```
>>> a = " a b c "
>>> a.rstrip()
' a b c'
```

去除全部空格

replace

```
>>> a = " a b c "
>>> a.replace(" ", "")
'abc'
```

split()+join()

split()可以去除字符串中的空格，并按空格分割成list

```
>>> a = " a b c "
>>> b = a.split() # 字符串按空格分割成列表
>>> b ['a', 'b', 'c']
>>> c = "".join(b) # 使用一个空字符串合成列表内容生成新的字符串
>>> c 'abc'
# 快捷用法
>>> a = " a b c "
>>> "".join(a.split())
'abc'
```

字符串与数组

数组转字符串：

```
''.join(['a','b','c'])#'abc'
```

字符串转数组：

```
list('abc')#['a','b','c']
'a b c'.split()#['a','b','c']
```

列表类

注意事项

列表的append使用的是**引用传递**，即加入的是原数据地址，对原数据进行pop等操作时同时也会改变使用append原数据的数据

解决方法：利用list.copy() 或者使用list[:]代替list

```
a = [1,2,3]
b = [a] # [[1,2,3]]
c = [3]
b.append(a) # [[1,2,3],[1,2,3]]
b.append(c) # [[1,2,3],[1,2,3],[3]]
a.pop()
c=[4]
b # [[1,2],[1,2],[3]]
b.append(a.copy()) # [[1,2],[1,2],[3],[1,2]]
b.append(a[:]) # [[1,2],[1,2],[3],[1,2],[1,2]]
a.pop()
b # [[1,2],[1,2],[3],[1,2],[1,2]]
```

在使用append时要注意，当目标list之前有其他变量引用时，值也会变化如：

```
a=[1]
b=a
c = a[:]
d =a+[2]
a.append(3)
# a [1,3]
# b [1,3]
# c [1]
# d [1,2]
```

enumerate循环语句

```
while True:
    try:
        string = list(input())
        list1 = []
        for i in string:
            if i.isalpha():
                list1.append(i)
                #大写全变为小写，sort可满足 当字母相同不影响本来输入的数据的顺序
        list1.sort(key=lambda c: c.lower())
        #此类循环可以得到index值
        for index, i in enumerate(string):
            if not i.isalpha():
                # 可通过insert直接插入得到结果，无需另构建一个空字符串相加获得结果
                list1.insert(index, i)
        print(''.join(list1))
    except:
        break
```

index方法

```
#用法，获取列表中值为val的索引值，但只能寻找一维的list
# 当列表中有多个相同val值时，只返回最小的索引值
a = [1,2,2,3]
val = 2
a.index(val)
# 1
```

insert方法

函数用于将指定对象插入列表的指定位置。

语法

insert()方法语法：

```
list.insert(index, obj)
```

参数

- index -- 对象 obj 需要插入的索引位置。
- obj -- 要插入列表中的对象。

返回值

该方法没有返回值，但会在列表指定位置插入对象。

实例

以下实例展示了 insert()函数的使用方法：

实例

```
#!/usr/bin/python

aList = [123, 'xyz', 'zara', 'abc']

aList.insert( 3, 2009)

print "Final List :", aList
```

以上实例输出结果如下：

```
Final List : [123, 'xyz', 'zara', 2009, 'abc']
```

删除元素

五种方式：

- remove():一次删除一个元素；如果列表内有重复元素则删除第一个；元素不存在时抛出异常 ValueError
- pop():删除一个指定的索引位置上的元素；指定索引不存在则抛出异常 IndexError 切片:一次至少删除一个元素
- clear():清空列表
- del:删除列表

remove ()

指定值

1. 一次删除一个元素；
2. 如果列表内有重复元素则删除第一个；
3. 元素不存在时抛出异常ValueError。

代码

```
lst = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print("原列表为: ", lst)
lst.remove(5)
print("删除后的列表为: ", lst)
```

```
原列表为: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
删除后的列表为: [0, 1, 2, 3, 4, 6, 7, 8, 9]
```

pop()

删除一个**指定的索引**位置上的元素；指定索引不存在则抛出异常IndexError

注意：如果不指定参数，则默认删除最后一个元素

```
lst = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print("原列表为: ", lst)
lst.pop(5)
print("删除后的列表为: ", lst)
```

```
原列表为: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
删除后的列表为: [0, 1, 2, 3, 4, 6, 7, 8, 9]

Process finished with exit code 0
```

切片

```
# 切片的本意为截取原有列表中指定的某一段，或者说是复制指定的那一段，并返回了新的列表
lst = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print("原列表为: ", lst)
# 借助列表的添加方式，原来添加的操作中，使得lst[start,stop] = lst1（此为添加的元素）
# 那么删除操作可以类似的，使lst1为空列表，那么原列表中指定一段的元素被空替代，则完成删除
lst[1:3] = [] # 将第1,2索引位的元素删除
print("删除后的列表为: ", lst)
```

clear与del

clear为清空列表中的元素，列表对象还存在

del直接删除了列表对象

```
lst = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print("原列表为: ", lst)
lst.clear()
print("删除后的列表为: ", lst)

lst = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print("原列表为: ", lst)
del lst
print("删除后的列表为: ", lst)
```

结果：

```
原列表为: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
删除后的列表为: [] ← clear
原列表为: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Traceback (most recent call last):
  File "D:\files\pythonProject\test\list\listremove.py", line 38, in <module>
    print("删除后的列表为: ", lst)
NameError: name 'lst' is not defined ← del
```

count方法

统计列表中某个元素出现的次数

```
#用法[ ].count(object)
[1,2,1,3].count(1)
# 2
```

数组实现栈

```
"""
isEmpty 判断是否为空
push    添加新元素（及最后被添加的元素为栈顶）
pop     移出栈顶元素
peek    查看当前栈顶元素
size    计算数量
"""

class Stack:#创建一个栈
    def __init__(self):#初始为空栈
        self.items = []
    def isEmpty(self):
        return self.items ==[]
    def push(self, item):
        self.items.append(item) #添加新成员，新成员为顶，旧骨干为底
    def pop(self):
        return self.items.pop() #移出顶，底上位
    def peek(self):
        #查看当前顶部成员
        return self.items[-1]
    def size(self):
        #查看当前成员数量
        return len(self.items)
```

差分与前缀和方法

适用题型：某一个区间同时加减一个数

如下题，每次对某一区间的数进行加减操作，最后查询某坐标的数

若干行输入，每行两个数字 $B[i], E[i]$ ($0 \leq B[i] \leq E[i] \leq 200000$) 表示这次刷的墙壁是哪一段
(假设每次刷的时候油漆颜色都和之前的不同)，以0 0结束
又若干行输入，每行两个数字 $begin[i], end[i]$ ($0 \leq begin[i] \leq end[i] \leq 200000$) 表示小诺询问的段，
以0 0结束

输出描述:

对于每个小诺的询问输出 $(end[i] - begin[i] + 1)$ 行, 表示对应询问段的每个点被多少种颜色的油漆覆盖过。

输入输出样例

输入样例#:

复制

```
1 20
5 10
10 20
0 0
4 6
10 11
0 0
```

输出样例#:

复制

```
1
2
2
3
2
```

思路

noobdream.com

普通方法: 创建一个数组, 对输入的每个区间分别循环, 区间内的所有值加一。此方法容易理解, 但是容易超时

优化方法:

- 1、差分: 只对每个输入的起点和终点进行加减操作
- 2、前缀和: 最后统一只循环一次, 每次迭代当前数目加前一个数的大小, 即为最终的统计数

```
data = []
check = []
while True:
    Bi, Ei = map(int, input().split())
    if Bi == Ei == 0:
        break
    data.append((Ei, Bi))

while True:
    bi, ei = map(int, input().split())
    if bi == ei == 0:
        break
    check += [i for i in range(bi, ei + 1)]

max_index = max(data)[0]
```

```

wall = [0 for i in range(max_index+5)]

# 记录差分,表示Bi到Ei区间存在值
for Ei, Bi in data:
    wall[Bi] += 1
    wall[Ei+1] -= 1

# 遍历一遍,求前缀和
for i in range(1, len(wall)):
    wall[i] += wall[i - 1]

for v in check:
    print(wall[v])

```

进制转换

x进制转10进制

```

a = input()#字符串
print(int(a,x))

```

10进制转x进制

将十进制decimal system转换成二进制binary system

```

print(bin(10))

'{0:b}'.format(10)

```

将十进制decimal system转换成八进制Octal

```

print(oct(10))

'{:o}'.format(10)

```

将十进制decimal system转换成十六进制Hexadecimal

```

print(hex(10))

'{:x}'.format(10)

```

python 逆序

逆序通常方法

```

x[::-1]#x可为string or list

```

reverse用法

```
# reverse 只对list作用, 对string无效
x = ['1', '2', '3']
x.reverse()
x #输出['3', '2', '1']

#reversed返回list逆序的迭代器
[a for a in reversed(x)]#输出['1', '2', '3'], x为list
```

负进制转换方法

方法与二进制一致,

```
def baseNeg2(N):
    res = ''
    while N:
        # N, k = -(N // 2), N % 2
        N, k = -(N >> 1), N % 2 #此处相比正进制多了负号, 因为存在奇偶正负问题
        res = str(k) + res
    return res if res else '0'
```

初始化长度N的字符串

```
print(3*"1")
#"333"
```

datetime类使用

创建日期对象

如需创建日期, 我们可以使用 datetime 模块的 `datetime()` 类 (构造函数)。

`datetime()` 类需要三个参数来创建日期: 年、月、日。

datetime对象间可以相加减, 比较大小。

但对于时间信号, 利用字符串比大小机制也能得出结果:

```
'2007-02-17 19:22:53,315'>'2007-01-27 19:22:53,315'# True
datetime.datetime(2007, 2, 17, 19, 22, 53, 315000)>datetime.datetime(2007, 1,
27, 19, 22, 53, 315000)# True
```

实例

创建日期对象:

```
import datetime

x = datetime.datetime(2020, 5, 17)

print(x)#2020-05-17 00:00:00
```

strftime() 方法

`datetime` 对象拥有把日期对象格式化为可读字符串的方法。

该方法称为 `strftime()`，并使用一个 `format` 参数来指定返回字符串的格式：

实例

显示月份的名称：

```
import datetime

x = datetime.datetime(2019, 10, 1)

print(x.strftime("%B"))#October
```

```
from datetime import datetime

date_string = "1 October, 2020"
print("date_string =", date_string)

date_object = datetime.strptime(date_string, "%d %B, %Y")
print("date_object =", date_object)
```

```
C:\anaconda3.7\python.exe D:/deeplearn/xuexicaog
date_string = 1 October, 2020
date_object = 2020-10-01 00:00:00
```

指令	描述	实例
%a	Weekday, 短版本	Wed
%A	Weekday, 完整版本	Wednesday
%w	Weekday, 数字 0-6, 0 为周日	3
%d	日, 数字 01-31	31
%b	月名称, 短版本	Dec
%B	月名称, 完整版本	December
%m	月, 数字01-12	12
%y	年, 短版本, 无世纪	18
%Y	年, 完整版本	2018
%H	小时, 00-23	17
%I	小时, 00-12	05
%p	AM/PM	PM
%M	分, 00-59	41
%S	秒, 00-59	08
%f	微妙, 000000-999999	548513
%z	UTC 偏移	+0100
%Z	时区	CST
%j	天数, 001-366	365
%U	周数, 每周的第一天是周日, 00-53	52
%W	周数, 每周的第一天是周一, 00-53	52

时间差——datetime.timedelta类

```

from datetime import datetime, date

t1 = date(year = 2018, month = 7, day = 12)
t2 = date(year = 2017, month = 12, day = 23)
t3 = t1 - t2
print("t3 =", t3)# t3 = 201 days, 0:00:00
print(t3.days) # 201

t4 = datetime(year = 2018, month = 7, day = 12, hour = 7, minute = 9, second = 33)
t5 = datetime(year = 2019, month = 6, day = 10, hour = 5, minute = 55, second = 13)
t6 = t4 - t5
print("t6 =", t6)# t6 = -333 days, 1:14:20

```



```
print("type of t3 =", type(t3)) # type of t3 = <class 'datetime.datetime'>
print("type of t6 =", type(t6)) # type of t6 = <class 'datetime.datetime'>

data_a = datetime.datetime(2022,1,1)
data_b = data_a+datetime.timedelta(days=61)# timedelta可用于datetime的计算
print("{:d}-{:02d}-{:02d}".format(data_b.year,data_b.month,data_b.day))# 2022-03-02
```

字符串与datetime类型互转

1.datetime对象转换成字符串

利用str或者strftime方法可以将datetime对象转换成字符串

```
from datetime import datetime # 导入datetime模块
stamp = datetime(2017, 10, 7) # 生成一个datetime对象
str(stamp) # 转换 #结果显示: '2017-10-07 00:00:00'
```

2.字符串转换成datetime对象

datetime.strptime可以用这些格式化编码将字符串转换成日期：

```
from datetime import datetime
value = '2017/10/7'
datetime.strptime(value, '%Y/%m/%d')
#datetime.datetime(2017, 10, 7, 0, 0)
```

字符串只读不写

Python中，字符串是不可变类型，即无法直接修改字符串的某一位字符。

直接修改会报错：'str' object does not support item assignment

```
#报错
#s = '312141'
#s[0] = '0'

#正确方法
s = '312141'
t = list(s)
t[0] = '0'
s = ''.join(t)
```

字典类

字典排序

根据字典的key排序

```
# 单独打印出排序后的key值
new_sys = sorted(sys)
print(new_sys)

new_sys = sorted(sys.keys())
print(new_sys)

#对于set可直接sorted就行
a = {'3', '2', '2', '1', '5'}
print(a)#{'2', '1', '5', '3'}
sorted(a)#[ '1', '2', '3', '5']

# 根据key的升序排列，把key value都打印出来
new_sys1 = sorted(sys.items(), key=lambda d: d[0], reverse=False)
print(new_sys1)

#python sorted自带稳定排序
new_sys1 = sorted(sys.items(), reverse=False)
print(new_sys1)
'''
['age', 'gender', 'name']
['age', 'gender', 'name']
[('age', '十八'), ('gender', 'man'), ('name', '张三')]
[('age', '十八'), ('gender', 'man'), ('name', '张三')]
'''
```

根据字典的value值进行排序

```
# 单独打印出排序后的value值
new_sys1 = sorted(sys.values())
print(new_sys1)

# 打印出根据value排序后的键值对的具体值
new_sys2 = sorted(sys.items(), key=lambda d: d[1], reverse=False)
print(new_sys2)
'''
['man', '十八', '张三']
[('gender', 'man'), ('age', '十八'), ('name', '张三')]
'''
```

元组创建字典

dict.fromkeys

```
#!/usr/bin/python
# -*- coding: UTF-8 -*-

seq = ('Google', 'Runoob', 'Taobao')

# 不指定默认的键值，默认为 None
thisdict = dict.fromkeys(seq)
print ("新字典为 : %s" % str(thisdict))

# 指定默认的键值
thisdict = dict.fromkeys(seq, 10)
print ("新字典为 : %s" % str(thisdict))
```

setdefault使用

用法：给定一组 (key, val) 和一个字典dict，dict.setdefault(key,val)，若dict内含有key，则返回对于key的值，否则加入新的 (key, val)

```
word = 'abc'
for i in word:
    tree = tree.setdefault(i, {})
# tree 值一直为{}
```

字典删除元素

pop() 删除字典给定键key所对应的值, 返回被删除的值

```
dict = {'name': 'jesse', 'age': 18, 'sex': 'male'}
name = dict.pop('name') #
print(name) # jesse
```

set使用

set() 函数创建一个无序不重复元素集，可进行关系测试，删除重复数据，还可以计算交集、差集、并集等。

判断set值是否存在

```
a={1,2,3}
print(4 in a)#False
```

set 集合中添加元素

使用 set 类型提供的 add() 方法实现

使用 add() 方法添加的元素，只能是数字、字符串、元组或者布尔类型 (True 和 False) 值，不能添加列表、字典、集合这类可变的数据，否则 Python 解释器会报 TypeError 错误。例如：

```
a = {1,2,3}
a.add((1,2))
print(a)
a.add([1,2])
print(a)
```

运行结果为：

```
{(1, 2), 1, 2, 3}
Traceback (most recent call last):
  File "C:\Users\mengma\Desktop\1.py", line 4, in <module>
    a.add([1,2])
TypeError: unhashable type: 'list'
```

set集合中删除元素

使用 remove() 方法或 discard() 方法

需要注意的是，使用remove() 如果被删除元素本就不包含在集合中，则此方法会抛出 KeyError 错误，例如：

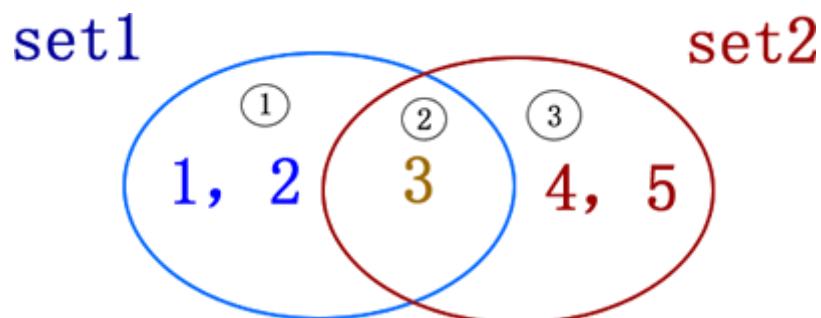
```
a = {1,2,3}
a.remove(1)
print(a)
a.remove(1)
print(a)
#a.discard(1) 此方法不会抛出异常
#print(a)
```

运行结果为：

```
{2, 3}
Traceback (most recent call last):
  File "C:\Users\mengma\Desktop\1.py", line 4, in <module>
    a.remove(1)
KeyError: 1
```

set集合做交集、并集、差集运算

集合最常做的就是进行交集、并集、差集以及对称差集运算。



上图有 2 个集合，分别为 set1={1,2,3} 和 set2={3,4,5}，它们既有相同的元素，也有不同的元素。以这两个集合为例，分别做不同运算的结果如表所示。

运算操作	Python运算符	含义	例子
交集	&	取两集合公共的元素	>>> set1 & set2 {3}
并集		取两集合全部的元素	>>> set1 set2 {1,2,3,4,5}
差集	-	取一个集合中另一集合没有的元素	>>> set1 - set2 {1,2} >>> set2 - set1 {4,5}
对称差集	^	取集合 A 和 B 中不属于 A&B 的元素	>>> set1 ^ set2 {1,2,4,5}

sort 与 sorted 区别:

sort 是应用在 list 上的方法，sorted 可以对所有可迭代的对象进行排序操作。

list 的 sort 方法返回的是对已经存在的列表进行操作，**无返回值**，而内建函数 sorted 方法返回的是一个**新的 list**，而不是在原来的基础上进行的操作。

注意：python 的sort排序为稳定排序，当条件都一致时，按照index的顺序排序

```
>>>a = [5,7,6,3,4,1,2]
>>> b = sorted(a)          # 保留原列表
>>> a
[5, 7, 6, 3, 4, 1, 2]
>>> b
[1, 2, 3, 4, 5, 6, 7]

>>> L=[('b',2),('a',1),('c',3),('d',4)]
>>> sorted(L, cmp=lambda x,y:cmp(x[1],y[1])) # 利用cmp函数 cmp为对比函数
[('a', 1), ('b', 2), ('c', 3), ('d', 4)]
>>> sorted(L, key=lambda x:x[1])             # 利用key
[('a', 1), ('b', 2), ('c', 3), ('d', 4)]

>>> students = [('john', 'A', 15), ('jane', 'B', 12), ('dave', 'B', 10)]
>>> sorted(students, key=lambda s: s[2])      # 按年龄排序
[('dave', 'B', 10), ('jane', 'B', 12), ('john', 'A', 15)]

>>> sorted(students, key=lambda s: s[2], reverse=True) # 按降序
[('john', 'A', 15), ('jane', 'B', 12), ('dave', 'B', 10)]
```

Sorted Key用法

key理解

传递给key参数的是一个函数，它指定可迭代对象中的每一个元素来按照该函数进行排序

```
# 这里先看一个不带key参数的sort()函数, 大家很容易知道结果
li = [[1, 7], [1, 5], [2, 4], [1, 1]]
li.sort()
print(li)
# [[1, 1], [1, 5], [1, 7], [2, 4]] 默认按照0维排序 再按照1维排序

def fun(li):
    return li[1]
# 这时将函数fun传递给参数key 得出结果,也可使用匿名函数 li.sort(key=lambda x:x[1])
li.sort(key=fun)
print(li) # [[1, 1], [2, 4], [1, 5], [1, 7]]
```

多条件排序

多条件排列规定, 当1条件相同时, 按2条件排序

```
#按start从小到大排列, 如果start相同, 则按end从小到大排列。
a = {
    'a': {'start': 10, 'end': 20},
    'd': {'start': 10, 'end': 19},
    'c': {'start': 13, 'end': 20}
}
#若想从大到小排序可以在条件前加- (数字可直接加-, 字符串用reverse=True)
d = sorted(a.keys(), key=lambda x: (a[x]['start'], a[x]['end']))

# d = ['d', 'a', 'c']
```

多条件排序, 不限于两个条件, 如:

```
#按int(x[-1])排序, 如果int(x[-1])相同, 按x[0]排序, 如x[0]相同, 按x[1]排序
res = sorted(data, key=lambda x: (int(x[-1]), x[0], x[1]))
```

cmp理解

sorted的key选择按数据的哪个条件进行排序, cmp则决定排序的规则, 使用时加上key = functools.cmp_to_key (cmp) 参数实现

```
import functools

def cmp_new(x,y):
    if (x+y)>(y+x):
        return 1
    elif (x+y)<(y+x):
        return -1
    else :
        return 0

n=input()
s=input().split()
s.sort(key=cmp_to_key(cmp_new), reverse=True)
print(''.join(s).rstrip("0"))
#或者如下
```

```
s_new = sorted(s, key = functools.cmp_to_key(cmp_new), reserve=True)
print(''.join(s_new).lstrip("0"))
```

例题

对N个长度最长可达到1000的数进行排序。

输入输出格式

输入描述:

输入第一行为一个整数N，(1≤N≤100)。
接下来的N行每行有一个数，数的长度范围为1≤len≤1000。
每个数都是一个正数，并且保证不包含前缀零。

输出描述:

可能有多组测试数据，对于每组数据，将给出的N个数从小到大进行排序，输出排序后的结果，每个数占一行。

输入输出样例

输入样例#:

```
3
11111111111111111111111111111111
22222222222222222222222222222222
33333333
```

复制

输出样例#:

```
33333333
11111111111111111111111111111111
22222222222222222222222222222222
```

复制

```
import functools
#注意要返回具体的1, -1, 0值, 不能返回逻辑值。
def cmp_new(x, y):
    if len(x) == len(y):
        #此次是按从小到大排序
        if x > y:
            return 1
        elif x < y:
            return -1
        else:
            return 0
    #return x<y 不直接写成这样, 要返回具体值如 1, -1, 0
    else:
        if len(x) > len(y):
            return 1
        elif len(x) < len(y):
            return -1

while True:
    try:
        n = int(input())
        data = []
        for i in range(n):
            data.append(input())
        res = sorted(data, key=functools.cmp_to_key(cmp_new))
        for v in res:
            print(v)
```

```
except:
    break
```

队列

定义

队列类似于一条管道，元素先进先出,进`put(arg)`，取`get()`。需要注意的是：队列都是在内存中操作，进程退出，队列清空，另外，队列也是一个阻塞的形态

方法

方法	用法说明
put	放数据，Queue.put()默认有block=True和timeout两个参数。当block=True时，写入是阻塞式的，阻塞时间由timeout确定。当队列q被（其他线程）写满后，这段代码就会阻塞，直至其他线程取走数据。Queue.put ()方法加上 block=False 的参数，即可解决这个隐蔽的问题。但要注意，非阻塞方式写队列，当队列满时会抛出 exception Queue.Full 的异常
get	取数据(默认阻塞),Queue.get([block[, timeout]])获取队列，timeout等待时间
empty	如果队列为空，返回True,反之False
qsize	显示队列中真实存在的元素长度
maxsize	最大支持的队列长度,使用时无括号
join	实际上意味着等到队列为空，再执行别的操作
task_done	在完成一项工作之后，Queue.task_done()函数向任务已经完成的队列发送一个信号
full	如果队列满了，返回True,反之False

队列分类

队列有很多种，但都依赖模块queue

队列方式	特点
queue.Queue	先进先出队列
queue.LifoQueue	后进先出队列
queue.PriorityQueue	优先级队列
queue.deque	双线队列

单向队列

```
import queue

#q=queue.Queue(5)    #如果不设置长度,默认为无限长
#print(q.maxsize)    #注意没有括号
from queue import Queue
# FIFO
queue_obj = Queue()  # 创建一个队列对象
for i in range(4):
    queue_obj.put(i)
while not queue_obj.empty():
    print(queue_obj.get())
```



```
# 输出顺序
0
1
2
3
```

后进先出队列

```
q = queue.LifoQueue()
q.put(12)
q.put(34)
print(q.get())
-----
34
```

优先级队列

需要注意的是，优先级队列put的可以是单个值，可以是一个**元组**，(优先级,数据)，优先级数越小，级别越高

```
q = queue.PriorityQueue()
q.put((3, 'aaaaa'))
q.put((3, 'bbbbbb'))
q.put((1, 'cccc'))
q.put((3, 'dddd'))
print(q.get())
print(q.get())
-----
(1, 'cccc')
(3, 'aaaaa')
```

N 佬
noobdream.com

当优先级一样，数据部分可以比较大小的时候，也可以排序

```
priorityQueue_obj = queue.PriorityQueue()
priorityQueue_obj.put((1,45))
priorityQueue_obj.put((1,42))
priorityQueue_obj.put((1,47))
while not PriorityQueue_obj.empty():
    print(PriorityQueue_obj.get())

(1, 42)
(1, 45)
(1, 47)

priorityQueue_obj = PriorityQueue()
priorityQueue_obj.put((1,[1,4]))
priorityQueue_obj.put((1,[2,4]))
priorityQueue_obj.put((1,[2,3]))
while not PriorityQueue_obj.empty():
    print(PriorityQueue_obj.get())
```

```
(1, [1, 4])
(1, [2, 3])
(1, [2, 4])
```

#当优先级一样的时候，会在比较数据部分的大小，同上字符串也可以比较大小，

优先级一样，数据部分不可以比较大小时，程序会报错

```
priorityQueue_obj = queue.PriorityQueue()
priorityQueue_obj.put((1,{"1":9}))
priorityQueue_obj.put((1,{"k":6}))
priorityQueue_obj.put((1,{"8":9}))
while not priorityQueue_obj.empty():
    print(priorityQueue_obj.get()) # 没有字典不能直接比较大小

# 报错内容
# TypeError: '<' not supported between instances of 'dict' and 'dict'
```

双线队列

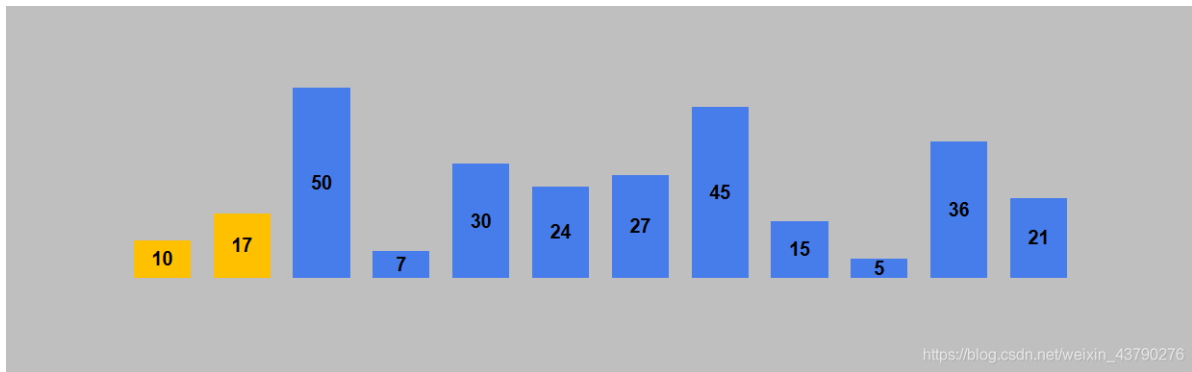
```
from collections import deque
q = deque()
q.append(123)
q.append(456)
q.appendleft(780)
print(q)
print(q.pop())#移除右边的元素，对应的append也不变
print(q.popleft())#移除左边的元素，对应的append为appendleft
-----
deque([780, 123, 456])
456
780
```

各类排序

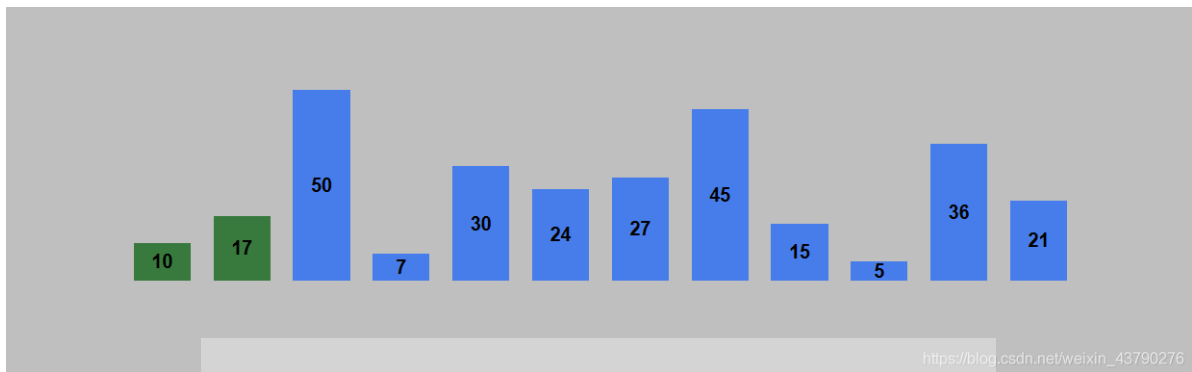
插入排序

效率(N^2)

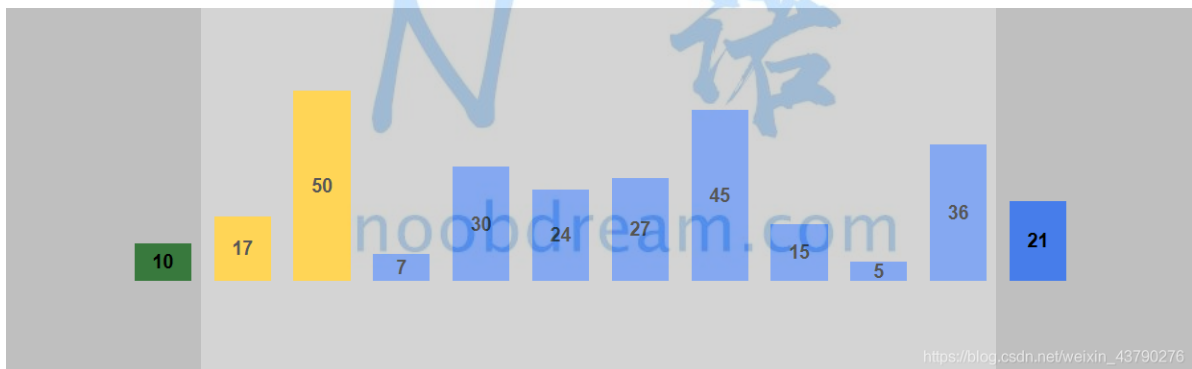
1. 将第一个数据当成已排序序列，后面的数据当成未排序序列。取未排序的第一个数据与已排序的最后一个数据进行比较，如果顺序错误则交换位置。17(未排序的第一个数据)比10(已排序的最后一个数据)大，不需要进行交换。



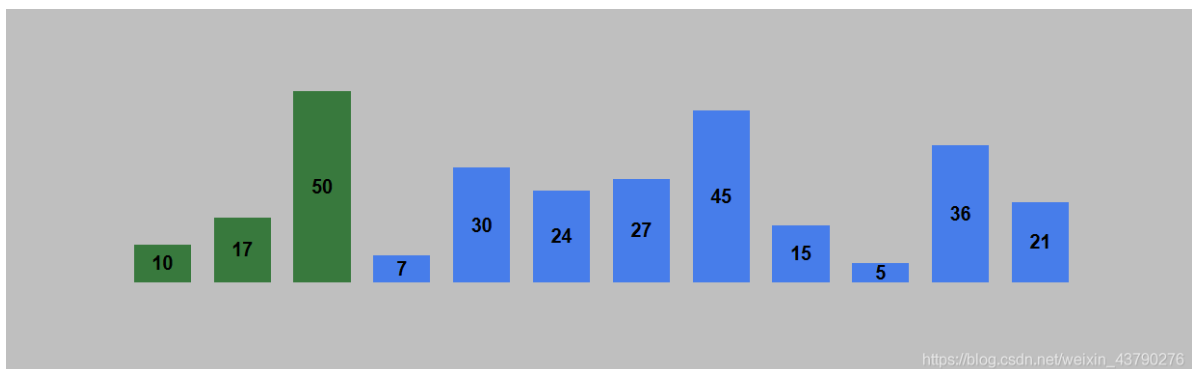
2. 当不需要交换时则此轮插入完成。已排序序列变成了两个数据，未排序序列数据减1。



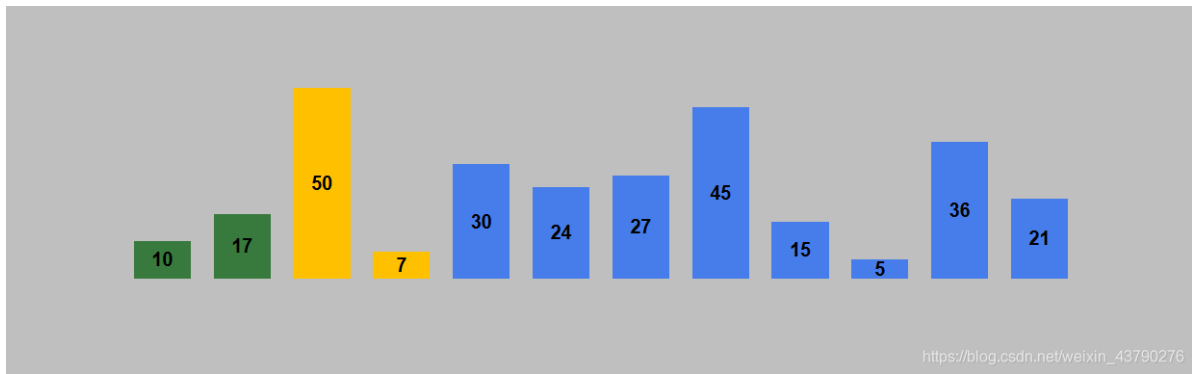
3. 继续取未排序的第一个数据与已排序的最后一个数据进行比较。如果顺序错误则交换位置。50比17大，不需要交换。



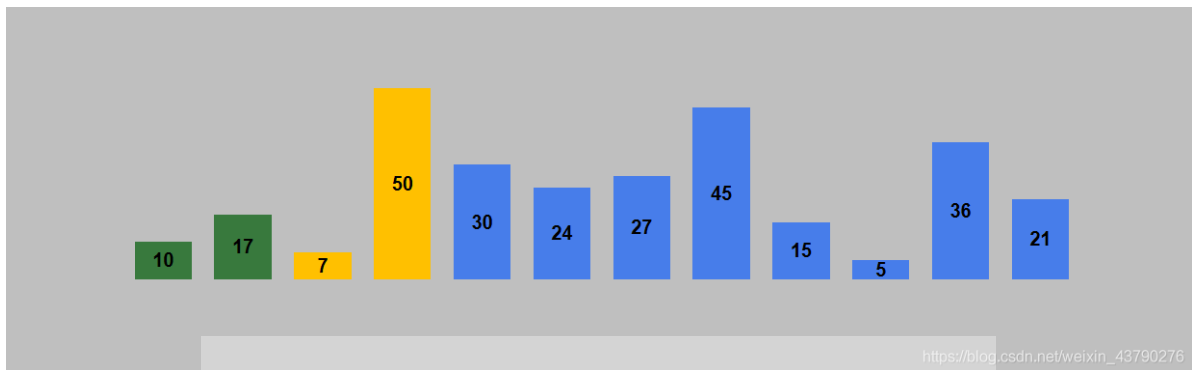
4. 第二轮插入完成，已排序序列数量加1，未排序序列减1。



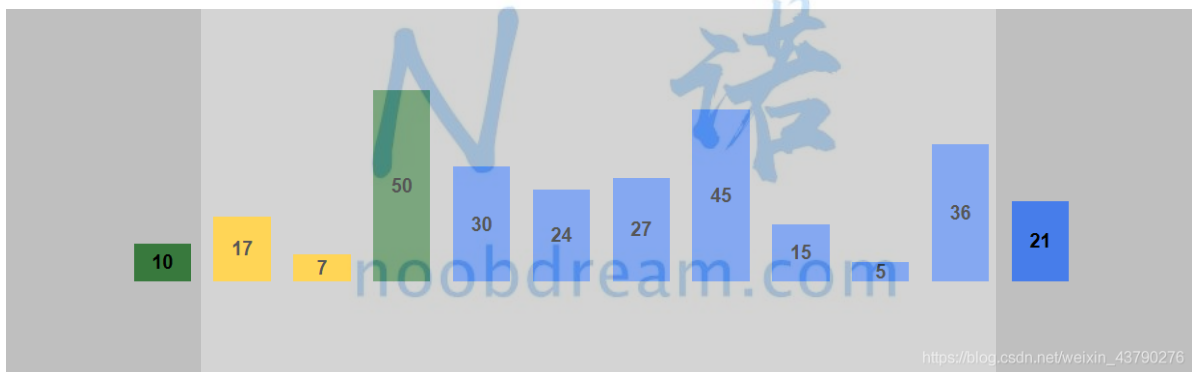
5. 重复上面的步骤，未排序的第一个数据与已排序的最后一个数据进行比较。



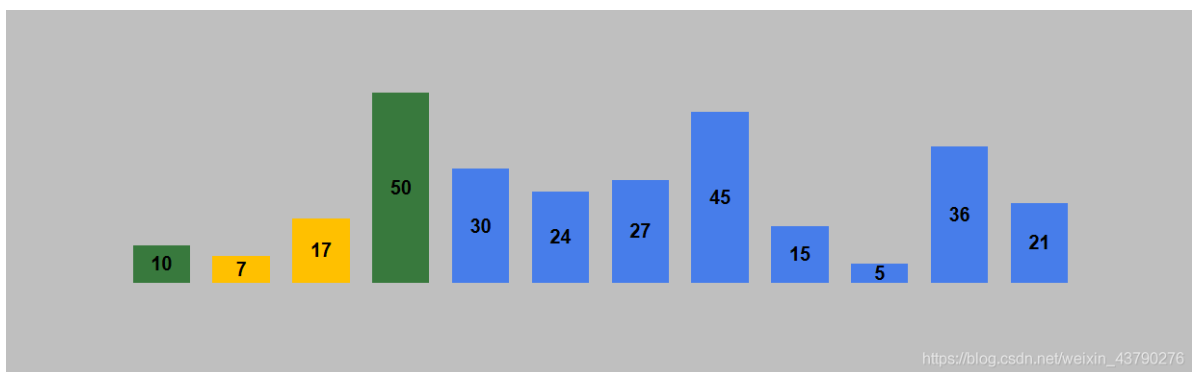
6. 如果位置错误则交换位置。 7比50小，交换位置。



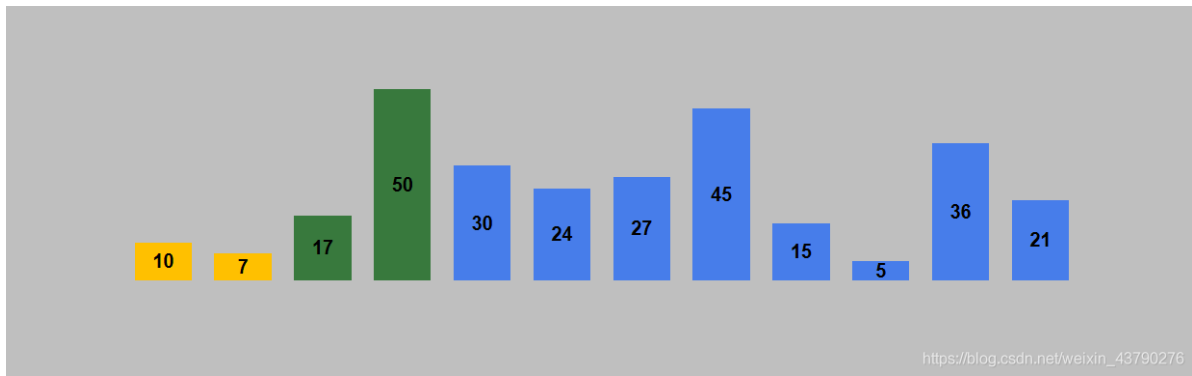
7. 交换位置后，继续将插入的数据与相邻的前一个数据进行比较。



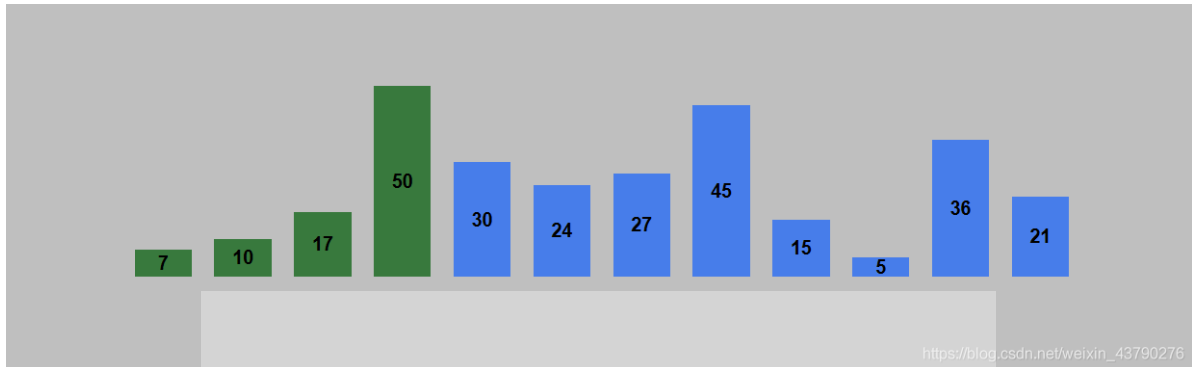
8. 如果位置错误则交换位置。 7比17小，交换位置。



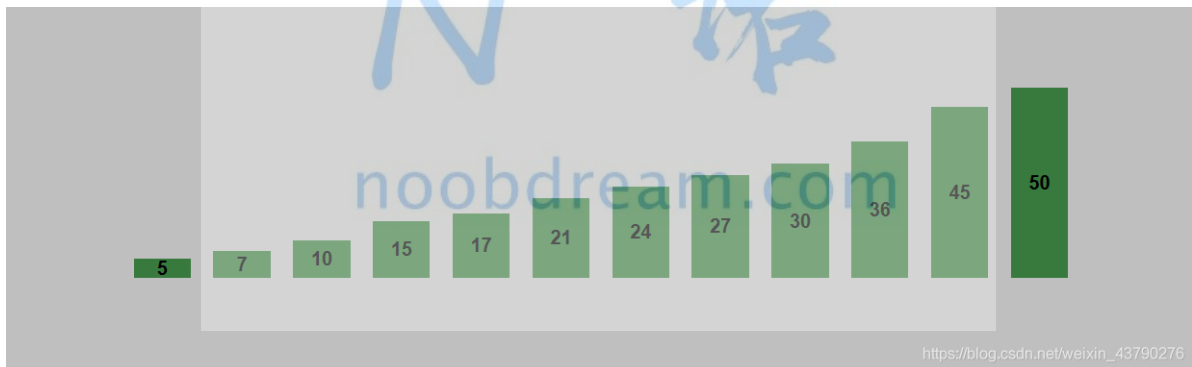
9. 重复步骤7，继续将插入的数据与相邻的前一个数据进行比较。



10. 如果位置错误则交换位置。7比10小，交换位置。此时插入的数据已经通过交换位置到达了数列的最前端，不需要再次交换了，此轮插入完成。



11. 一直重复取未排序的第一个数据插入已排序序列中，直到所有数据都已经插入到已排序序列中，列表排序完成。排序结果如下图。



代码

```
# coding=utf-8
def insertion_sort(array):
    for i in range(len(array)):
        cur_index = i
        while array[cur_index-1] > array[cur_index] and cur_index-1 >= 0:
            array[cur_index], array[cur_index-1] = array[cur_index-1], array[cur_index]
            #对新来数字在前已排序的序列中循环排序
            cur_index -= 1
    return array

if __name__ == '__main__':
    array = [10, 17, 50, 7, 30, 24, 27, 45, 15, 5, 36, 21]
    print(insertion_sort(array))
```

希尔排序

希尔排序的具体思路步骤：(n等于列表的长度) **效率(最差 N^2 ，最好 N)**

- 1) 首先取一个整数 $d_1 = n // 2$ ，将元素分为 d_1 个组，每组相邻元素之间的距离为 d_1 ，在各组内进行插入排序
- 2) 取第二个整数 $d_2 = d_1 // 2$ ，重复上述分组排序，则到 $d_i = 1$ 时，即所有元素在同一组内进行插入排序即可完成排序
- 3) 希尔排序每趟并不是使某些元素有序，而是使整体数据越来越接近有序；最后一趟排序使得所有数据有序

这里取列表 [5, 7, 4, 6, 3, 1, 2, 9, 8]

第一次分组 $d_1 = 9 // 2 = 4$ ，分成4组，依次对[5, 3, 8], [7, 1], [4, 2], [6, 9] 进行插入排序

5	7	4	6	3	1	2	9	8
5				3				8
	7				1			
		4				2		
			6				9	

第一次分组插入排序后：[3, 1, 2, 6, 5, 7, 4, 9, 8]

第二次分组 $d_2 = d_1 // 2 = 2$ ，依次对[3, 2, 5, 4, 8]、[1, 6, 7, 9]进行插入排序

3	1	2	6	5	7	4	9	8
3		2		5		4		8
	1		6		7		9	

第二次分组插入排序后：[2, 1, 3, 6, 4, 7, 5, 9, 8]

第三次分组 $d_3 = d_2 // 2 = 1$ 则对 [2, 1, 3, 6, 4, 7, 5, 9, 8] 进行插入排序，排序后[1, 2, 3, 4, 5, 6, 7, 8, 9]

代码

```
def insert_sort_gap(li, gap): # gap 代表分成几组
    for i in range(gap, len(li)):
        tmp = li[i] # 第一组中下一个值
        k = i - gap # 第一组中前一个值
        # 插入排序
        while k >= 0 and li[k] > tmp:
            li[k + gap], li[k] = li[k], li[k + gap]
            k -= gap
        li[k + gap] = tmp
    print(li)

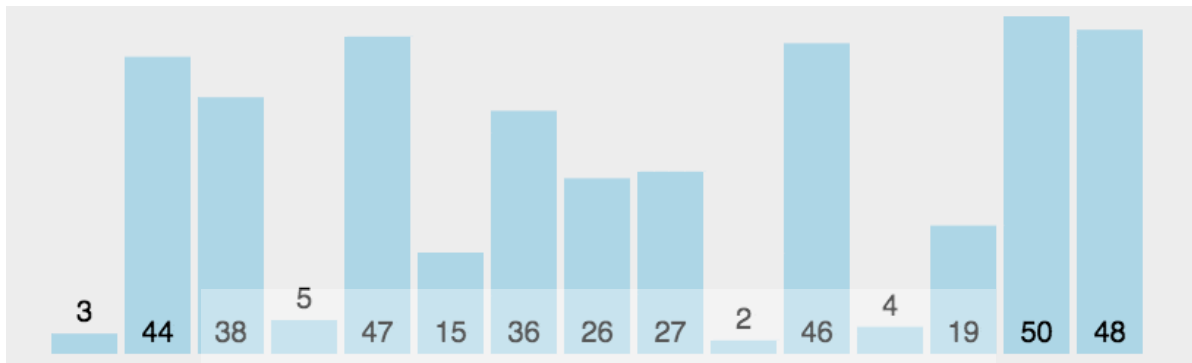
def shell_sort(li):
    d = len(li) // 2
    while d >= 1:
        insert_sort_gap(li, d)
        d = d // 2
```

```
d //= 2
```

```
li = [5, 7, 4, 6, 3, 1, 2, 9, 8]
shell_sort(li)
print(li)
```

直接选择排序

最简单的排序，原理：在未排序序列中找到最小（大）元素，存放到排序序列的起始位置，然后，再从剩余未排序元素中继续寻找最小（大）元素，然后放到已排序序列的末尾。以此类推，直到所有元素均排序完毕。效率(N^2)



代码

```
def simpleSelect(a):
    # 简单选择排序：小->大
    for i in range(0, len(a) - 1):
        min_index = i
        # 找到i~len(a)的最小值的坐标
        for j in range(i, len(a)):
            if a[min_index] > a[j]:
                min_index = j
        a[i], a[min_index] = a[min_index], a[i]
    return a

print simpleSelect([11, 1, 6, 9, 8, 5])
```

快速排序

采用“分而治之”的思想，把大的拆分为小的，小的拆分为更小的。其原理是，对于给定的记录，选择一个基准数，通过一趟排序后，将原序列分为两部分，使得前面的比后面的小，然后再依次对前后进行拆分进行快速排序，递归该过程，直到序列中所有记录均有序。

步骤

设当前待排序序列为 $R[\text{low}:\text{high}]$ ，其中 $\text{low} \leq \text{high}$ ，如果待排序的序列规模足够小，则直接进行排序，否则分3步处理。

1、分解

在 $R[\text{low}:\text{high}]$ 中选定一个元素 $R[\text{pivot}]$ ，以此为标准将要排序的序列划分为两个序列 $R[\text{low}:\text{pivot}-1]$ 与 $R[\text{pivot}+1:\text{high}]$ ，并使序列 $R[\text{low}:\text{pivot}-1]$ 中所有元素的值小于等于 $R[\text{pivot}]$ ，序列 $R[\text{pivot}+1:\text{high}]$ 所有的值大于 $R[\text{pivot}]$ ，此时基准元素以位于正确位置，它无需参加后续排序。

2、递归

对于子序列 $R[\text{low}:\text{pivot}-1]$ 与 $R[\text{pivot}+1:\text{high}]$ ，分别调用快速排序算法来进行排序。

3、合并

由于对序列 $R[\text{low}:\text{pivot}-1]$ 与 $R[\text{pivot}+1:\text{high}]$ 的排序是原地进行的，所以 $R[\text{low}:\text{pivot}-1]$ 与 $R[\text{pivot}+1:\text{high}]$ 都已经排好序后，不需要进行任何计算，就已经排好序。

注：基准元素，一般来说选取有几种方法

- 取第一个元素
- 取最后一个元素
- 取第中间位置元素
- 取第一个、最后一个、中间位置3者的中位数元素

图解

假设当前排序为 $R[\text{low}:\text{high}]$ ，其中 $\text{low} \leq \text{high}$ 。

1：首先取序列第一个元素为基准元素 $\text{pivot}=R[\text{low}]$ 。 $i=\text{low}, j=\text{high}$ 。2：从后向前扫描，找小于等于 pivot 的数，如果找到， $R[i]$ 与 $R[j]$ 交换， $i++$ 。3：从前往后扫描，找大于 pivot 的数，如果找到， $R[i]$ 与 $R[j]$ 交换， $j--$ 。4：重复2~3，直到 $i=j$ ，返回该位置 $\text{mid}=i$ ，该位置正好为 pivot 元素。完成一趟排序后，以 mid 为界，将序列分为两部分，左序列都比 pivot 小，右序列都比 pivot 大，然后再分别对这两个子序列进行快速排序。

以序列 (30, 24, 5, 58, 18, 36, 12, 42, 39) 为例，进行演示

1、初始化， $i=\text{low}, j=\text{high}, \text{pivot}=R[\text{low}]=30$

i							j
30	24	5	58	18	36	12	42

2、从后往前找小于等于 pivot 的数，找到 $R[j]=12$

i						j	
30	24	5	58	18	36	12	42

$R[i]$ 与 $R[j]$ 交换， $i++$

	i					j	
12	24	5	58	18	36	30	42

3、从前往后找大于 pivot 的数，找到 $R[i]=58$

			i			j		
12	24	5	58	18	36	30	42	39

R[i]与R[j]交换, j--

			i			j		
12	24	5	30	18	36	58	42	39

4、从后往前找小于等于pivot的数, 找到R[j]=18

			i	j				
12	24	5	30	18	36	58	42	39

R[i]与R[j]交换, i++

			i	j				
12	24	5	18	30	36	58	42	39

5、从前往后找大于pivot的数, 这时i=j, 第一轮排序结束, 返回i的位置, mid=i

low			mid-1	mid	mid+1			high
12	24	5	18	30	36	58	42	39

此时已mid为界, 将原序列一分为二, 左子序列为 (12, 24, 5, 18) 元素都比pivot小, 右子序列为 (36, 58, 42, 39) 元素都比pivot大。然后在分别对两个子序列进行快速排序, 最后即可得到排序后的结果。

- 平均时间复杂度为: $O(n\log n)$
- 平均空间复杂度为: $O(\log n)$

代码

```
def quickSort(list, i, j):
    if i >= j:
        return list
    pivot = list[i]
    # 保持上下界
    low = i
    high = j
```

```

# 寻找mid
while i < j:
    # 右值
    while i < j and list[j] > pivot:
        j -= 1
    # 在右值找到小于pivot的值, 交换基准位置
    list[j], list[i] = list[i], list[j]
    # 注意要判断i, j不越界才能加减
    if i < j: i += 1
    # 左值
    while i < j and list[i] < pivot:
        i += 1
    # 在左值找到大于pivot的值, 交换基准位置
    list[j], list[i] = list[i], list[j]
    # 注意要判断i, j不越界才能加减
    if i < j: j -= 1
mid = i
quickSort(list, low, mid - 1)
quickSort(list, mid + 1, high)
return list

if __name__=="__main__":
    lists=[30,24,5,58,18,36,12,42,39]
    print("排序前的序列为: ")
    for i in lists:
        print(i,end=" ")
    print("\n排序后的序列为: ")
    for i in quick_sort(lists,0,len(lists)-1):
        print(i,end=" ")

>>>
排序前的序列为:
30 24 5 58 18 36 12 42 39
排序后的序列为:
5 12 18 24 30 36 39 42 58

#简单快排
def quick(array):
    if array==[]:
        return []
    else:
        afirst = array[0]
        #无low, high交互, 直接找出大于与小于基准的集合
        aless = quick([l for l in array[1:] if l < afirst])
        amore = quick([m for m in array[1:] if m >= afirst])
        return aless + [afirst] + amore

```

二路归并排序算法

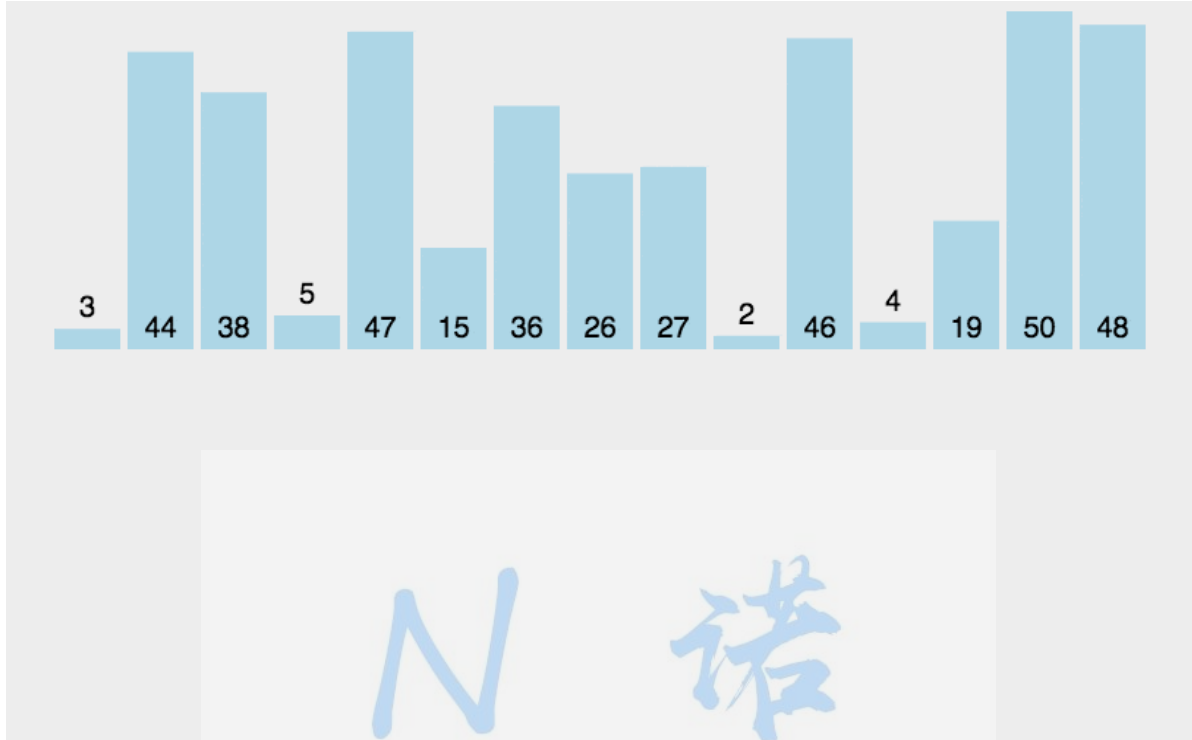
稳定排序, 时间复杂度 $O(N\log(N))$, 空间复杂度 $O(N)$

归并含义是将两个或两个以上的**有序表**组合成一个新的有序表。无论是顺序存储结构还是链表存储结构, 都可在 $O(m+n)$ 的时间量级上实现。

合并两个有序数组过程:

例如 $a = [1, 2]$, $b = [0, 3, 4]$ 两个有序数组和一个空数组 $res = []$, 设置两个指针, i 指向 a 数组第一个元素, j 指向 b 数组第一个元素。首先比较这两个元素 $1 < 0$, 将0添加到 res 数组[0]; 然后让 j 指针递增指向 3 元素, 此时比较 i 和 j 指向元素 对比 $1 < 3$, 将 1 添加到 res 数组[0, 1]; 让 i 指针递增指向2 元素, 此时比较 i 和 j 指向元素 对比 $2 < 3$, 将2添加到 res 数组[0, 1, 2]。这个时候 $i = len(a)$ 已经把 a 数组元素添加完, 还剩 b 数组中3 和 4元素, 直接把剩下 b 数组元素添加到 res [0, 1, 2 ,3 ,4], 这样就实现了合并两个有序数组为一个有序数组。

如此递归的分割排序后, 再合并



代码

```
def MergeSort(lists):
    if len(lists) <=1:
        return lists
    num = len(lists)//2
    left = MergeSort(lists[:num])
    right = MergeSort(lists[num:])
    return Merge(left,right)

def Merge(left,right):
    r,l=0,0
    reslut=[]
    while l<len(left) and r<len(right):
        if left[l] < right[r]:
            reslut.append(left[l])
            l+=1
        else:
            reslut.append(right[r])
            r+=1
    reslut+= right[r:]
    reslut+= left[l:]
    return reslut

if __name__ == '__main__':
```

```
arr = [4,2,15,4,6,7,1]
print(MergeSort(arr))
```

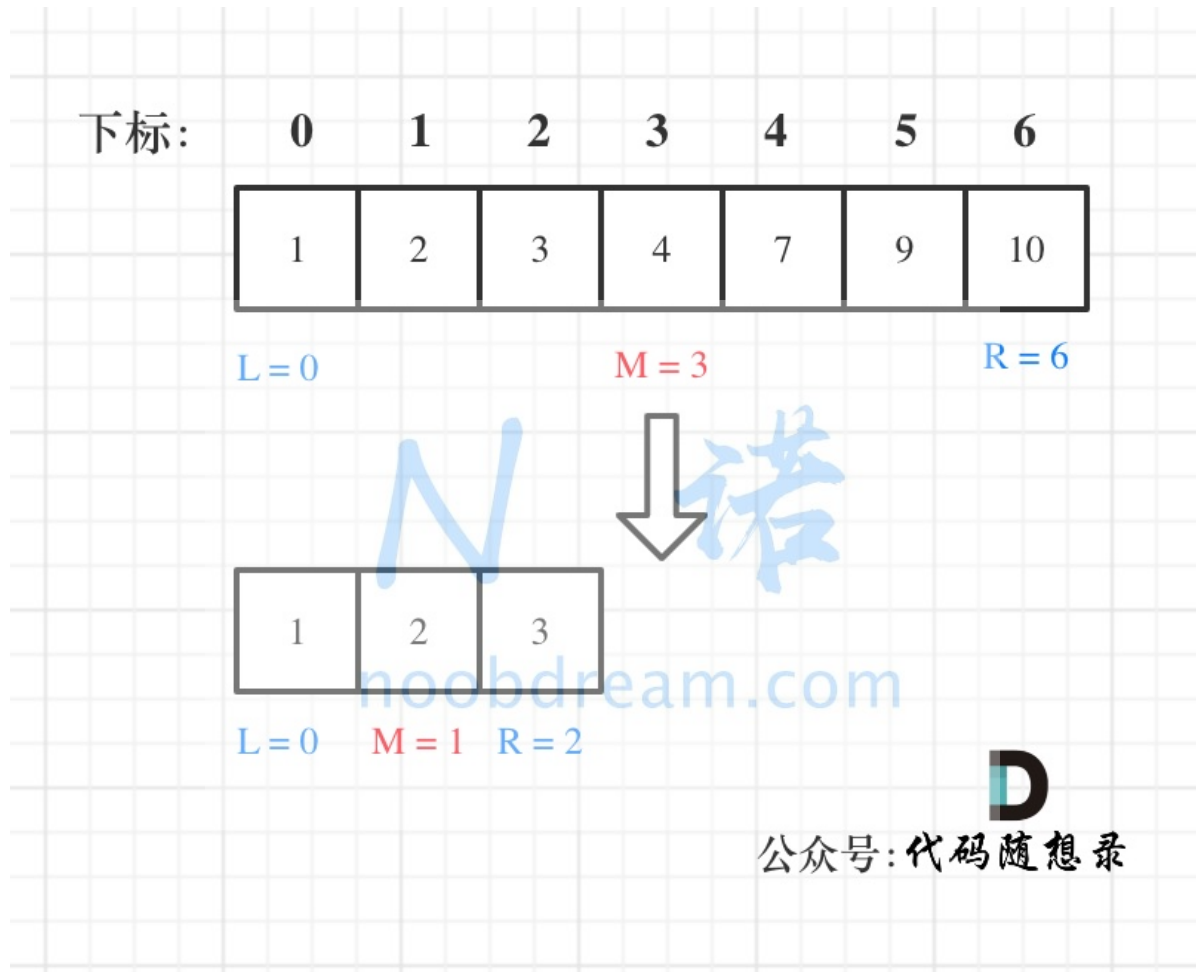
各类查找

二分法

在写程序前，先想清楚如何定义边界：

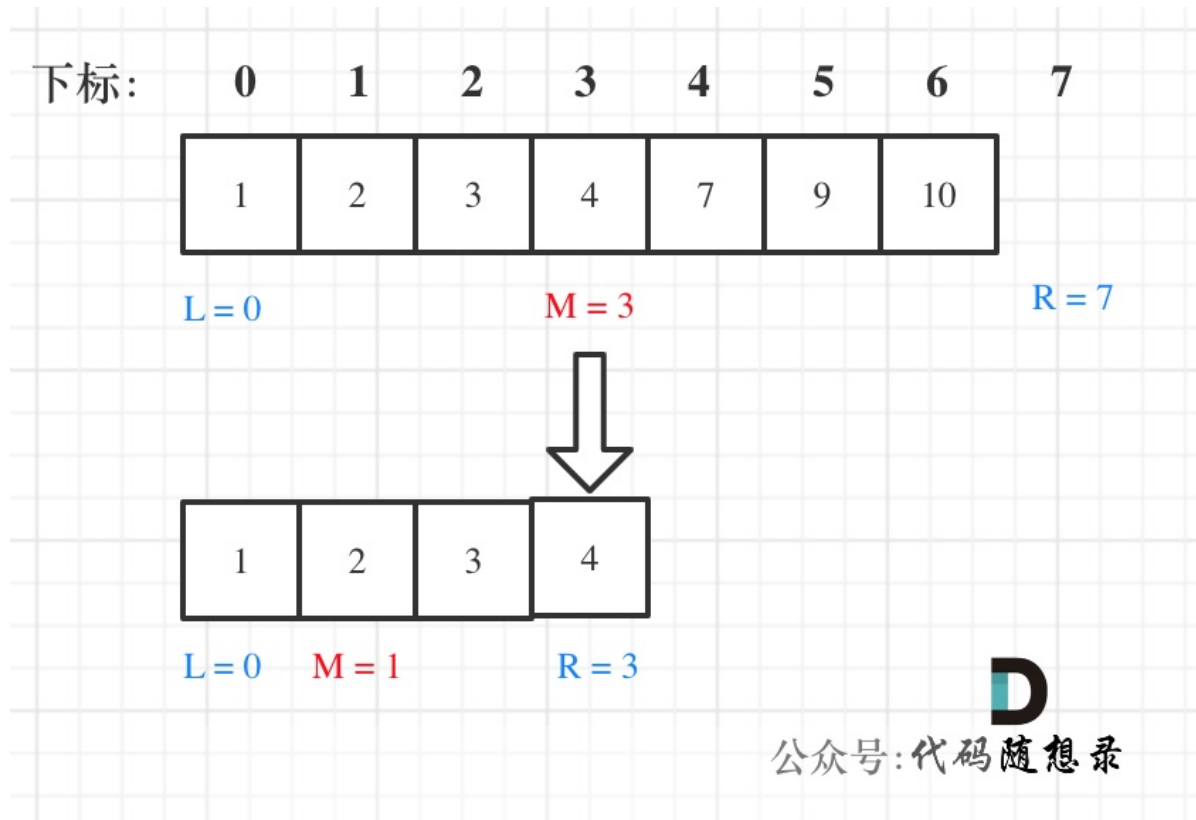
两种定义边界方式（在每次递归后，用规划好的闭合区间进行推算。）：

1、左闭右闭：



```
# 左闭右闭
def search(li, target):
    h = len(li) - 1
    l = 0
    #因为都是闭区间，h和l都可以取到并且相等
    while h >= l:
        mid = l + (h - l) // 2
        if li[mid] == target:
            return mid
        elif li[mid] > target:
            h = mid - 1
        else:
            l = mid + 1
    return -1
```

2、左闭右开：



```
#左闭右开
def search(li, target):
    h = len(li)
    l = 0
    #因为右是开区间，h和l不可能相等，所以是开区间
    while h > l:
        mid = l + (h - l) // 2
        if li[mid] == target:
            return mid
        elif li[mid] > target:
            h = mid
        else:
            l = mid + 1
    return -1
```

最大公约数

按部就班法

从两数的最小值开始遍历寻找最大公约数

```
for i in range(min(a,b), 0, -1): #从两数中小的一个开始到1，一个个验证
    if a % i == 0 and b % i == 0: #如果能够被两数同时整除
        print(i) #输出该数后不再循环
        break
```

辗转相除法

辗转相除法又叫欧几里得算法,是欧几里得最先提出来的.辗转相除法的实现,是基于下面的原理(在这里用(a,b)表示a和b的最大公因数):

$(a,b)=(a,ka+b)$,其中a、b、k都为自然数

也就是说,两个数的最大公约数,将其中一个数加到另一个数上,得到的新数,其公约数不变。

基于上面的原理,就能实现我们的迭代相减法:

$(78,14)=(64,14)=(50,14)=(36,14)=(22,14)=(8,14)=(8,6)=(2,6)=(2,4)=(2,2)=(0,2)=2$

即 $78\%14=8 \rightarrow (8,14) \rightarrow 14\%8=6 \rightarrow (8,6) \rightarrow 8\%6=2 \rightarrow (2,6) \rightarrow 6\%2=0 \rightarrow (0,2)=2$

```
if a<b: #如果a<b, 则交换两数位置, 否则不交换
    a,b = b,a
r = a % b #求a/b的余数
while r != 0: #在余数不为零时, 始终进行交换和相除
    a,b = b,r
    r = a % b
print(b) #余数为零后, 打印输出b
```

改进

非递归法

```
while b:
    a,b=b,a%b
print(a)
```

递归法

```
def gcd(x, y):
    if y == 0:
        return x
    else:
        return gcd(y, x%y)

gcd(a,b)
```

最小公倍数

对于求两个数的最小公倍数, 只需要记住下面这个公式即可。

$$\text{LCM}(x, y) = x * y / \text{GCD}(x, y)$$

翻译一下就是: 两个数的最小公倍数等于两个数的乘积除以两个数的最大公约数。

所以要求两个数的最小公倍数, 我们只需求出他们的最大公约数即可。

```
def gcd(x , y):
    if y == 0:
        return x
    else:
        return gcd(y, x%y)

LCM = int(a*b/gcd(a,b))
```

素数/质数

只能被1或自己整除的数，注意0，1都是非素数

判断素数

普通方法

针对输入的数字 x ，我们可以遍历从2到 $x-1$ 这个区间中的数，如果 x 能被这个区间中任意一个数整除，那么它就不是质数。

```
def is_prime1(x):
    for i in range(2, x):
        if num % i == 0:
            return False
    return True
```

优化

对算法一的优化，事实上只需要遍历从2到 $\sqrt{x}$ 即可。

```
def is_prime2(x):
    for i in range(2, int(x ** 0.5) + 1):
        if x % i == 0:
            return False
    return True
```

再优化

任何一个自然数，总可以表示成以下六种形式之一： $6n$ ， $6n+1$ ， $6n+2$ ， $6n+3$ ， $6n+4$ ， $6n+5$ ($n=0,1,2,\dots$) 我们可以发现，除了2和3，只有形如 $6n+1$ 和 $6n+5$ 的数有可能是质数。且形如 $6n+1$ 和 $6n+5$ 的数如果不是质数，一定是它们的因数含有形如 $6n+1$ 或者 $6n+5$ 的数，如 $7=6*1+1$ 是素数， $49=6*8+1$ 非素数，因为含有因数7。

```
def is_prime4(x):
    if (x == 2) or (x == 3):
        return True
    if (x % 6 != 1) and (x % 6 != 5):
        return False
    for i in range(5, int(x ** 0.5) + 1, 6):
        # x % i == 0意为%(6n+5), x % (i + 2) == 0意为%(6n+1)
        if (x % i == 0) or (x % (i + 2) == 0):
            return False
    return True
```

筛选素数

埃氏筛法的算法原理：

1. 从2开始造一个自然数序列：2, 3, 4, 5, 6, 7.....
2. 取第一个数2，2一定是一个素数，然后用2把序列中2的倍数筛掉；
3. 取新序列中的第一个数3，用3把序列中3的倍数筛掉；
4. 取新序列第一个数5，把5的倍数全部筛掉；
5. 不断筛下去，就可以得到所有的素数；

```
def sieve_of_eratosthenes(n):#埃拉托色尼筛选法，返回< n的素数
    primes = [True] * (n+1)#范围0到n的列表
    p = 2#这是最小的素数
    while p * p <= n:#一直筛到sqrt(n)就行了
        if primes[p]:#如果没被筛，一定是素数
            for i in range(p * 2, n + 1, p):#筛掉它的倍数即可
                primes[i] = False
            p += 1
    primes = [element for element in range(2, n) if primes[element]]#得到所有少于n
    的素数
    #primes = [element for element in range(2, n+1) if primes[element]]#得到所有少
    于等于n的素数
    return primes
print(sieve_of_eratosthenes(100))
```

幂次方 mod方法

pow函数用法

在python中内置函数pow()一共有两个参数，x和y，并返回 xy（x的y次方）的值，语法如下：

```
pow(x, y[, z])
#用法 pow(2, 3, 5)=2**3%5
```

参数介绍：

x — 数值表达式(整数或者浮点数)；

y — 数值表达式(整数或者浮点数)；

z — 数值表达式(整数或者浮点数)，默认不设置z值；

返回值：返回 xy(x的y次方)的值；如果设置了z值,则再对结果进行取模,其结果等效于pow(x,y) %z；

二分快速幂

求 $(x^y) \% p$ 当 y 很大的时候，我们不能直接用 for 去一个一个的乘，因为这样的方法复杂度是 $O(N)$

任意一个 x^y ，我们都可以将 y 分解成 2 的幂次的形式。例如 $5^{13} = 5^8 * 5^4 * 5^1$

代码：

```
def mod_pow(x,y,mod):
    res=1
    while y>0:
        # 检测到最低位为1，以5^13为例，当前的结果则为(5^8)%mod*(5^4)%mod*(5^1)%mod
        if y&1:
            res=(res*x)%mod
        x=(x*x)%mod
        # 向前移位
        y>>=1
    return res
```

匹配问题

基本思路——栈

匹配是典型的栈的应用，python的栈直接用列表模拟即可

普通匹配问题

Memory Limit: 256 mb

假设表达式中允许包含两种括号:圆括号和方括号。编写一个算法判断表达式中的括号是否正确配对。

输入输出格式

输入描述:

由括号构成的字符串，包含“(”、“)”、“[”和“]”。

#此处可以不构建stack类，直接在数组上操作就行

```
class Stack:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def push(self, x):
        self.items.append(x)

    def pop(self):
        return self.items.pop()
```

```

def peek(self):
    return self.items[-1]

def size(self):
    return len(self.items)

match_map = {'[': ']', '(': ')'}
data = Stack()
line = list(input())
for v in line:
    if data.size() == 0:
        data.push(v)
        continue
    if data.peek() in match_map.keys() and match_map[data.peek()] == v:
        data.pop()
    else:
        data.push(v)
if data.size() == 0:
    print('YES')
else:
    print('NO')

```

条件匹配的问题

可以在stack加入元组元素，元组中带有匹配条件

问题一

括号的匹配

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

题意描述：在算术表达式中，除了加、减、乘、除等运算外，往往还有括号。包括有大括号{}，中括号[]，小括号()，尖括号<>等。对于每一对括号，必须先左边括号，然后右边括号；如果有多个括号，则每种类型的左括号和右括号的个数必须相等；对于多重括号的情形，按运算规则，从外到内的括号嵌套顺序为：大括号->中括号->小括号->尖括号。例如，{[()]}, {()},{[]},{<>}为一个合法的表达式，而{([)]}, {[()]},{<>}}都是非法的。

输入输出格式

输入输出样例

输入样例#:

复制

```
5
{ [( < > ) ] }
[ ( ) ]
< > ( ) [ ] { }
[ { } ]
{ ( ) }
```

输出样例#:

复制

```
YES
YES
YES
NO
YES
```

问题二

括号匹配问题

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

在某个字符串（长度不超过100）中有左括号、右括号和大小写字母；规定（与常见的算数式子一样）任何一个左括号都从内到外与在它右边且距离最近的右括号匹配。写一个程序，找到无法匹配的左括号和右括号，输出原来字符串，并在下一行标出不能匹配的括号。不能匹配的左括号用"\$"标注,不能匹配的右括号用"?"标注。

noobdream.com

输入输出样例

输入样例#:

复制

```
bge) ) ) ) ) ) ) )
((IIII) ) ) ) )
() () () (uuu
) ) ) UUUU ( ( )
```

输出样例#:

复制

```
bge) ) ) ) ) ) ) )
      ?????????
((IIII) ) ) ) )
      ????
() () () (uuu
      $
) ) ) UUUU ( ( )
????    $$
```

```
while True:
    try:
        line = input()
        s = []
        signs = line
        for i, v in enumerate(line):
            if v == '(' or v == ')':
```

```

        if len(s) == 0:
            # 加入元组 (括号, index)
            s.append((v, i))
            continue
        k = s[-1][0]
        # 判断匹配
        if k == '(' and v == ')':
            s.pop()
            continue
        else:
            s.append((v, i))

    while len(s) != 0:
        v, i = s.pop()
        if v == '(':
            signs = signs[:i]+'$'+signs[i+1:]
        elif v == ')':
            signs = signs[:i]+'?'+signs[i+1:]

    for i in range(len(signs)):
        if signs[i] != '?' and signs[i] != '$':
            signs = signs[:i]+' '+signs[i+1:]
    print(line)
    print(signs)

except:
    break

```

哈希算法

原理

使用map统计个数的方法，也用于减少时间复杂度

三数/四数之和问题

如下题所示

abc

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 32768 mb

设a、b、c均是0到9之间的数字，abc、bcc是两个三位数，且有： $abc+bcc=532$ 。求满足条件的所有a、b、c的值。

输入输出格式

输入描述:

题目没有任何输入

该题用暴力解法，对a，b，c循环，需要 $O(n^3)$ 的复杂度，用哈希算法复杂度可以减少到 $O(n^2)$

- 1、化简条件： $100*a+110*b+12*c=532$
- 2、让一个变量移到等式另一边： $100*a+110*b=532-12*c$
- 3、用map存储 $532-12*c$ 可能的值
- 4、用两个循环遍历a，b。遍历 $100*a+110*b$ 可能的val值，若val值出现在map里，则 $(a,b,(532-val)/12)$ 即为所求值

```
# 哈希解法
dic = set()
for c in range(10):
    dic.add(266-6*c)

for a in range(10):
    for b in range(10):
        val = 50*a+55*b
        if val in dic:
            print(a,b,(266-val)//6)
```

哈夫曼编码

原理

为减小编码冗余，并保证编码的唯一性

例 5-7 设有离散无记忆信源

$$\begin{bmatrix} X \\ P \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & a_3 & a_4 & a_5 \\ 0.4 & 0.2 & 0.2 & 0.1 & 0.1 \end{bmatrix}$$

可有两种哈夫曼编码方法,如表 5-5 和表 5-6 所示,码树如图 5-7(a)和(b)所示。

表 5-5 哈夫曼编码方法一

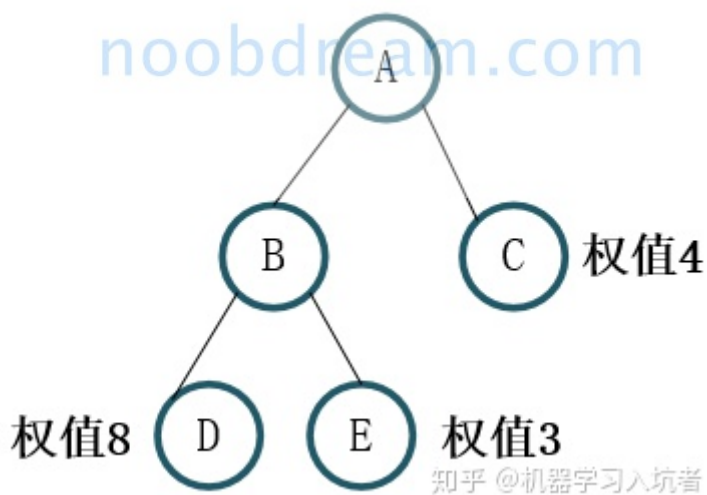
信源符号 a_i	概率 $p(a_i)$	编码过程	码字 W_i	码长 K_i
a_1	0.4		1	1
a_2	0.2		01	2
a_3	0.2		000	3
a_4	0.1		0010	4
a_5	0.1		0011	4

表 5-6 哈夫曼编码方法二

信源符号 a_i	概率 $p(a_i)$	编码过程	码字 W_i	码长 K_i
a_1	0.4		00	2
a_2	0.2		10	2
a_3	0.2		11	2
a_4	0.1		010	3
a_5	0.1		011	3

https://blog.csdn.net/qq_39658933

权值



1) 节点的带权路径长度

根据上述“节点路径”和“路径长度”的定义, 将从根节点到某节点的路径长度和节点权值的乘积, 称为节点的带权路径长度。比如上图中节点A到节点D的路径长度为2, 权值为8, 则带权路径长度为 $2 \times 8 = 16$ 。

2) 树的带权路径长度

根据上述“节点的带权路径长度”的定义, 将树中所有叶子节点的带权路径长度, 称为树的带权路径长度。比如上图中, 三个叶子节点C、D、E, 对应的路径长度分别为1、2、2, 对应的权值分别为4、8、3, 则树的带权路径长度为:

$$1 \times 4 + 2 \times 8 + 2 \times 3 = 26$$

将上述概念形式化，可以使用下面的公式进行计算：

$$WPL = l_1 \times w_1 + l_2 \times w_2 + \cdots + l_n \times w_n$$

其中 l 表示路径长度， w 表示权值， n 表示叶子节点个数。

WPL结论

非叶子节点结点值的和等于WPL

以上图为例WPL也可以表示为 $WPL = A+B+C+D+E$ 所有节点相加，与带权路径相乘求和结果相等

WPL含义：以任意一个字符串为例，如“AAAAABCD”，按哈夫曼编码规则，需将A，B，C，D的频次统计后，按小到大的排序后再编码。其编码后的叶子节点的权重则为频次大小，其WPL意义为该“AAAAABCD”字符串编码后的二进制位长度

二叉树

- 深度优先遍历
 - 前序遍历（递归法，迭代法）
 - 中序遍历（递归法，迭代法）
 - 后序遍历（递归法，迭代法）
- 广度优先遍历
 - 层次遍历（迭代法）

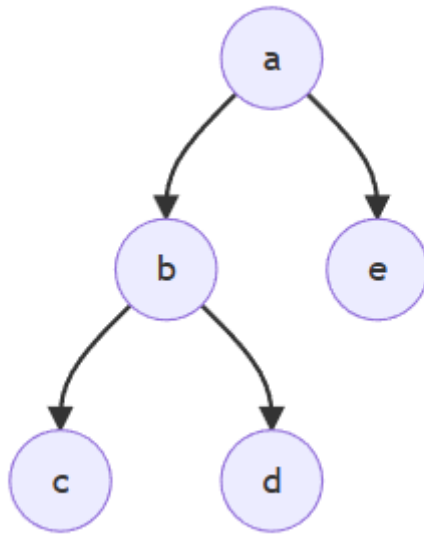
二叉树的构建

定义

```
class TreeNode():#二叉树节点
    def __init__(self, val, lchild=None, rchild=None):
        self.val=val          #二叉树的节点值
        self.lchild=lchild    #左孩子
        self.rchild=rchild    #右孩子
```

前序遍历数据格式构建二叉树

例子：给定前序遍历的结果字符串“abc##d##e##”，“#”为空。构建二叉树，二叉树结构如下



做递归问题，怎么回答三个问题：

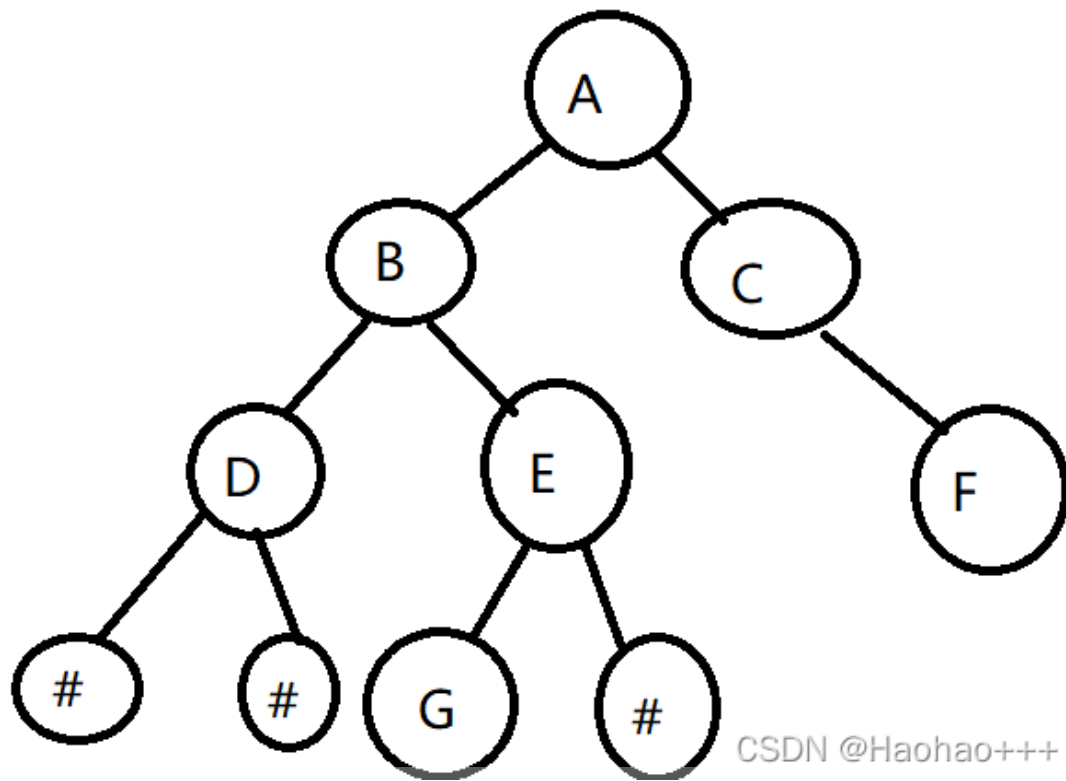
- 在我们的任务中，终止条件是什么？
[给我们的字符用完，也就不需要再创建节点了]
- 在我们的任务中，本次递归要干嘛？
[本次递归要创建三个节点，一个根节点，一个左节点，一个右节点]
- 在我们的任务中，本次递归要返回给上一次递归的是啥？
[当然是返回一个本层构造好的树的根节点]

```
def Creat_Tree(Root,vals):  
    if len(vals)==0:#终止条件：val用完了  
        return Root  
    if vals[0]!='#':#本层需要干的就是构建Root、Root.lchild、Root.rchild三个节点。  
        Root = TreeNode(vals[0])  
        vals.pop(0)  
        Root.lchild = Creat_Tree(Root.lchild,vals)  
        Root.rchild = Creat_Tree(Root.rchild,vals)  
        return Root#本次递归要返回给上一次的本层构造好的树的根节点  
    else:  
        Root=None  
        vals.pop(0)  
        return Root#本次递归要返回给上一次的本层构造好的树的根节点
```

层序遍历数据格式构建二叉树

以下面这棵树为例，则我们的输入为“ABCDE#F##G#####”，此类问题必给出满二叉树的完整字符串

利用完全二叉树的性质：设当前节点的序号为第k个，则左子节点的序号为 $2*k$ ，右子节点的序号为 $2*k+1$



CSDN @Haohao+++

```

class BiTreeNode:
    def __init__(self, data):
        self.data = data
        self.lchild = None
        self.rchild = None

class CreateBiTree:
    """
    str_tree: 传入字符串
    return: 返回根结点
    """
    def __init__(self, str_tree):
        self.str_tree = str_tree

    def create_tree(self):
        l1 = list(self.str_tree)
        l1 = [None if i == '#' else i for i in l1] # 将#转为None
        nodes = [BiTreeNode(l1[i]) for i in range(len(l1))] # 初始化各个结点
        root = nodes[0]
        for i in range(len(l1)):
            # 循环到了最后一层，因为上一层已设置好子节点，所以这一层直接退出
            if 2*i+2 > len(l1): # 循环到最后的右孩子结束，因为i从0开始，所以加2
                break
            cur_node = nodes[i]
            if nodes[2*i+1].data:
                cur_node.lchild = nodes[2*i+1] # 当前结点的左孩子
            if nodes[2*i+2].data:
                cur_node.rchild = nodes[2*i+2] # 当前结点的右孩子
        return root

    def __call__(self):
        return self.create_tree()

a = CreateBiTree('ABCDE#F##G#####')
  
```

```
root = a() # 调用__call__方法
print('根结点为: ', root.data)
```

深度遍历

看如下中间节点的顺序，就可以发现，中间节点的顺序就是所谓的遍历方式

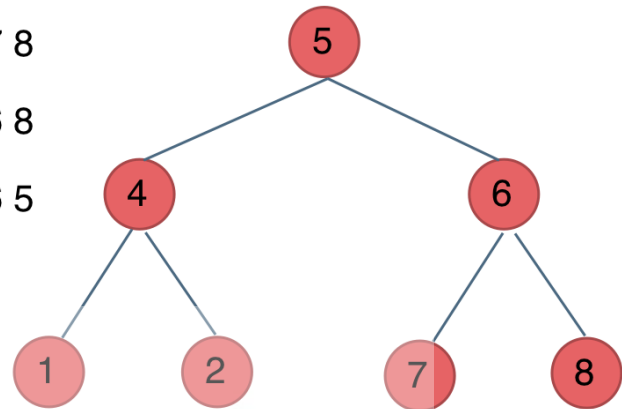
- 前序遍历：中左右
- 中序遍历：左中右
- 后序遍历：左右中

大家可以对着如下图，看看自己理解的前后中序有没有问题。

前序遍历（中左右）：5 4 1 2 6 7 8

中序遍历（左中右）：1 4 2 5 7 6 8

后序遍历（左右中）：1 2 4 7 8 6 5



前序遍历

递归法

递归函数的执行顺序会一直执行2的递归代码，即一直执行2的函数范围，直到2有返回值即遍历到最左下节点，才有递归函数终止条件的返回值!!!

所以递归法更容易理解

```
class Solution:
    def preorderTraversal(self, root: TreeNode) -> List[int]:
        # 保存结果
        result = []

        def traversal(root: TreeNode):
            if root == None:
                return
            result.append(root.val) # 前序
            traversal(root.left)    # 左
            traversal(root.right)   # 右

        traversal(root)
        return result
```

通用迭代法

```
class Solution:
    def preorderTraversal(self, root: TreeNode) -> List[int]:
        result = []
        st = []
```

```

if root:
    st.append(root)
while st:
    node = st.pop()
    if node != None:
        # 因为栈的先进后出原则，所以进栈时的顺序与理解的顺序相反，前序从中-左-右变成
        # 右-左-中
        if node.right: #右
            st.append(node.right)
        if node.left: #左
            st.append(node.left)
        st.append(node) #中，调整好了顺序，将node放回去
        st.append(None) # None是为了标记可以将后面的值加入结果了，在node==None
        # 时，将stack的None元素后的值加入到结果中
    else:
        # 从栈中取出要加入到结果的node节点
        node = st.pop()
        result.append(node.val)
return result

```

中序遍历

递归法

```

class Solution:
    def inorderTraversal(self, root: TreeNode) -> List[int]:
        result = []

        def traversal(root: TreeNode):
            if root == None:
                return
            traversal(root.left) # 左
            result.append(root.val) # 中序
            traversal(root.right) # 右

        traversal(root)
        return result

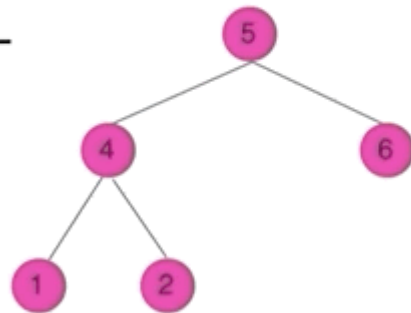
```

通用迭代法

以中序遍历为例

二叉树统一写法中序遍历（迭代法）

result数组:



D
代码随想录

```
class Solution:
    def inorderTraversal(self, root: TreeNode) -> List[int]:
        result = []
        st = []
        if root:
            st.append(root)
        while st:
            node = st.pop() #相较于C++这里是st.top，在下面再弹出节点，python这里已经取出
            #了节点
            if node != None:
                if node.right: #添加右节点（空节点不入栈）
                    st.append(node.right)

                st.append(node) #添加中节点
                st.append(None) #中节点访问过，但是还没有处理，加入空节点做为标记。

                if node.left: #添加左节点（空节点不入栈）
                    st.append(node.left)
            else: #只有遇到空节点的时候，才将下一个节点放进结果集，根据前中后序遍历到的下一个
            #节点，并加入结果集
                node = st.pop() #重新取出栈中元素
                result.append(node.val) #加入到结果集
        return result
```

后序遍历

递归法

```

class Solution:
    def postorderTraversal(self, root: TreeNode) -> List[int]:
        result = []

        def traversal(root: TreeNode):
            if root == None:
                return
            traversal(root.left)    # 左
            traversal(root.right)   # 右
            result.append(root.val) # 后序

        traversal(root)
        return result

```

通用迭代法

```

class Solution:
    def postorderTraversal(self, root: TreeNode) -> List[int]:
        result = []
        st = []
        if root:
            st.append(root)
        while st:
            node = st.pop()
            if node != None:
                st.append(node) #中
                st.append(None)

                if node.right: #右
                    st.append(node.right)
                if node.left: #左
                    st.append(node.left)
            else:
                node = st.pop()
                result.append(node.val)
        return result

```

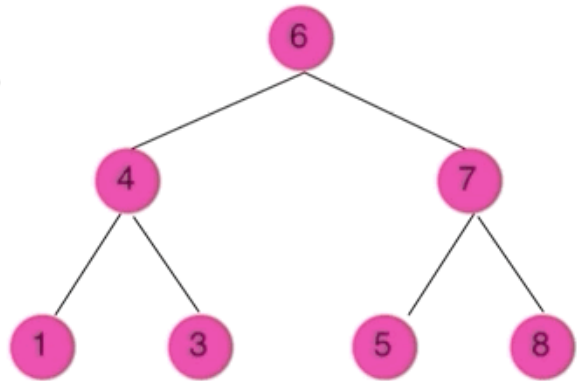
层序遍历（广度遍历）

方法

使用**双线队列**实现二叉树广度优先遍历，动画如下：



层序遍历结果:



D
代码随想录

模板代码

```
class Solution:
    """二叉树层序遍历迭代解法"""

    def levelOrder(self, root: TreeNode) -> List[List[int]]:
        results = []
        if not root:
            return results

        from collections import deque
        que = deque([root])

        while que:
            # 固定size, 直接用que.qsize()不行, 因为会动态变化
            size = len(que)
            result = []
            # 对每一层
            for _ in range(size):
                cur = que.popleft()
                result.append(cur.val)
                if cur.left:
                    que.append(cur.left)
                if cur.right:
                    que.append(cur.right)
            results.append(result)

        return results
```

层序遍历构建N叉树

题型: 给定的输入数据格式与二叉树构建小结的构建二叉树不一致 (满二叉树字符串数据)

以下题为例, 若给定数据如下形式, 则可用此方法

三叉树

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

一个关于三叉树的题目, 小于100的值代表树叶, 大于100的值为分支点, 建树的过程是水平方向建树,

输入格式:

先输入n, 代表有n组数据, 接下来n行, 输入4个数, 第一个数代表根节点, 接下来分别代表三个子节点, -1代表子节点不存在, 输入的顺序按照层次遍历的次序。接下来, 要求寻找叶子节点的最短路径, 最短路径是指不经过重复的边。输入方式, 首先输入一个值m, 代表m行, 接下来m行输入m个叶子节点和对应的优先级, 要求按优先级输出从上次到达的位置到该节点的最短路径, 每条路径的最后一个节点要求输出目标叶子节点, 最后要求回到根节点。

输入输出样例

输入样例#:

复制

```
10
100 101 108 107
101 1 102 2
108 103 104 105
107 17 109 18
102 3 4 5
103 7 8 9
104 10 106 11
105 15 16 -1
109 19 20 21
106 12 13 14
5
8 1
14 3
16 2
5 0
19 4
```

输出样例#:

复制

```
100 101 102 5
102 101 100 108 103 8
103 108 105 16
105 108 104 106 14
106 104 108 100 107 109 19
109 107 100
```

建树方法

利用层序遍历的方式构建树

```
class threeTreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.mid = None
        self.right = None

def Node3(root: threeTreeNode, li):
    if li[1] != -1: root.left = threeTreeNode(li[1])
    if li[2] != -1: root.mid = threeTreeNode(li[2])
    if li[3] != -1: root.right = threeTreeNode(li[3])
    return root

def createThreeTree(lis):
    from collections import deque
    # 先创建根节点
    root = threeTreeNode(lis[0][0])
    q = deque([root])
    # 初始化lis的index:
    index = 0
    while q:
        size = len(q)
        for _ in range(size):
            cur = q.popleft()
            # 判断下一层是否有叶子节点
            if cur.val >= 100:
                # root下有三个分支，此处是构建逻辑部分
                cur = Node3(cur, lis[index])
                # 判断逻辑，判断并添加下一层需要构建的节点
                if cur.left:
                    q.append(cur.left)
                if cur.mid:
                    q.append(cur.mid)
                if cur.right:
                    q.append(cur.right)
            index += 1
        return root

n = int(input())
lis = []
for i in range(n):
    lis.append(list(map(int, input().split())))

root = createThreeTree(lis)
```

根据遍历结果还原树

此类问题会给出中序遍历结果和前序或后序遍历结果，要求重构二叉树（必须给出中序，仅给出前序和后序无法保证二叉树唯一）

中序和后序

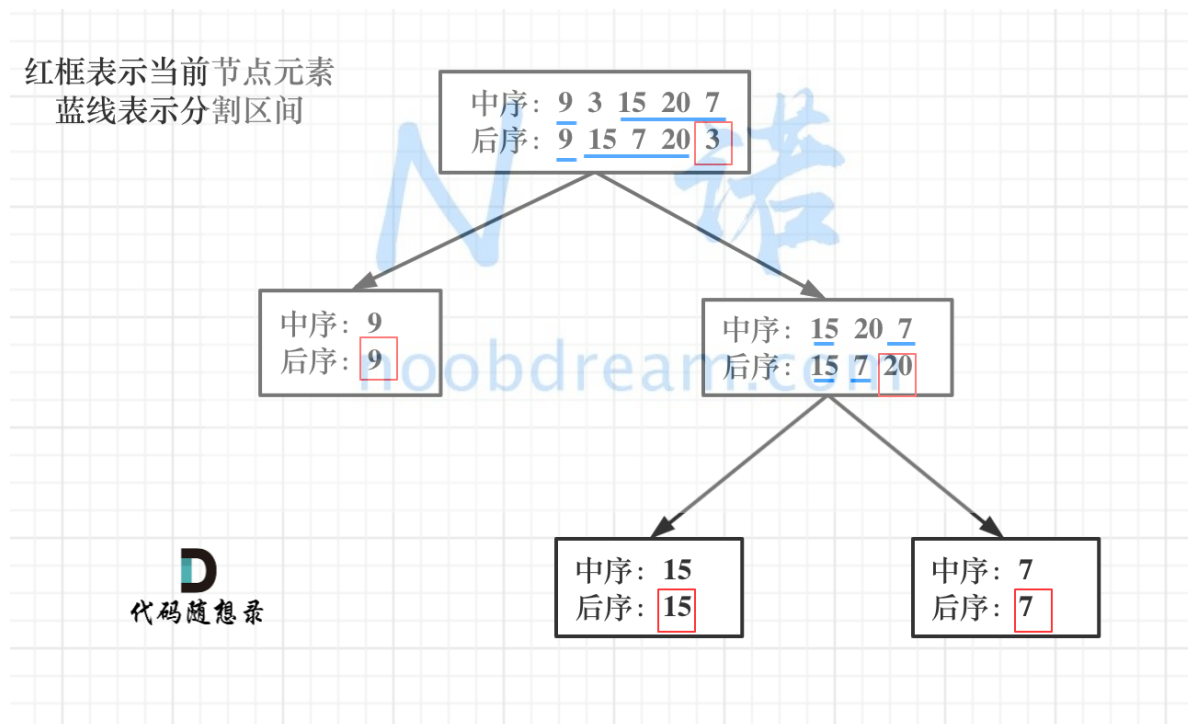
以下题为例：

中序遍历 inorder = [9,3,15,20,7] 后序遍历 postorder = [9,15,7,20,3] 返回如下的二叉树：

```
      3
     / \
    9  20
     / \
    15  7
```

如果让我们肉眼看两个序列，画一棵二叉树的话，应该分分钟都可以画出来。其中红色为当前子树下目标根节点

流程如图：



那么代码应该怎么写呢？

说到一层一层切割，就应该想到了递归。

来看一下一共分几步：

- 第一步：如果数组大小为零的话，说明是空节点了。
- 第二步：如果不为空，那么取后序数组最后一个元素作为节点元素。
- 第三步：找到后序数组最后一个元素在中序数组的位置，作为切割点
- 第四步：切割中序数组，切成中序左数组和中序右数组（顺序别搞反了，一定是先切中序数组）
- 第五步：切割后序数组，切成后序左数组和后序右数组
- 第六步：递归处理左区间和右区间

```
class Solution:
```

```

def buildTree(self, inorder: List[int], postorder: List[int]) ->
Optional[TreeNode]:
    # 第一步：特殊情况讨论：树为空。或者说是递归终止条件
    if not postorder:
        return

    # 第二步：后序遍历的最后一个就是当前的中间节点
    root_val = postorder[-1]
    root = TreeNode(root_val)

    # 第三步：找切割点。
    root_index = inorder.index(root_val)

    # 第四步：切割inorder数组。得到inorder数组的左,右半边。
    left_inorder = inorder[:root_index]
    right_inorder = inorder[root_index + 1:]

    # 第五步：切割postorder数组。得到postorder数组的左,右半边。
    # ☆ 重点1：中序数组大小一定跟后序数组大小是相同的。
    left_postorder = postorder[:len(left_inorder)]
    right_postorder = postorder[len(left_inorder): len(postorder) - 1]

    # 第六步：递归
    root.left = self.buildTree(left_inorder, left_postorder)
    root.right = self.buildTree(right_inorder, right_postorder)

    # 第七步：返回答案
    return root

```

中序和前序

noobdream.com

思路一致，只是前序的根节点在子树的第一个位置，后序在子树的最后的位置

```

class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> TreeNode:
        # 第一步：特殊情况讨论：树为空。或者说是递归终止条件
        if not preorder:
            return None

        # 第二步：前序遍历的第一个就是当前的中间节点。
        root_val = preorder[0]
        root = TreeNode(root_val)

        # 第三步：找切割点。
        separator_idx = inorder.index(root_val)

        # 第四步：切割inorder数组。得到inorder数组的左,右半边。
        inorder_left = inorder[:separator_idx]
        inorder_right = inorder[separator_idx + 1:]

        # 第五步：切割preorder数组。得到preorder数组的左,右半边。
        # ☆ 重点1：中序数组大小一定跟前序数组大小是相同的。利用长度一致原理，可以用于分割前
        preorder_left = preorder[1:1 + len(inorder_left)]

```

序序列

```
preorder_right = preorder[1 + len(inorder_left):]

# 第六步：递归
root.left = self.buildTree(preorder_left, inorder_left)
root.right = self.buildTree(preorder_right, inorder_right)

return root
```

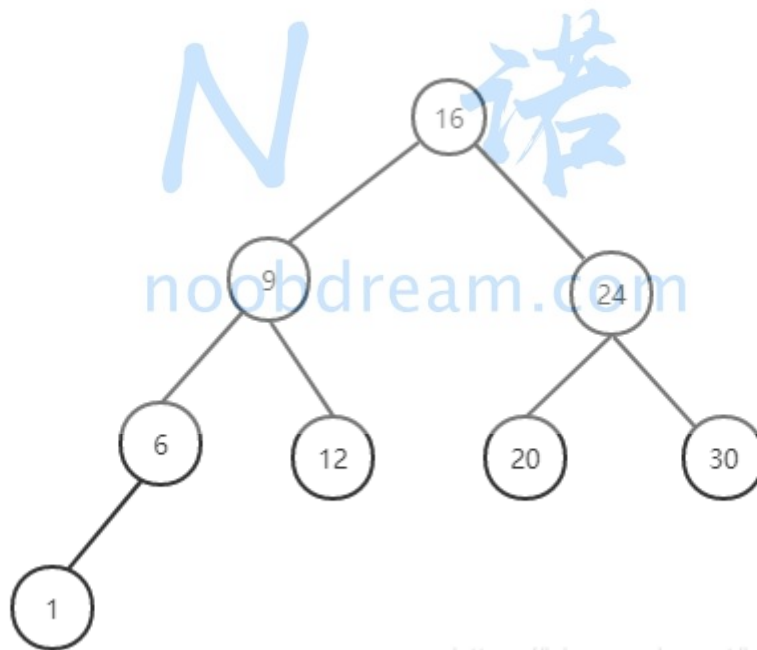
二叉搜索/排序/查找树

概念

二叉查找树要么是一棵空树，要么是一棵具有如下性质的非空二叉树：

- 若左子树非空，则左子树上的所有节点的关键字值均小于根节点的关键字值。
- 若右子树非空，则右子树上的所有节点的关键字值均大于根节点的关键字值。
- 左、右子树本身也分别是一棵二叉查找树（二叉排序树）。
- 二叉搜索树中无相等的值

可以看出，二叉搜索树中各个节点都不相同，且对**二叉查找树进行中序遍历，可以得到一个递增的有序序列**。如图所示，构建二叉搜索树时，需要注意节点元素的大小。（可用中序遍历后为有序数组的性质来判断树是否为搜索树）



<https://blog.csdn.net/Herishwater>

构建方法

在给定一组序列的条件下，如“1 6 5 9 8”，利用二叉树的左子树小，右子树大的特征，利用递归一个个值插入来构建树

```
def insert_SearchTree(root: TreeNode, x):
    # 当搜索到root为None时，即叶子节点时，构建该节点
    if root == None:
        root = TreeNode(x)
        return root
    # 排除掉相等的值
    if root.val == x:
```

```

        return root
    # 左子树小于根节点
    # 第一个root.left表示接收下一个递归的值作为左子树节点，第二个root.left表示x小于根节点，
    则向树的左子树方向遍历
    if root.val > x:
        root.left = insert_SearchTree(root.left, x)
    # 右子树大于根节点
    else:
        root.right = insert_SearchTree(root.right, x)
    return root

data = list(map(int, input().split()))
root = None
# 循环插入
for v in data:
    root = insert_SearchTree(root, v)

```

搜索方法

递归法：

```

class Solution:
    def searchBST(self, root: TreeNode, val: int) -> TreeNode:
        # 为什么要有返回值：
        # 因为搜索到目标节点就要立即return，
        # 这样才是找到节点就返回（搜索某一条边），如果不加return，就是遍历整棵树了。

        if not root or root.val == val:
            return root

        if root.val > val:
            return self.searchBST(root.left, val)

        if root.val < val:
            return self.searchBST(root.right, val)

```

迭代法：

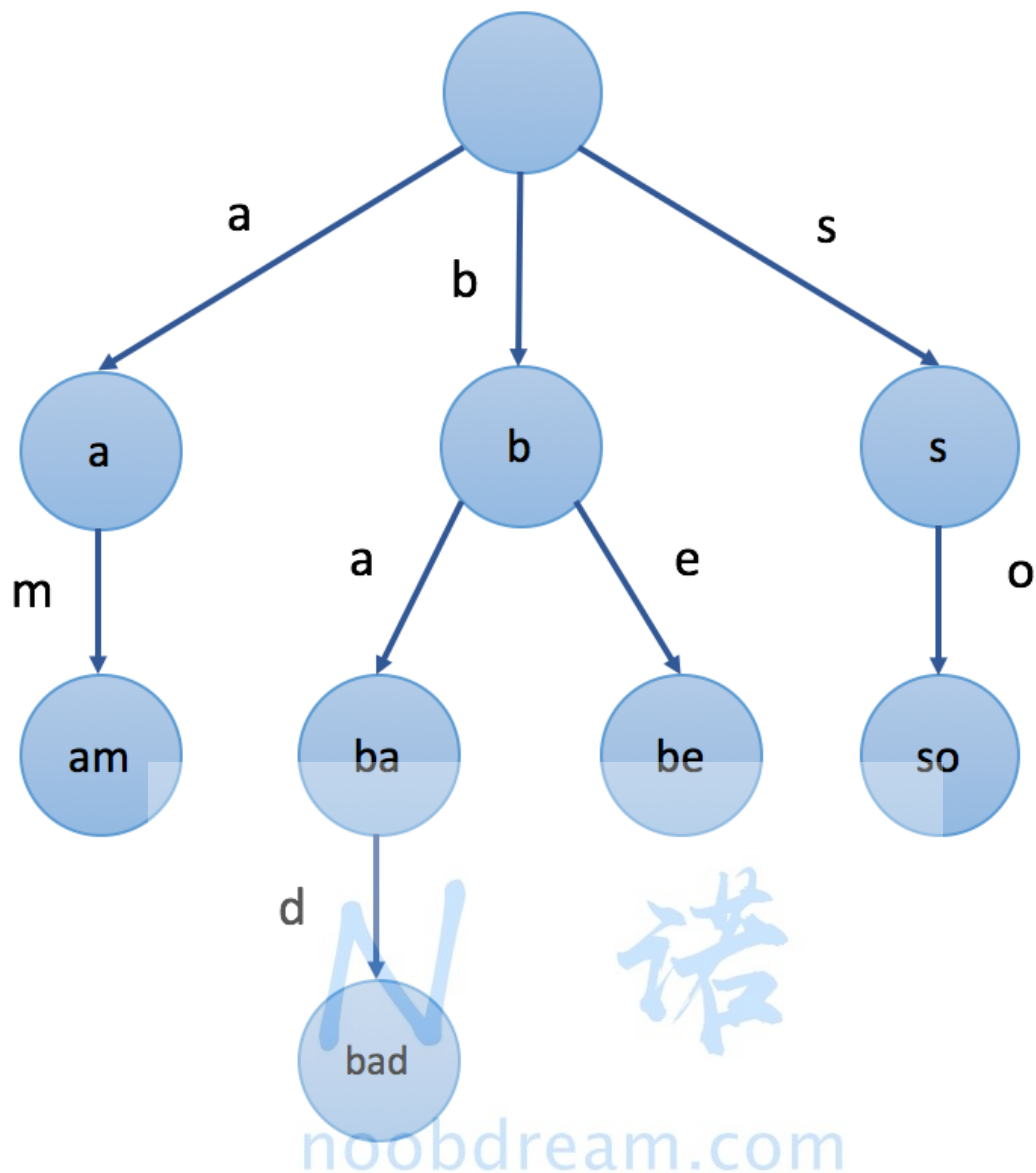
```

class Solution:
    def searchBST(self, root: TreeNode, val: int) -> TreeNode:
        while root is not None:
            if val < root.val: root = root.left
            elif val > root.val: root = root.right
            else: return root
        return None

```

前缀树

用于方便存储和检索字符串的N叉树，如下图



字典实现

```
class Trie:
    def __init__(self):
        """
        初始化
        """
        self.dic = {}

    def insert(self, word: str) -> None:
        """
        前缀树中插入字符串
        """
        tree = self.dic
        for i in word:
            # setdefault返回的是i对应的val值，即{}，此处可实现self.dic递归加入word，构建
            tree = tree.setdefault(i, {})
        # 用于标记该字符串结束
        tree['#'] = '#'
```

```

def search(self, word: str) -> bool:
    """
    搜索前缀树中是否有该字符串
    """
    tree = self.dic
    for i in word:
        if i not in tree:
            return False
        tree = tree[i]
    return '#' in tree

def startswith(self, prefix: str) -> bool:
    """
    判断前缀树中，是否含有prefix前缀的字符串
    """
    tree = self.dic
    for i in prefix:
        if i not in tree:
            return False
        tree = tree[i]
    return True

# 统计树中的叶子个数
def getLeaves(self)->int:
    sum = []
    def leaves(tree):
        # sum
        if tree == {'#': '#'}:
            sum.append(tree)
            return
        for k in tree.keys():
            # 排除掉遍历到某个word的结尾，但是没遍历到叶子节点的情况
            if k != '#':
                leaves(tree[k])
        return
    leaves(self.dic)
    return len(sum)

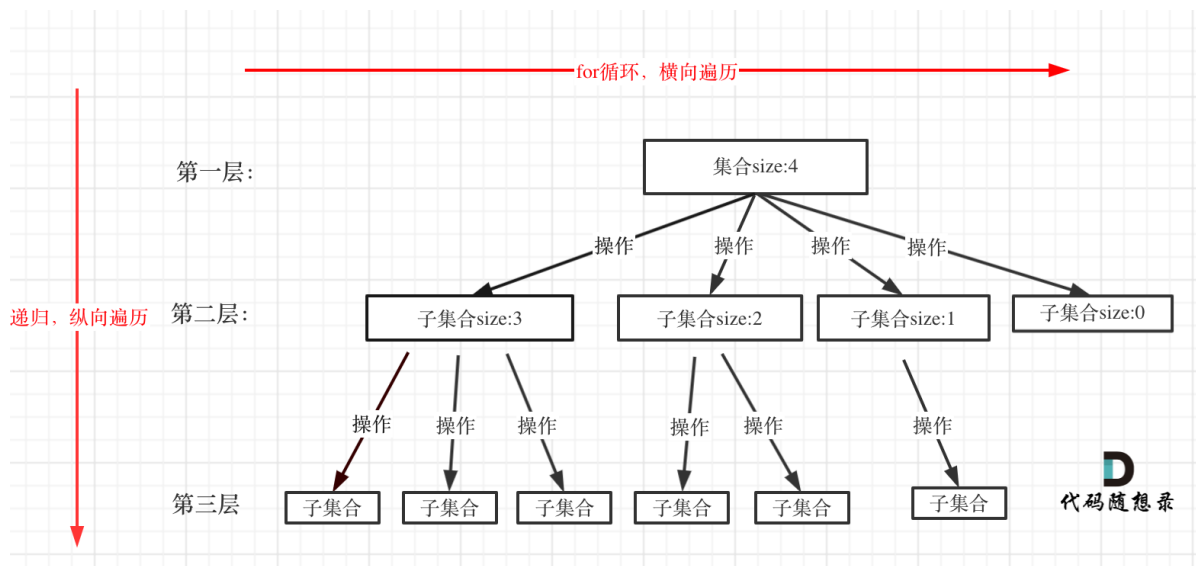
```

回溯

回溯模板

三部曲

递归函数的返回值以及参数，回溯函数终止条件，单层搜索的过程



```
void backtracking(参数) {  
    if (终止条件) {  
        存放结果;  
        return;  
    }  
  
    for (选择: 本层集合中元素 (树中节点孩子的数量就是集合的大小)) { //横向遍历  
        处理节点;  
        backtracking(路径, 选择列表); // 递归，深度遍历  
        回溯，撤销处理结果  
    }  
}
```

组合问题

noobdream.com

第77题. 组合

[力扣题目链接](#)

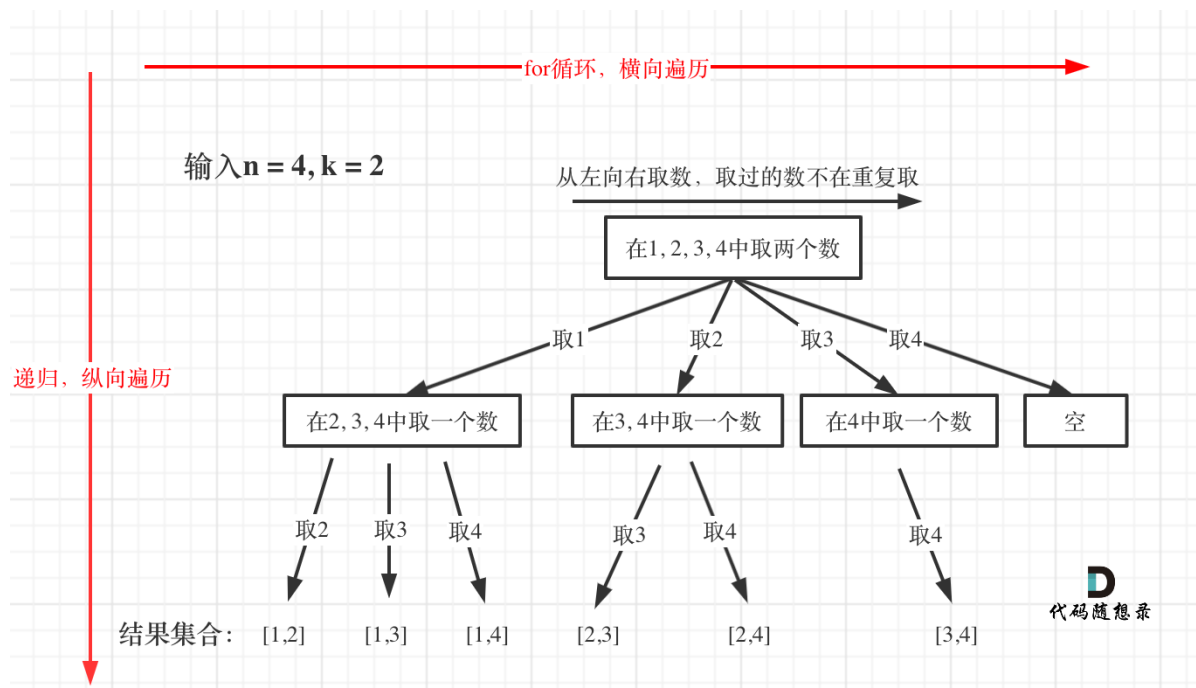
给定两个整数 n 和 k ，返回 $1 \dots n$ 中所有可能的 k 个数的组合。

示例:

输入: $n = 4, k = 2$

输出:

```
[  
  [2,4],  
  [3,4],  
  [2,3],  
  [1,2],  
  [1,3],  
  [1,4],  
]
```



```
class Solution:
    def combine(self, n: int, k: int) -> List[List[int]]:
        # 递归函数的返回值以及参数
        res = []
        path = []
        def backtrack(n, k, startIndex):
            # 回溯函数终止条件
            if len(path) == k:
                res.append(path[:])
                return
            # 单层搜索的过程
            for i in range(startIndex, n + 1):
                path.append(i) # 处理
                backtrack(n, k, i+1) # 递归下一层
                path.pop() # 回溯
        backtrack(n, k, 1)
        return res
```

全排列问题

输入输出样例

输入样例#:

复制

abc

输出样例#:

复制

abc
acb
bac
bca
cab
cba


```

def combine(n, li):
    res = []
    path = []

    def backtrack(n, li):
        if len(path) == n:
            res.append(''.join(path))
            return
        # 因为是全排列，所以不用start_index过滤，需要从头开始选
        for i in range(0, n):
            # 过滤掉重复选一个字母的情况
            if li[i] in path:
                continue
            path.append(li[i])
            backtrack(n, li)
            path.pop()

    backtrack(n, li)
    return res

while True:
    try:
        line = list(input())
        if not line:
            break
        res = combine(len(line), line)
        res.sort()
        for v in res:
            print(v)
    except:
        break

```

遍历N叉树路径

以下题为例

三叉树

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

一个关于三叉树的题目，小于100的值代表树叶，大于100的值为分支点，建树的过程是水平方向建树，

输入格式：

先输入n，代表有n组数据，接下来n行，输入4个数，第一个数代表根节点，接下来分别代表三个子节点，-1代表子节点不存在，输入的顺序按照层次遍历的次序。接下来，要求寻找叶子节点的最短路径，最短路径是指不经过重复的边。输入方式，首先输入一个值m，代表m行，接下来m行输入m个叶子节点和对应的优先级，要求按优先级输出从上次到达的位置到该节点的最短路径，每条路径的最后一个节点要求输出目标叶子节点，最后要求回到根节点。

输入输出样例

输入样例#:

复制

```
10
100 101 108 107
101 1 102 2
108 103 104 105
107 17 109 18
102 3 4 5
103 7 8 9
104 10 106 11
105 15 16 -1
109 19 20 21
106 12 13 14
5
8 1
14 3
16 2
5 0
19 4
```

输出样例#:

复制

```
100 101 102 5
102 101 100 108 103 8
103 108 105 16
105 108 104 106 14
106 104 108 100 107 109 19
109 107 100
```

方法：回溯三部曲

```
class threeTreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.mid = None
        self.right = None
```

```

def Node3(root: threeTreeNode, li):
    if li[1] != -1: root.left = threeTreeNode(li[1])
    if li[2] != -1: root.mid = threeTreeNode(li[2])
    if li[3] != -1: root.right = threeTreeNode(li[3])
    return root

def createThreeTree(lis):
    from collections import deque
    # 先创建根节点
    root = threeTreeNode(lis[0][0])
    q = deque([root])
    # 初始化lis的index:
    index = 0
    while q:
        size = len(q)
        for _ in range(size):
            cur = q.popleft()
            if cur.val >= 100:
                # root下有三个分支
                cur = Node3(cur, lis[index])
                if cur.left:
                    q.append(cur.left)
                if cur.mid:
                    q.append(cur.mid)
                if cur.right:
                    q.append(cur.right)
                index += 1
        return root

# 回溯
# 搜索全路径, 拿到所有目标叶子节点的路径, tar为
def getAllPath(root: threeTreeNode, tars: map):
    res = []
    path = []

    def backtrack(root: threeTreeNode, tars):
        # 回溯函数终止条件, 遍历到叶子节点即返回
        # 注意此处代表的是, 每次最底层的递归的返回条件, 非整体的任务条件
        if root == None:
            return

        # 单层逻辑
        path.append(root.val)
        # 若该节点为目标叶子节点, 则添加路径和优先级到res
        if root.val in tars:
            res.append((path[:], tars[root.val])) #[:]为list的值传递

        # 横向遍历
        if root.left:
            backtrack(root.left, tars)
        if root.mid:
            backtrack(root.mid, tars)

```

```

        if root.right:
            backtrack(root.right, tars)
        path.pop() #回溯

    backtrack(root, tars)
    return res

def get_path2path(li1:list,li2:list):
    path = []
    li1 = li1[::-1]
    for i,v in enumerate(li1):
        # 遍历到第一个相同的父节点时退出
        if v in li2:
            turn_index = li2.index(v)
            path = li1[1:i]+li2[turn_index:]
            break
    return path

n = int(input())
lis = []
for i in range(n):
    lis.append(list(map(int, input().split())))

m = int(input())
cks = {}
for i in range(m):
    tar,p=map(int, input().split())
    cks[tar] =p

root = createThreeTree(lis)
paths = getAllPath(root,cks)
# 根据优先级排序
paths.sort(key= lambda x:x[1])
paths =[v[0] for v in paths]

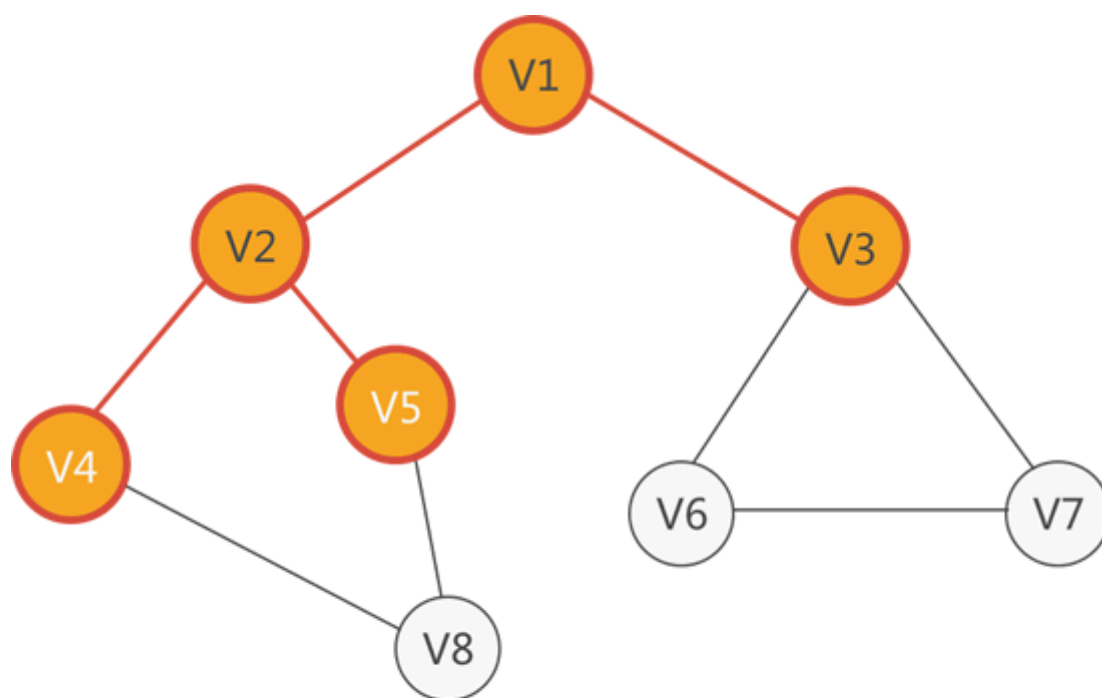
# 第一组保留原路径
print(' '.join([str(v) for v in paths[0]]))
for i in range(1,len(paths)):
    tar_path = get_path2path(paths[i-1],paths[i])
    print(' '.join([str(v) for v in tar_path]))
# 最后一个叶子节点到根节点路径
end_path = paths[-1][::-1][1:]
print(' '.join([str(v) for v in end_path]))

```

广度优先搜索（BFS）

概念

类似二叉树的层次遍历，一层一层的扩散搜索，可利用队列实现



顶点访问顺序：

V1	V2	V3	V4	V5			
----	----	----	----	----	--	--	--

例子

例1

N 诺

noobdream.com

迷宫

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

小 A 同学现在被困在了一个迷宫里面，他很想从迷宫中走出来，他可以向上、向下、向左、向右移动、每移动一格都需要花费 1 秒的时间，不能够走到边界之外。假设小 A 现在的位置在 S，迷宫的出口在 E，迷宫可能有多个出口。问小 A 想要走到迷宫出口最少需要花费多少秒？

输入输出格式

输入描述:

有多组测试数据。
第一行输入两个正整数 H ($0 < H \leq 100$) 和 W ($0 < W \leq 100$)，分别表示迷宫的高和宽。
接下来 H 行，每行 W 个字符（其中 '*' 表示路，'#' 表示墙，'S' 表示小 A 的位置，'E' 表示迷宫出口）。
当 H 与 W 都等于 0 时程序结束。

输出描述:

输出小 A 走到迷宫出口最少需要花费多少秒，如果永远无法走到出口则输出“-1”。

输入输出样例

输入样例#:

[复制](#)

```
3 3
S*#
**#
#*E
0 0
```

输出样例#:

[复制](#)

```
4
```

```
class point:
    def __init__(self, x, y, step=0):
        self.x = x
        self.y = y
        self.step = step
```

bfs实现

```
def bfs(start_point: point):
    from collections import deque
    q = deque()
    q.append(start_point)
```

```

check[start_point.y][start_point.x] = 1
ans = -1
while q:
    cur = q.popleft()
    # 节点的处理逻辑部分，第一个到达的路径，取得其step大小
    if mpt[cur.y][cur.x] == 'E':
        ans = cur.step
        break

    # 每次扩散的逻辑处理，层次遍历这里是对树每一层的size遍历添加子节点。在这里层的概念即为
    # step次数，所以已无需对层遍历
    for i in range(4):
        next_x = cur.x + dir[0][i]
        next_y = cur.y + dir[1][i]
        # 判断越界，越界后继续下一个操作
        if not (0 <= next_x < w and 0 <= next_y < h):
            continue
        # 判断路径是否正确，并避免重复遍历
        if (mpt[next_y][next_x] == '*' or mpt[next_y][next_x] == 'E') and
check[next_y][next_x] == 0:
            q.append(point(next_x, next_y, cur.step + 1))
            check[next_y][next_x] = 1

    return ans

while True:
    # 地图的高和宽
    h, w = map(int, input().split())
    if h == w == 0:
        break

    mpt = []
    # 定义一个数组避免节点重复遍历实现剪枝，此处也可以不加check数组（此处可以加check的原因是，
    # 遍历到重复节点的路径必不为最短路径）
    check = []
    # 定义一个数组代表上下左右(0,1,2,3)的坐标变化
    dir = [[0, 0, -1, 1], [-1, 1, 0, 0]]
    start_i = -1
    start_j = -1
    for i in range(h):
        line = list(input())
        if 'S' in line:
            start_i = i
            start_j = line.index('S')
        mpt.append(line)
        check.append([0 for _ in range(len(line))])

    ans = bfs(point(start_j, start_i, 0))
    print(ans)

```

例2

题型：需要获取到达目的地所需要去除的最少的障碍物

迷宫逃离2

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

江鸟突然想到了一个迷宫逃离的游戏，话说有三个人被困于一个 $n*m$ 的迷宫里，他们三人都可以向上、向下、向左、向右四个方向进行走动，当然他们所在的初始位置没有障碍物，同时只能走到没有障碍物的格子上，现在江鸟要问你最少需要去掉多少个格子的障碍物，可以使他们三人之间两两互相可达。

输入输出格式

输入描述:

输入包括多组测试数据，每组测试数据第一行为两个整数 n 和 m ($2 \leq n, m \leq 100$)，接下来 n 行，每行 m 个字符，其中：'w'、'W'、'f' 分别代表那三个人；'.' 代表没有障碍物的格子，'#' 代表有障碍物的格子。

输入输出样例

输入样例#:

[复制](#)

输出样例#:

[复制](#)

```
4 4
W...
####
.##f
W#.
4 4
W...
....
.##f
.W..
```

```
2
0
```

```
class point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

# check_tar大小与map一致，对于check_tar的(x, y)点是存储到达该点所移除的最小障碍物个数
def bfs(start_point: point, check_tar):
    from collections import deque
    q = deque()
    q.append(start_point)
```



```

check_tar[start_point.y][start_point.x] = 0
block = []
while q:
    cur = q.popleft()
    # 上下左右
    for i in range(4):
        next_x = cur.x + dir[0][i]
        next_y = cur.y + dir[1][i]
        # 判断越界,越界后继续下一个操作
        if not (0 <= next_x < w and 0 <= next_y < h):
            continue
        # 处理移除障碍物个数逻辑
        # temp是当前点的最少障碍物个数
        temp = check_tar[cur.y][cur.x]
        p = point(next_x, next_y)
        if mpt[next_y][next_x] == '#':
            temp += 1
        # **关键**, 与上个例子不同的是, 此处是如果从其他位置移动至当前位置需要去掉的障碍物更少, 那么更新check_tar矩阵中此位置的值, 并加入该节点
        # 上个例是check_tar用来避免重复遍历, 此处可以重复遍历节点, 只要该路径能移动更少的障碍物即可。上例是满足移动条件的p(路径)能通过就加入到列队中。此处是添加最优路径
        if temp < check_tar[next_y][next_x]:
            q.append(p)
            check_tar[next_y][next_x] = temp
    return block

```

```
import copy
```

```

# 地图的高和宽
h, w = map(int, input().split())
mpt = []
# 定义一个数组存储到达某点去除的最少障碍物数量
check = []
# 定义一个数组代表上下左右(0,1,2,3)的坐标变化
dir = [[0, 0, -1, 1], [-1, 1, 0, 0]]
# 定义一个字典, 代表f和w到达步数
start_w = []
start_w = []
start_f = []
for i in range(h):
    line = list(input())
    if 'w' in line:
        start_y = i
        start_x = line.index('w')
        start_w = [start_x, start_y]
    if 'W' in line:
        start_y = i
        start_x = line.index('W')
        start_w = [start_x, start_y]
    if 'f' in line:
        start_y = i
        start_x = line.index('f')

```

```
start_f = [start_x, start_y]
mpt.append(line)
check.append([h * w for _ in range(len(line))])

check_w = copy.deepcopy(check)
bfs(point(start_w[0], start_w[1]), check_w)
```

例3

统计"块"的数量

石油储藏

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

有一个GeoSurvComp地质勘探公司正在负责探测地底下的石油块。这个公司在一个时刻调查一个矩形区域，并且创建成一个个的格子用来表示众多正方形块。它然后使用测定设备单个的分析每块区域，决定是否有石油。一块有石油小区域被称为一个pocket，假设两个pocket是相邻的，然后他们就是相同石油块的一部分，石油块可能非常的大并且包涵很多的pocket。你的任务就是在一个网格中存在多少个石油块。输入首先给出图的大小，然后给出这个图。*代表没有石油，@代表存在石油。输出每种情况下石油块的个数。

noobdream.com

输入输出格式

输入描述:

输入包含一个或多个网格。每个网格都以包含m和n的行开始，网格中的行和列数为m和n，以单个空格分隔。如果m = 0，则表示输入结束。否则为 $1 \leq m \leq 100$ 和 $1 \leq n \leq 100$ 。这之后是m行，每行n个字符（不计算行末字符）。每个字符对应一个情节，要么是代表没有油的“*”，要么是代表油囊的“@”。

输出描述:

在水平，垂直或**对角线上都算作相邻** 输出每种情况下石油块的个数。

输入输出样例

输入样例#:

[复制](#)

```
5 5
*****@
*@@@*@
*@***@
@@@*@@
@@**@@
0 0
```

输出样例#:

[复制](#)

```
2
```

基本思路:

把'*'看成墙，利用一个数组，存储所有'@'的坐标，每次遍历一个块，则去掉块内对应'@'的坐标，当一次遍历结束则说明统计完了一个块。接着从'@'数组内继续取没去掉的'@'的坐标，继续遍历，直到'@'数组为空。

```
class point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

def bfs(p: point):
    from collections import deque
    d = deque([p])
    check[p.y][p.x] = 1
    tar_indexes.remove((p.x, p.y))
    while d:
        cur = d.popleft()
        # 因为此处对角线也算作块，所以point一步不仅可以上下左右，还可以向左斜上角，左斜下角，
        # 右斜上角，右斜下角移动，所以是每次对8个方向遍历。
        for i in range(8):
            x_next = cur.x + dir[0][i]
            y_next = cur.y + dir[1][i]
            # 判断越界
            if not (0 <= x_next < w and 0 <= y_next < h):
```

```

        continue

    if mpt[y_next][x_next] == '@' and check[y_next][x_next] == 0:
        d.append(point(x_next, y_next))
        tar_indexes.remove((x_next, y_next))
        check[y_next][x_next] = 1

while True:
    h, w = map(int, input().split())
    if h == w == 0:
        break
    mpt = []
    # 防止重复遍历
    check = []
    # 定义数组模拟上下左右，左斜上角，左斜下角，右斜上角，右斜下角移动
    dir = [[0, 0, -1, 1, -1, -1, 1, 1], [-1, 1, 0, 0, -1, 1, -1, 1]]
    # 定义数组存储@的位置
    tar_indexes = []
    for i in range(h):
        line = list(input())
        for j, v in enumerate(line):
            if v == '@':
                tar_indexes.append((j, i))
        mpt.append(line)
        check.append([0 if v == '@' else 1 for v in line])

    sum = 0
    while tar_indexes:
        start_p = point(tar_indexes[0][0], tar_indexes[0][1])
        bfs(start_p)
        sum += 1
    print(sum)

```

递归问题

除了三个问题（返回结果，终止条件，单次递归逻辑），还需要对单次递归逻辑细化，是对什么递归？

例1的结论：需要知道n与n-1的递推关系，**单次递归逻辑即为n-1递归到n的递推逻辑**

例子

例1



Hanoi塔问题

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

(n 阶Hanoi塔问题) 假设有三个分别命名为A、B、C的塔座，在塔座A上插有 n ($n < 20$) 个直径大小各不相同、依小到大编号为1, 2, ..., n 的圆盘。现要求将A轴上的 n 个圆盘移至塔座C上并按同样顺序叠排，圆盘移动时必须遵循下列规则：1) 每次只能移动一个圆盘；2) 圆盘可以插在A、B、C中的任一塔座上；3) 任何时刻都不能将一个较大的圆盘压在较小的圆盘之上。请通过编程来打印出移动的步骤。

输入输出格式

输入描述:

只有一组输入数据。输入数据 N (表示在开始时A塔座上的盘子数)，当输入0时程序结束。

输出描述:

输出移动的步骤。如"A-->C", "A-->B"等。每两的步骤之间有三个空格隔开，每输出5个步骤就换行。详细的见Sample Output。

输入输出样例

输入样例#:

复制

```
5
2
0
```

输出样例#:

复制

```
A-->C   A-->B   C-->B   A-->C
B-->C   A-->C   A-->B   C-->B
B-->A   C-->B   A-->C   A-->B
A-->C   B-->A   B-->C   A-->C
C-->B   C-->A   B-->A   B-->C
A-->B   C-->B   A-->C   B-->A
A-->C
A-->B   A-->C   B-->C
```

原理分析:

案例1

- 只有一个盘子时，则直接有A，移动到C 完成。

案例2

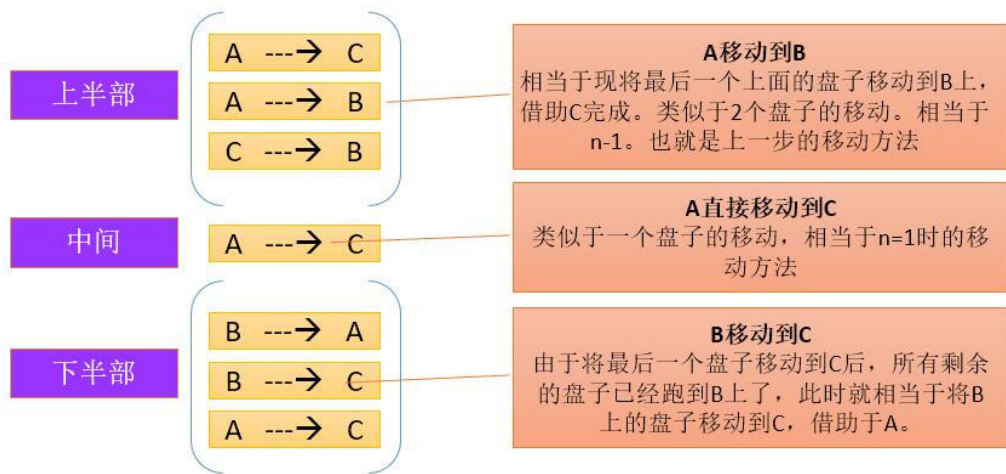
- 如果有两个盘子时，移动方案如下：
A—>B #借助B，帮助C拿到最后一个盘子

A→C #最后一个盘子移动完成后，剩余的盘子已经全部到B上了

B→C #将剩余的盘子移动到C上，完成

案例3

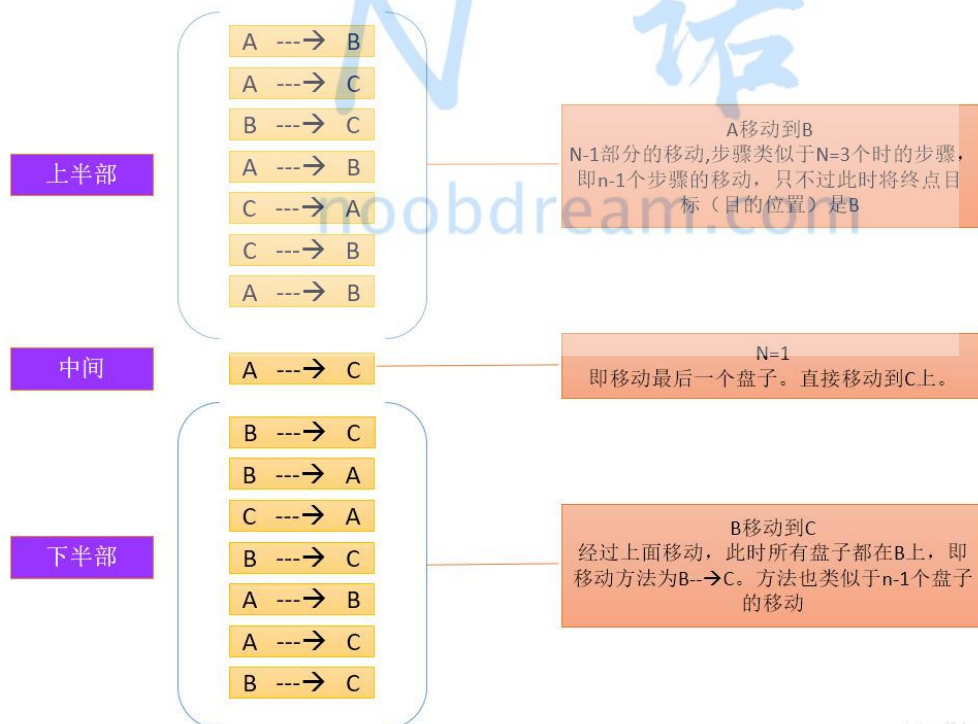
- 如果有三个盘子，盘子数量 $N = 3$,移动方案如下：



<https://blog.csdn.net/u013008795>

案例4

- 如果有四个盘子，盘子数量 $N = 4$,移动方案如下：



<https://blog.csdn.net/u013008795>

1. 当盘子只有一个是，即 $N=1$,只有一个动作，从A移动到C即结束

2. 当有 N 个盘子时：

- 上半部分：移动一定和 $n-1$ 盘子移动时的动作相同。出发地都是A,只不过， $n-1$ 个盘子移动的目的地是B，而不是C。这样方便取出最后一个盘子
- 中间部分：一定是由A移动到C
- 下半部分：此时等待移动的盘子已经都在B上，而不是A上，移动步骤类似于 $n-1$ 个盘子的移动。出发地是B，目的地是C

递归分析

1. 经过上面例子可以得到。如果想要移动n个盘子到指定目的，那么一定要先将n-1个盘子移动到备用柱上。
2. 当n-1个盘子移动后，必定要将最后一个盘子移动到C上。
3. 经过上面的调换后，剩下待处理的盘子还有n-1个，此时盘子已经在B上而不是在A上。此时需要重新判断剩余的盘子个数。如果依然大于1个那么还是需要按照1的步骤移动第n-1个盘子之上的所有盘子共n-2到A上。从而好让第n-1个盘子从B移动到C。

总结：到第3步时，发现和第一步第二步类似地方。只需要改变盘子的出发点和目的点，就能刚好递归

```
def hanoi(a, b, c, n):
    count = 0

    def recur(a, b, c, n):
        nonlocal count
        if n == 1:
            count += 1
            if count % 5 == 0:
                print("%s-->%s"%(a,c))
            else:
                print("%s-->%s"%(a,c),end='    ')
        else:
            # 上半部分移到与n-1一致，只是目的地变为从a移到b
            recur(a, c, b, n - 1)
            # 将底盘移到c塔上
            recur(a, b, c, 1)
            # 下半部分移到与n-1一致，只是目的地变为从b移到c
            recur(b, a, c, n - 1)
        recur(a, b, c, n)

    while True:
        n = int(input())
        if n == 0:
            break
        hanoi('A', 'B', 'C', n)
        print()
```

并查集

并查集(Disjoint-Set)是一种可以动态维护若干个不重叠的集合，并支持合并与查询两种操作的一种数据结构。

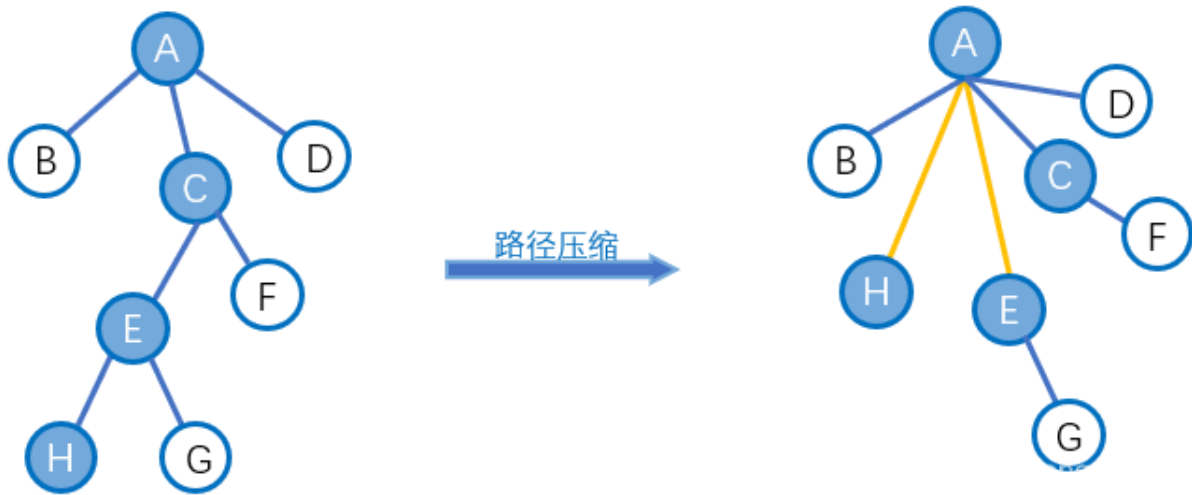
基本操作

1. 合并(Union)：合并两个集合。
2. 查询(Find)：查询元素所属集合。

实际操作时，我们会使用一个点来代表整个集合，即一个元素的根结点(可以理解为父亲)。

路径压缩：实际上，我们在查询过程中只关心根结点是什么，并不关心这棵树的形态(有一些题除外)。因此我们可以在查询操作的时候将访问过的每个点都指向树根，这样的方法叫做路径压缩，单次操作复杂度为 $O(\log N)$ 。

路径压缩示例：经过一次FIND节点H，后数据结构的变化。



具体实现

1.初始化

初始化是将每个元素看作一个集合，并将元素本身作为集合中的代表元素。

```
#假设有64个元素，我们使用一个等长的列表来存储每个元素所属列表的代表元素
n = 64
parents = [0]*64
for i in range(n):
    parents[i] = i

#使用字典初始化例子，给定data_list集合
father_dict = {}
size_dict = {}

for node in data_list:
    father_dict[node] = node
```

2.查询

查询过程就是查询元素所在集合的代表元素的过程。

```
def find(x):
    if parents[x] == x:
        return x
    else:
        parents[x]=find(parents[x])
        return parents[x]
```

3.合并

合并的过程就是将属于相同集合的元素进行合并的过程。相当于例子中的帮派的吞并过程

```
def unite(x,y):
    parents[find(x)] = find(y)
```

比较完整的代码：

我们建立一个数组 `father_dict` 字典表示一个并查集, `father_dict[i]` 表示的父节点。并且设置一个 `size_dict` 字典, `size_dict[i]` 表示 `i` 的后代节点的个数, 包括其本身。

1. **初始化**: 每一个点都是一个集合, 因此自己的父节点就是自己 `father_dict[i]=i`, `size_dict[i]=1`。
2. **查询**: 每一个节点不断寻找自己的父节点, 若此时自己的父节点就是自己, 那么该点为集合的根结点, 返回该点。
3. **合并**: 合并两个集合只需要合并两个集合的根结点, `size_dict` 大吃小, 为了尽可能的降低树高。

```
class UnionFindSet(object):
    """并查集"""
    def __init__(self, data_list):
        """初始化两个字典, 一个保存节点的父节点, 另外一个保存父节点的大小
        初始化的时候, 将节点的父节点设为自身, size设为1"""
        self.father_dict = {}
        self.size_dict = {}

        for node in data_list:
            self.father_dict[node] = node
            self.size_dict[node] = 1

    def find(self, node):
        """使用递归的方式来查找父节点

        在查找父节点的时候, 顺便把当前节点移动到父节点上面
        这个操作算是一个优化
        """
        father = self.father_dict[node]
        if (node != father):
            if father != self.father_dict[father]: # 在降低树高优化时, 确保父节点大小字典正确
                self.size_dict[father] -= 1
                father = self.find(father)
            self.father_dict[node] = father
        return father

    def is_same_set(self, node_a, node_b):
        """查看两个节点是不是在一个集合里面"""
        return self.find(node_a) == self.find(node_b)

    def union(self, node_a, node_b):
        """将两个集合合并在一起"""
        if node_a is None or node_b is None:
            return

        a_head = self.find(node_a)
        b_head = self.find(node_b)

        if (a_head != b_head):
            a_set_size = self.size_dict[a_head]
            b_set_size = self.size_dict[b_head]
            if (a_set_size >= b_set_size):
                self.father_dict[b_head] = a_head
                self.size_dict[a_head] = a_set_size + b_set_size
            else:
                self.father_dict[a_head] = b_head
                self.size_dict[b_head] = a_set_size + b_set_size
```

最小生成树

kruskal算法

算法描述

1. 新建图G，G中拥有原图中相同的节点，但没有边
2. 将原图中所有的边按权值从小到大排序
3. 从权值最小的边开始，如果这条边连接的两个节点于图G中不在同一个连通分量中，则添加这条边到图G中

去环原理：利用并查集，并查集里的每一个集合都是一个连通图(或连通分量)（所有节点必有公共的根节点），如果一条边的两个顶底都在同一个集合中，则必成环

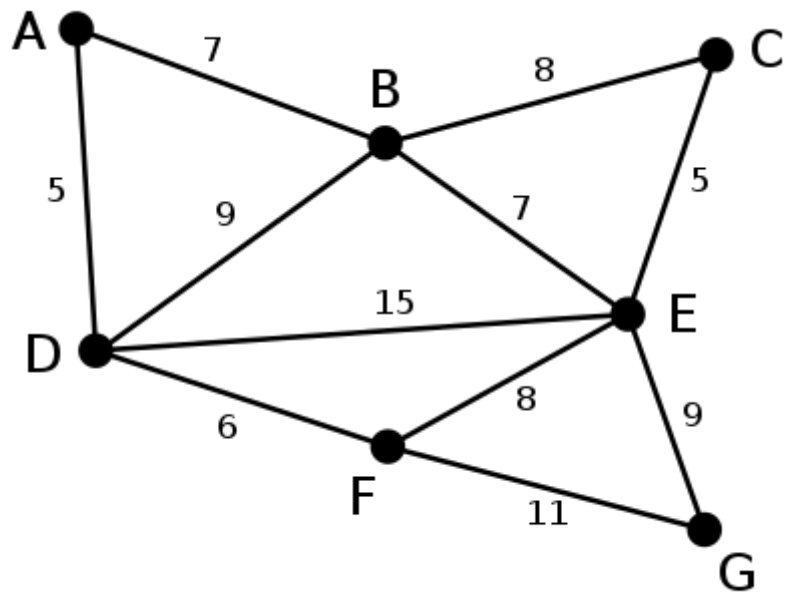
4. 重复3，直至图G中所有的节点都在同一个连通分量中

算法示例

图例	说明
	AD和CE是最短的两条边，长度为5，其中AD被任意选出，以高亮表示。
	现在CE是不属于环的最短边，长度为5，因此第二个以高亮表示。
	下一条边是长度为6的DF，同样地以高亮表示。
	接下来的最短边是AB和BE，长度均为7。AB被任意选中，并以高亮表示。边BD用红色高亮表示，因为B和D之间已经存在一条（标为绿色的）路径，如果选择它将会构成一个环（ABD）。
	以高亮表示下一条最短边BE，长度为7。这时更多的边用红色高亮标出：会构成环BCE的BC、会构成环DBEA的DE以及会构成环FEBAD的FE。
	最终，标记长度为9的边EG，得到最小生成树，结束算法过程。

模板

对于图：



简易版：

```
x = dict()

def make_set(point):
    x[point] = point

# 并查集
def find(point):
    if x[point] != point:
        x[point] = find(x[point])
    return x[point]

def kruskal(vertices, edges):
    '''KRUSKAL算法实现'''
    for vertex in vertices:
        make_set(vertex)
    minu_tree = []
    edges.sort() # 按照权重从小到大排序
    for edge in edges:
        weight, vertex1, vertex2 = edge
        # 防止选到环路（当find(vertex1) == find(vertex2)则成环），
        if find(vertex1) != find(vertex2):
            x[find(vertex1)] = find(vertex2)
            minu_tree.append(edge)
    return minu_tree

vertices = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
edges = [(7, 'A', 'B'),
         (5, 'A', 'D'),
```

```

        (8, 'B', 'C'),
        (9, 'B', 'D'),
        (7, 'B', 'E'),
        (5, 'C', 'E'),
        (15, 'D', 'E'),
        (6, 'D', 'F'),
        (8, 'E', 'F'),
        (9, 'E', 'G'),
        (11, 'F', 'G'),
    ]

    print(kruskal(vertices, edges))
# 输出[(5, 'A', 'D'), (5, 'C', 'E'), (6, 'D', 'F'), (7, 'A', 'B'), (7, 'B', 'E'),
(9, 'E', 'G')]

```

高级版：

以全局变量X定义节点集合，即类似{'A': 'A', 'B': 'B', 'C': 'C', 'D': 'D'}，如果A、B两点连通，则会更改为{'A': 'B', 'B': 'B', ...}，即任何两点连通之后，两点的值value将相同。

X = dict()

R是为了使parent树高度最低，搜索更快

R = dict() # 各点的初始等级均为0，如果被做为连接的的末端，则增加1

def make_set(point):

 X[point] = point

 R[point] = 0

def find(point):

 if X[point] != point:

 X[point] = find(X[point])

 return X[point]

def merge(point1, point2):

 '''连接两个分量（节点）

 '''

 if R[r1] > R[r2]:

 X[r2] = r1

 else:

 X[r1] = r2

 if R[r1] == R[r2]:

 R[r2] += 1

def kruskal(vertices, edges):

 '''KRUSKAL算法实现

 '''

 for vertice in vertices:

 make_set(vertice)

 minu_tree = []

 edges.sort() # 按照权重从小到大排序

 for edge in edges:

 weight, vertice1, vertice2 = edge

 if find(vertice1) != find(vertice2):

 merge(vertice1, vertice2)

 minu_tree.append(edge)

 return minu_tree

vertices = ['A', 'B', 'C', 'D', 'E', 'F', 'G']

edges = [(7, 'A', 'B'),

```

(5, 'A', 'D'),
(8, 'B', 'C'),
(9, 'B', 'D'),
(7, 'B', 'E'),
(5, 'C', 'E'),
(15, 'D', 'E'),
(6, 'D', 'F'),
(8, 'E', 'F'),
(9, 'E', 'G'),
(11, 'F', 'G'),
]
print(kruskal(vertices, edges))
# 输出[(5, 'A', 'D'), (5, 'C', 'E'), (6, 'D', 'F'), (7, 'A', 'B'), (7, 'B', 'E'),
(9, 'E', 'G')]

```

例子

给出n个点的图，和其点的坐标，输出图的最短路径

输入输出样例

输入样例#:

复制

```

3
1.0 1.0
2.0 2.0
2.0 4.0

```

输出样例#:

复制

```
3.41
```

组合排列获取边集合信息

```

def dfs(loc):
    edges = []
    path = []

    def backtrack(loc, n, start_index):
        if len(path) == 2:
            v1 = path[0][:]
            v2 = path[1][:]
            w = ((v1[0] - v2[0]) ** 2 + (v1[1] - v2[1]) ** 2) ** 0.5
            # 节点1, 节点2, 边权重
            edges.append((v1[2], v2[2], w))
            return
        for i in range(start_index, n):
            path.append(loc[i])
            # 因为避免重复取如 (2,2) 的情况, 所以是i+1
            backtrack(loc, n, i + 1)
            path.pop()
        return

    backtrack(loc, len(loc), 0)
    return edges

```

```

def find(x):
    if parents[x] != x:
        parents[x] = find(parents[x])
    return parents[x]

while True:
    try:
        parents = {}
        # n个节点有n*(n-1)/2条边
        n = int(input())
        loc = []
        for i in range(n):
            x, y = map(float, input().split())
            loc.append((x, y, i))

        for i in range(n):
            parents[i] = i
        edges = dfs(loc)
        edges.sort(key=lambda x: x[-1])

        cos = 0
        for edge in edges:
            u, v, w = edge
            u_p = find(u)
            v_p = find(v)
            if u_p != v_p:
                parents[u_p] = v_p
                cos += w
        print("{:.2f}".format(cos))
    except:
        break

```

最短路径

多源

即求多个源到目的地的多个最短路径（当然也可以求单个）

Floyd算法

Floyd-Warshall算法是解决任意两点间的最短路径的一种算法。通常可以在任何图中使用，包括有向图、带负权边的图。

存储方式采用邻接矩阵，随后改变矩阵的值，使得矩阵任意一点(i,j)表示为i到j的最短路径

- `distance[][]`：用来储存每个点到其他点的最短距离
- `path[][]`：用来储存每个点到其他点最短距离的路径

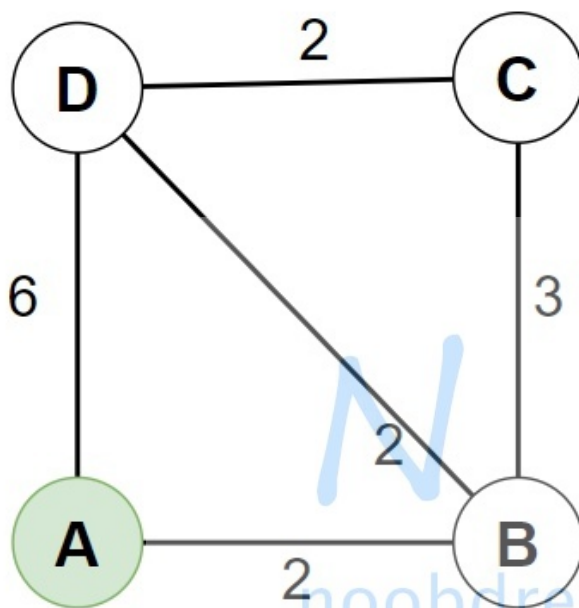
要点：以每个点为「中转站」，刷新所有「入度」和「出度」的距离。

所以我们要：遍历每一个顶点 --> 遍历点的每一个入度 --> 遍历每一个点的出度，以这个点为「中转站」距离更短就刷新距离（比如 B 点为中转站 $AB + BD < AD$ 就刷新 A 到 D 的距离）

初始化：

初始化距离 distance 为图结构 graph，初始化路径 path，path中0,1,2,3分别代表“A, B, C, D”，path[0][2]为例，表示A->B中间以B为中转节点，为初始图结构的路径如下

```
====distance====
0 2 -1 6
2 0 3 2
-1 3 0 2
6 2 2 0
====path====
0 1 2 3
0 1 2 3
0 1 2 3
0 1 2 3
```



	A	B	C	D
A	0	2	-1	6
B	2	0	3	2
C	-1	3	0	2
D	6	2	2	0

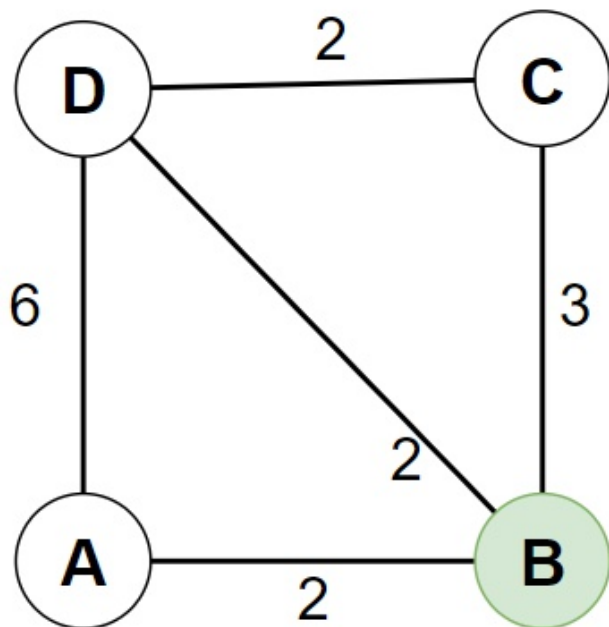
1、以 A 为「中转站」，刷新所有「入度」和「出度」的距离

A 的入度有 B、D 2 点，A 的出度也是 B、D 2 点

$$BA + AD > BD$$

$$DA + AB > DB$$

所以没有更小的距离，不能「刷新距离」，`distance[][]` 和 `path[][]` 不刷新



	A	B	C	D
A	0	2	-1	6
B	2	0	3	2
C	-1	3	0	2
D	6	2	2	0

知乎 @半亩荒唐

2、(重点) 以 B 为「中转站」，刷新所有的「入度」和「出度」的距离

B 的入度有 A、C、D；B 的出度有 A、C、D。（所以一共有 6 种组合）

$AB + BC < AC$ ($2 + 3 < \text{无穷大}$ ，这里的 -1 代表无穷大)

- 刷新距离：
- 刷新距离：将 $AB + BC$ 的距离 5 赋值给 AC 距离 -1，即 `distance[0][2] = distance[0][1] + distance[1][2]`
- 刷新最短路径：AC 的最短距离不再是直线 AC 的最短距离，引入「中转站」B 点，即 `path[0][2] = 1`

$AB + BD < AD$ ($2 + 2 < 6$)

- 刷新距离：
- 刷新距离：将 $AB + BD = 4$ 的值赋值给 AD ，即 `distance[0][3] = distance[0][1] + distance[1][3]`
- 刷新最短路径：AD 的最短距离不再是直线 AD 的最短距离，引入「中转站」B 点，即 `path[0][3] = 1`

$CB + BA < CA$ ($2 + 3 < \text{无穷大}$ 同理第一个 $AB + BC < AC$ ，刷新距离)

$CB + BD > CD$ ($3 + 2 > 2$ ，不用刷新距离)

$DB + BA < DA$ ($2 + 2 < 6$ ，同理第二个 $AB + BD < AD$ ，刷新距离)

$DB + BC < DC$ ($2 + 3 < 2$ ，不用刷新距离)

刷新后的 `distance[][]` 和 `path[][]` 入下所示

====distance====

0 2 5 4

2 0 3 2

5 3 0 2

4 2 2 0

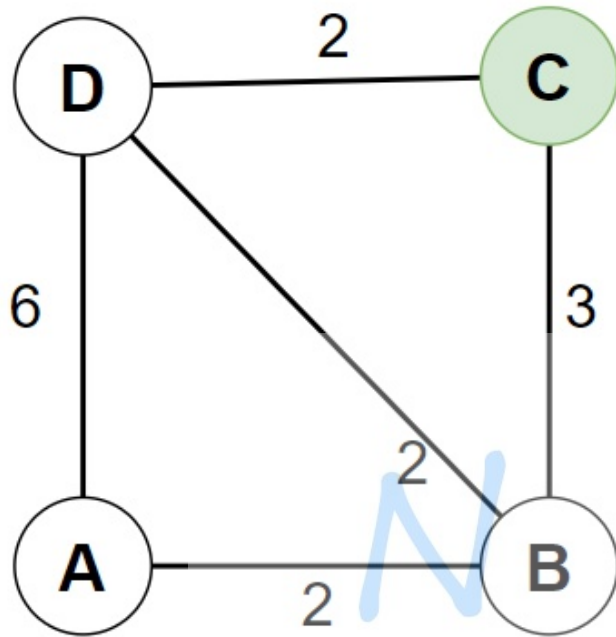
====path====

0 1 1 1

0 1 2 3

1 1 2 3

1 1 2 3



	A	B	C	D
A	0	2	5	4
B	2	0	3	2
C	5	3	0	2
D	4	2	2	0

知乎 @半亩荒唐

3、以 C 点为「中转站」，刷新所有「出度」和「入度」的距离

类似 3 步骤，这里不赘述

4、以 D 点为「中转站」，刷新所有「出度」和「入度」的距离

类似 3 步骤，这里不赘述，结束算法

模板代码

```
import math
```

```
nodes = ('A', 'B', 'C', 'D', 'E')
```

```
# dis矩阵为方阵
```

```
dis = [[0,1,2,math.inf,4],
        [1,0,math.inf,8,2],
        [2,math.inf,0,math.inf,6],
        [math.inf,8,math.inf,0,3],
        [4,2,6,3,0]]
```

```
def shortDistance(dis):
```

```
    node_num = len(dis)
```

```
    for k in range(node_num):          # 对角线遍历，十字交叉法的位置位置，先列后行，对角  
        line的点即为跳板节点
```

```
for i in range(node_num): # 列 十字交叉的列。遍历行列，即遍历经过当前跳板k是
    否有更短的路径？
    for j in range(node_num): # 行 十字交叉的行
        # 先列后行，形成一个传递关系，若比原来距离小，则更新
        dis[i][j] = min(dis[i][k] + dis[k][j],dis[i][j])
```

例1

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

在每年的校赛里，所有进入决赛的同学都会获得一件很漂亮的t-shirt。但是每当我们的工作人员把上百件的衣服从商店运回到赛场的时候，却是非常累的！所以现在他们想要寻找最短的从商店到赛场的路线，你可以帮助他们吗？

输入输出格式

输入描述:

输入包括多组数据。每组数据第一行是两个整数N、M ($N \leq 100$, $M \leq 10000$)，N表示成都的大街上有几个路口，标号为1的路口是商店所在地，标号为N的路口是赛场所在地，M则表示在成都有几条路。N=M=0表示输入结束。接下来M行，每行包括3个整数A，B，C ($1 \leq A, B \leq N, 1 \leq C \leq 1000$)，表示在路口A与路口B之间有一条路，我们的工作人员需要C分钟的时间走过这条路。
输入保证至少存在1条商店到赛场的路线。

输出描述:

对于每组输入，输出一行，表示工作人员从商店走到赛场的最短时间

输入输出样例

输入样例#:

复制

```
2 1
1 2 3
3 3
1 2 5
2 3 5
3 1 2
0 0
```

输出样例#:

复制

```
3
2
```

```
import math
```

```

def floyd():
    for k in range(1, n + 1):
        for i in range(1, n + 1):
            for j in range(1, n + 1):
                # mpt[i][k] + mpt[k][j] 为另一条路径
                mpt[i][j] = min(mpt[i][k] + mpt[k][j], mpt[i][j])

while True:
    n, m = map(int, input().split())
    if n == m == 0:
        break
    edges = []
    # 初始化, 除对角线的边 (点自己到自己), 全部设为inf
    mpt = [[0] * (n + 1) for i in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            if i == j:
                mpt[i][j] = 0
            else:
                mpt[i][j] = math.inf

    for _ in range(m):
        u, v, w = map(int, input().split())
        edges.append((u, v, w))
    for edge in edges:
        u, v, w = edge
        # 注意重边, 保证当前边的权重最小
        if mpt[u][v] > w:
            mpt[u][v] = w
            mpt[v][u] = w

    floyd()
    print(mpt[1][n])

```

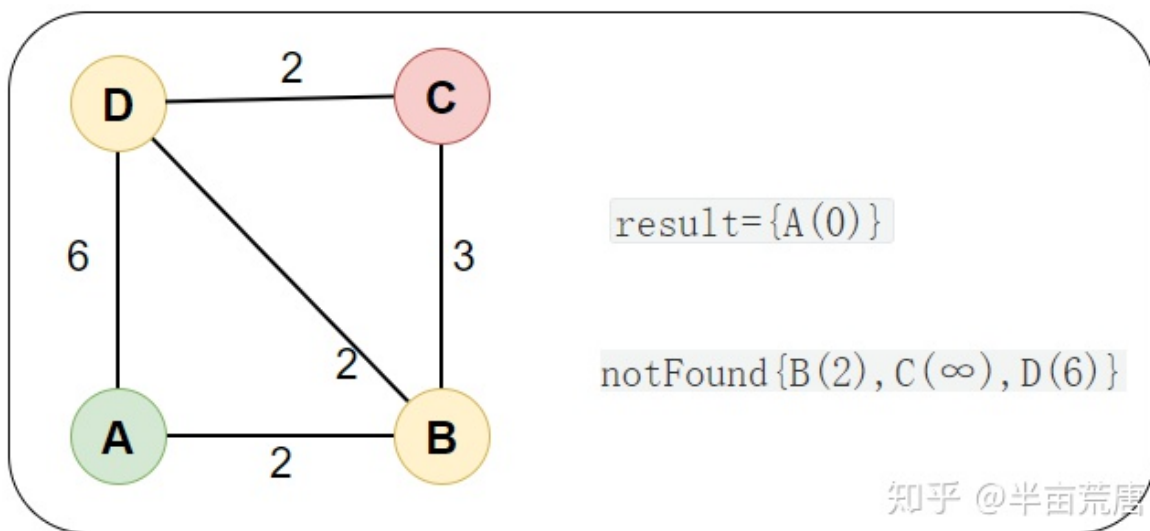
单源

Dijkstra算法

- **result**: 已求出 最小路径的顶点
- **notFound**: 未求出最小路径的顶点, 里面的值是 **到起点的距离**

每次从「未求出最短路径的点」中 取出 距离距离起点 最近的点, 以这个点为桥梁 刷新「未求出最短路径的点」的距离

初始, `result={A(0)}` 中只有起点 A, `notFound={B(2),C(∞),D(6)}` 中是除了 A 点的其他点, 里面的值是到起点的距离 (例如 B(2) 代表 B点到起点的距离为 2)



然后，从「未求出最短路径的点」notFound 中取出 最短路径的点 B(2)，然后通过 B(2) 为桥梁 刷新「未求出最短路径的点」的距离

1. 取出 最短路径的点：

从「未求出最短路径的点」notFound 中取出 最短路径的点 B(2)，放入结果 result 中，结果如下：

「未求出最短路径点」 notFound={C(∞), D(6)}，「已求出最短路径的点」 result={A(0), B(2)}

1. 刷新距离：

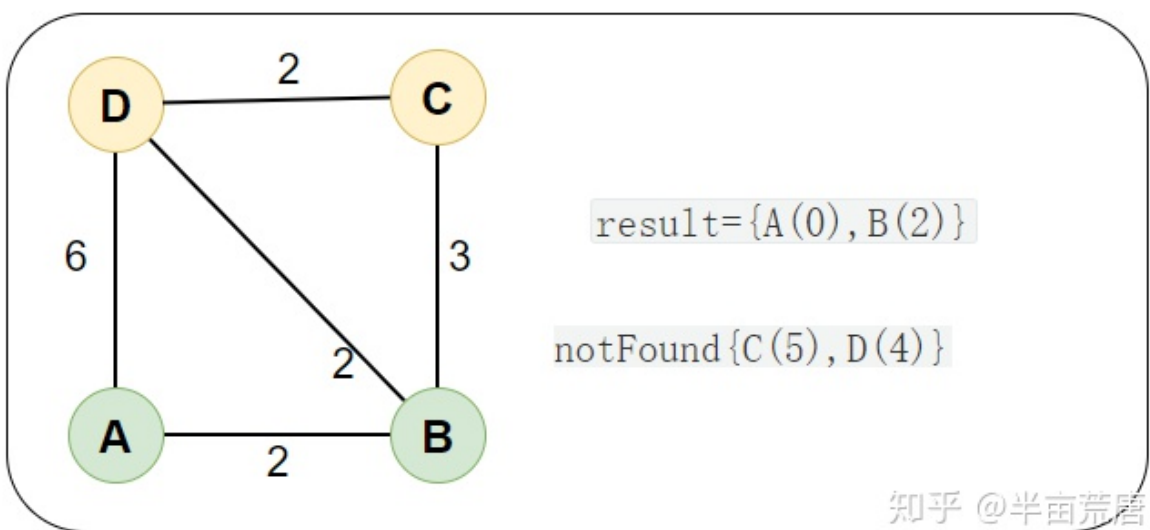
通过 B(2) 为桥梁，刷新距离。

例如 $AD = 6 < AB + BD = 4$ 以 B(2) 为桥梁的距离更短，就刷新「未求出最短路径点」D(6) 的距离为 D(4)

notFound={C(∞), D(4)}

同理刷新 C(∞) 的距离为 C(5)，最后结果如下：

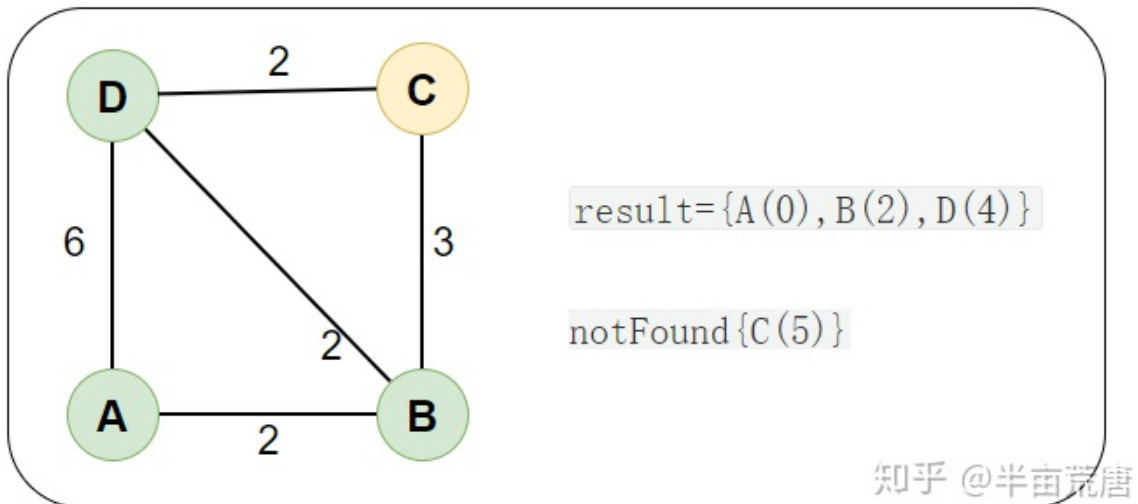
「未求出最短路径点」 notFound={C(5), D(4)}，「已求出最短路径的点」 result={A(0), B(2)}



然后，从「未求出最短路径的点」notFound 中取出 最短路径的点 D(4)，然后通过 D(4) 为桥梁 刷新「未求出最短路径的点」的距离

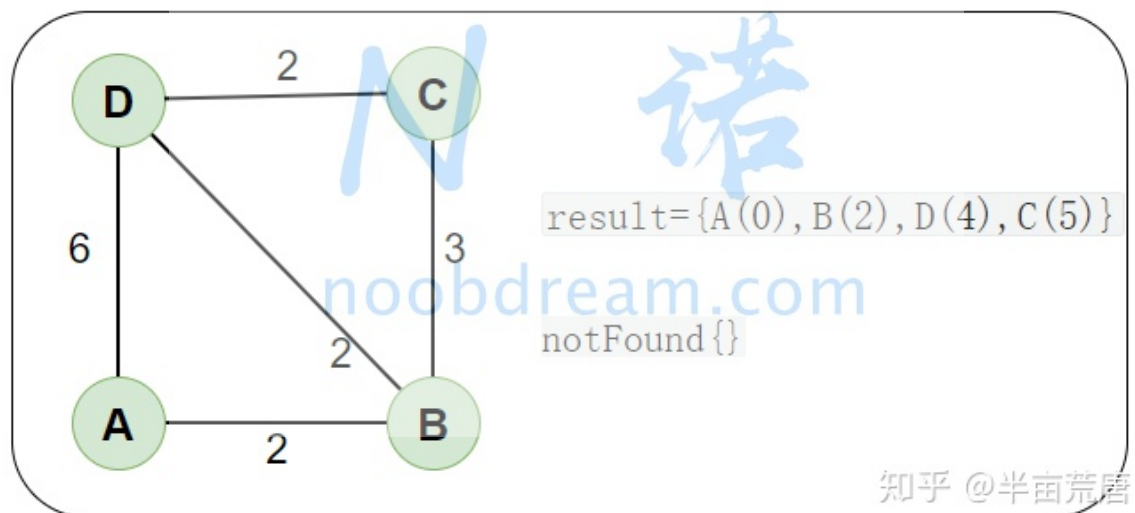
同理，最后结果如下：

「未求出最短路径点」 `notFound={C(5)}`，「已求出最短路径的点」 `result={A(0),B(2),D(4)}`



然后，从「未求出最短路径的点」 `notFound` 中取出 最短路径的点 `C(5)`，算法结束

`result={A(0), B(2), D(4), C(5)}` 就是最终所求的 最短距离



模板代码

```
# 图中的顶点数
v = 6
# 标记数组: used[v] 值为False说明该顶点还没有访问过，保证notfound内取过之后不放回
used = [False for _ in range(v)]
# 距离数组: distance[i]表示从源点s到i的最短距离，distance[s]=0
distance = [float('inf') for _ in range(v)]
# cost[u][v]表示边e=(u,v)的权值，不存在时设为INF
cost = [[float('inf') for _ in range(v)] for _ in range(v)]

def dijkstra(s):
    # 起始节点cost为0
    distance[s] = 0
    while True:
        # v在这里相当于是一个哨兵，对包含起点s做统一处理！
        v = -1
```

从未使用过的顶点中选择一个距离最小的顶点,此处相当于从not_found里取出能到达的cost最小的节点

```
for u in range(v):
    if not used[u] and (v == -1 or distance[u] < distance[v]):
        v = u
if v == -1: # 说明所有顶点都维护到S中了!
    break
```

将选定的顶点加入到S中, 同时进行距离更新

```
used[v] = True
```

更新U中各个顶点到起点s的距离。之所以更新U中顶点的距离,是由于上一步中确定了k是求出最短路径的顶点,从而可以利用k来更新其它顶点的距离;例如,(s,v)的距离可能大于(s,k)+(k,v)的距离。即每次更新与跳板v的最短路径的顶点

```
for u in range(v):
    distance[u] = min(distance[u], distance[v] + cost[v][u])
```

```
if __name__ == '__main__':
```

```
    Inf = float('inf')
```

```
    cost = [[0, 1, 12, Inf, Inf, Inf],
             [Inf, 0, 9, 3, Inf, Inf],
             [Inf, Inf, 0, Inf, 5, Inf],
             [Inf, Inf, 4, 0, 13, 15],
             [Inf, Inf, Inf, Inf, 0, 4],
             [Inf, Inf, Inf, Inf, Inf, 0]]
```

```
    s = int(input('请输入一个起始点(0-5): '))
```

```
    dijkstra(s)
```

```
    print(distance) # [0, 1, 8, 4, 13, 17]
```

例1

路径距离最短的情况下, 还需要对花费排序

最短路径问题

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

给你 n 个点， m 条无向边，每条边都有长度 d 和花费 p ，给你起点 s 终点 t ，要求输出起点到终点的最短距离及其花费，如果最短距离有多条路线，则输出花费最少的。

输入输出格式

输入描述:

输入 n, m ，点的编号是 $1 \sim n$ ，然后是 m 行，每行4个数 a, b, d, p ，表示 a 和 b 之间有一条边，且其长度为 d ，花费为 p 。最后一行是两个数 s, t ；起点 s ，终点 t 。 n 和 m 为0时输入结束。
($1 < n \leq 1000$, $0 < m < 100000$, $s \neq t$)

输出描述:

输出 一行有两个数， 最短距离及其花费。

输入输出样例

输入样例#:

```
3 2
1 2 5 6
2 3 4 5
1 3
0 0
```

复制

输出样例#:

```
9 11
```

复制

两种实现方法:

```
def floyd():
    for k in range(1, n + 1):
        for i in range(1, n + 1):
            for j in range(1, n + 1):
                cur_l = cost[i][j][0]
                cur_w = cost[i][j][1]
                new_l = cost[i][k][0] + cost[k][j][0]
                new_w = cost[i][k][1] + cost[k][j][1]
                if cur_l > new_l or (cur_l == new_l and cur_w > new_w):
                    cost[i][j] = (new_l, new_w)

def dijkstra(s, distant):
    distant[s] = (0, 0)
```



```

while True:
    v = -1
    # 寻找cost最小的路径
    for i in range(1, n + 1):
        if not used[i] and (v == -1 or (distant[v][0] > distant[i][0] or (
            distant[v][0] == distant[i][0] and distant[v][1] >
distant[i][1]))):
            v = i
    # 找不到了
    if v == -1:
        break

    used[v] = True

    for i in range(1, n + 1):
        new_l = distant[v][0] + cost[v][i][0]
        new_w = distant[v][1] + cost[v][i][1]
        cur_l = distant[i][0]
        cur_w = distant[i][1]
        if new_l < cur_l or (cur_l == new_l and new_w < cur_w):
            distant[i] = (new_l, new_w)
    return distant

while True:
    n, m = map(int, input().split())
    if n == m == 0:
        break
    cost = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(n + 1):
            if i == j:
                cost[i][j] = (0, 0)
            else:
                cost[i][j] = (float('inf'), float('inf'))

    edges = []
    for _ in range(m):
        u, v, l, w = map(int, input().split())
        edges.append((u, v, l, w))
    s, e = map(int, input().split())
    for edge in edges:
        u, v, l, w = edge
        # 防止重边
        if cost[u][v][0] > l or (cost[u][v][0] == l and cost[u][v][1] > w):
            cost[u][v] = (l, w)
            cost[v][u] = (l, w)

    used = [False for _ in range(n + 1)]
    distant = [(float('inf'), float('inf')) for _ in range(n + 1)]
    # floyd()
    distant = dijkstra(s, distant)
    # print(cost[s][e][0], cost[s][e][1])
    print(distant[e][0], distant[e][1])

```

例2

题目:

这个国家正面临着一场可怕的内战---这个国家的城市被分为两部分，支持不同的领导人。M先生是商人，不关心政治，但他知道形势的严峻，你的任务是帮助他尽快回家。“为了安全起见，”m先生说，“你的路线**最多应该包含一条连接两个不同阵营**的城市的道路。”你能告诉M先生至少要多久才能到达他甜蜜的家吗？

输入描述:

输入包含多个测试用例。

每个案例的第一行是一个整数N ($2 \leq N \leq 600$)，表示该国的城市数量。

第二行包含一个整数M ($0 \leq M \leq 10000$)，这是道路的数量。

下面的M线是道路的信息。每一行包含三个整数A、B、T，表示从A城市到B城市的道路需要花费时间T。T的取值范围为[1500]。

下一部分包含N个整数，它们不是1就是2。第i个整数表示第i个城市的支持领导者。

为了简化问题，我们假设M先生从城市1开始，他的目标是城市2。城市1始终支持领导者1，而城市2则在领导者2的同一侧。

注意，所有的道路都是双向的，两个城市之间最多只有一条路。

输入以N=0的大小写结束。

输出描述:

对于每个测试用例，输出一个表示到达家的最小时间的整数。

如果不可能按照M先生的要求到达家，则输出-1。

输入输出样例

输入样例#:

```
2
1
1 2 100
1 2
3
3
1 2 100
1 3 40
2 3 50
1 2 1
5
5
3 1 200
5 3 150
2 5 160
4 3 170
4 2 170
1 2 2 2 1
0
```

输出样例#:

```
100
90
540
```

题目总结: 求最短路径，但是节点被划分了阵营，路径最多不能经过两次不同的阵营。即对路径有条件限制，在构建distant数组时用元组来保存条件信息。

```

def dijkstra(s):
    distant[s] = (0, 0)
    while True:
        v = -1
        for i in range(1, n + 1):
            if not used[i] and (v == -1 or distant[v][0] > distant[i][0]):
                v = i
        if v == -1:
            break
        used[v] = True
        for i in range(1, n + 1):
            now_d_c = distant[i][0]
            new_d_c = distant[v][0] + cost[v][i]
            # 计算出当前交换经过的阵营次数，若阵营不同则abs(p[i] - p[v])等于1，表示交换了阵
            new_d_sum = distant[v][1] + abs(p[i] - p[v])
            if new_d_c < now_d_c and new_d_sum <= 1:
                distant[i] = (new_d_c, new_d_sum)

while True:
    try:
        n = int(input())
        if n == 0:
            break
        m = int(input())
        cost = [[float('inf')] * (n + 1) for _ in range(n + 1)]
        for i in range(n + 1):
            for j in range(n + 1):
                if i == j:
                    cost[i][j] = 0
        # 元组信息为：(最短路径， 交换的阵营次数)
        distant = [(float('inf'), 0) for _ in range(n + 1)]
        used = [False for _ in range(n + 1)]
        edges = []
        for i in range(m):
            s, e, w = map(int, input().split())
            edges.append((s, e, w))
        # 阵营信息
        p = [0] + list(map(int, input().split()))

        for edge in edges:
            s, e, w = edge
            # 防重边
            if cost[s][e] > w:
                cost[s][e] = w
                cost[e][s] = w
        dijkstra(1)
        ans = distant[2][0]
        if ans == float('inf'):
            print(-1)
        else:
            print(ans)
    except:
        break

```

拓扑排序

原理

1、定义：对一个有向无环图G进行拓扑排序，是将G中所有顶点排成一个线性序列，使得图中任意一对顶点u和v，若边 $(u,v) \in E(G)$ ，则u在线性序列中出现在v之前。通常，这样的线性序列称为满足拓扑次序的序列，简称**拓扑序列**。

2、条件：

- 每个顶点出现且只出现一次
- 若A在序列中排在B的前面，则在图中不存在从B到A的路径

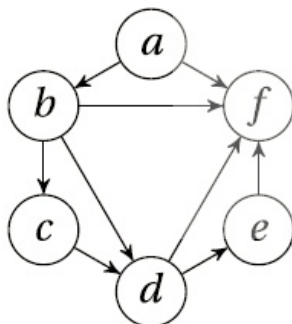
3、特性：

- 拓扑排序并不唯一
- 有向无环图一定存在拓扑排序

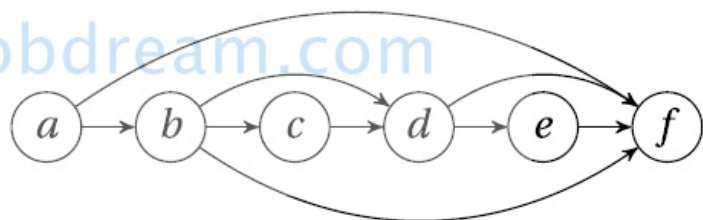
4、步骤：

- 选择一个入度为0的顶点并输出之；
- 从图中删除此顶点及其所有出边；
- 循环执行以上两步，直到不存在入度为0的顶点为止；如果剩下入度非0的顶点（输出的顶点数小于图中的顶点数），就说明有回路，不存在拓扑排序。

5、示例：



DAG

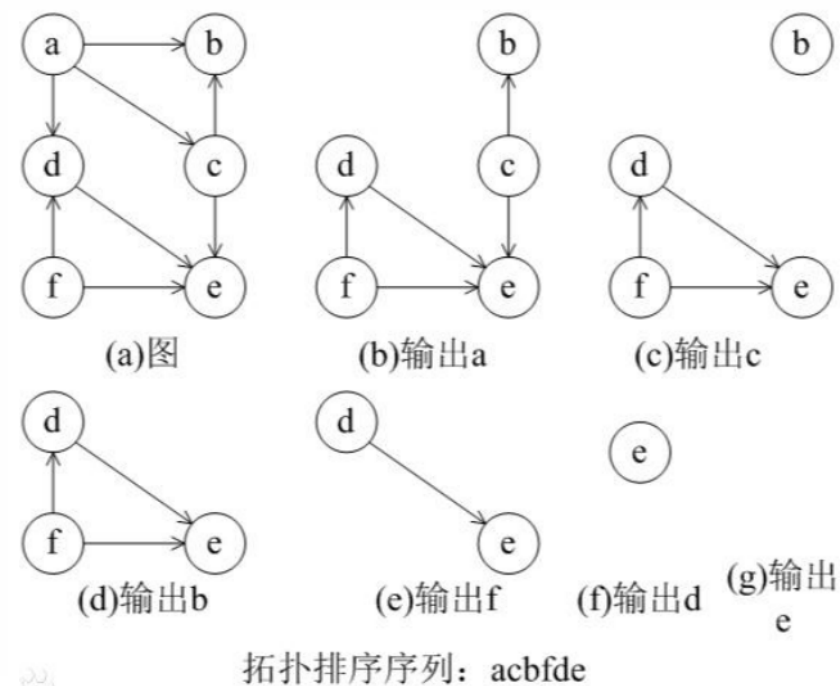


Topologically Sorted DAG

将上图用字典表示： $G = \{ 'a': 'bf', 'b': 'cdf', 'c': 'd', 'd': 'ef', 'e': 'f', 'f': '' \}$

算法过程（图以a为例）：

下面是算法的演示过程。



代码模板

```
def topoSort(graph):
    in_degrees = dict((u,0) for u in graph) #初始化所有顶点入度为0
    num = len(in_degrees)
    for u in graph:
        for v in graph[u]:
            in_degrees[v] += 1 # 存储每个顶点的入度
    Q = [u for u in in_degrees if in_degrees[u] == 0] # 筛选入度为0的顶点
    Seq = [] # 排序结果
    while Q:
        u = Q.pop() # 此处可以处理对一个入度为0的顶点排序逻辑
        Seq.append(u)
        for v in graph[u]:
            in_degrees[v] -= 1 #移除其所有出边，如删'a'，则'a'->'b'的边删去，'b'的入
            # 度减一
            if in_degrees[v] == 0:
                Q.append(v) #再次筛选入度为0的顶点，删除节点后，再次补充出度为0
                # 的节点
    if len(Seq) == num: #输出的顶点数是否与图中的顶点数相等，若是无环图，每个顶点的入
        # 度必为0一次，即被添加到seq列表里
        return Seq
    else:
        return None

G = {
    'a': 'bf',
    'b': 'cdf',
    'c': 'd',
    'd': 'ef',
    'e': 'f',
    'f': ''
}

print(topoSort(G))
```

输出结果:

['a', 'b', 'c', 'd', 'e', 'f']

例1

确定比赛名次

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

有 N 个比赛队 ($1 \leq N \leq 500$)，编号依次为1, 2, 3, ..., N 进行比赛，比赛结束后，裁判委员会要将所有参赛队伍从前往后依次排名，但现在裁判委员会不能直接获得每个队的比赛成绩，只知道每场比赛的结果，即 $P1$ 赢 $P2$ ，用 $P1, P2$ 表示，排名时 $P1$ 在 $P2$ 之前。现在请你编程序确定排名。

输入输出格式

输入描述:

输入有若干组，每组中的第一行为二个数 N ($1 \leq N \leq 500$)， M ；其中 N 表示队伍的个数， M 表示接着有 M 行的输入数据。接下来的 M 行数据中，每行也有两个整数 $P1, P2$ 表示即 $P1$ 队赢了 $P2$ 队。

输出描述:

给出一个符合要求的排名。输出时队伍号之间有空格，最后一名后面没有空格。

其他说明：符合条件的排名可能不是唯一的，此时要求输出时编号小的队伍在前；输入数据保证是正确的，即输入数据确保一定能有一个符合要求的排名。

输入输出样例

输入样例#:

[复制](#)

```
4 3
1 2
2 3
4 3
```

输出样例#:

[复制](#)

```
1 2 4 3
```

```
from queue import PriorityQueue
```

```
def toposort(G):
```

```

in_degrees = dict((u, 0) for u in G)
# 初始化入度
for k in G:
    for v in G[k]:
        in_degrees[v] += 1
# 结果要求编号小的在前保证结果唯一，这里可以用优先队列实现
Q = PriorityQueue()
for u in in_degrees:
    if in_degrees[u] == 0:
        Q.put(u)
# Q = [u for u in in_degrees if in_degrees[u] == 0]
seq = []
while not Q.empty():
    q = Q.get()
    seq.append(q)
    for v in G[q]:
        in_degrees[v] -= 1
        if in_degrees[v] == 0:
            Q.put(v)
if len(seq) == len(G):
    return seq
# 成环
else:
    return None

while True:
    try:
        n, m = map(int, input().split())
        G = dict((u, []) for u in range(1, n + 1))
        for i in range(m):
            w, l = map(int, input().split())
            G[w].append(l)
        seq = topoSort(G)
        print(' '.join([str(v) for v in seq]))
    except:
        break

```

动态规划

思路

对于动态规划问题，我将拆解为如下五步曲，这五步都搞清楚了，才能说把动态规划真的掌握了！

1. 确定dp数组（dp table）以及下标的含义（重要）
2. 确定递推公式
3. dp数组如何初始化
4. 确定遍历顺序
5. 举例推导dp数组

最大子段和

很多题目能用利用最大子段和的思路结题，有时候需要转换思路求解，如例1

思路

- 1、假设 $dp[i]$ 表示，以 i 索引为结束的最大子序列和（即包括 $datas[i]$ 的最大子序列和）
- 2、那么原问题就转化为 $dp[i]$, 其中 $i = 0, 1, 2, \dots, n-1$ 中的最大值（即以每个 i 为结束的所有情况的最大值）
- 3、递推公式：很容易得到

```
dp[i] = max(dp[i-1] + datas[i], datas[i])
```

（即最大值要么就是 $dp[i-1]$ 加上 i 为索引的值，或者当 $dp[i-1] + datas[i]$ 为负数，最大值 i 为索引的值本身）

模板

最大序列和

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 32768 mb

给出一个整数序列 S ，其中有 N 个数，定义其中一个非空连续子序列 T 中所有数的和为 T 的“序列和”。对于 S 的所有非空连续子序列 T ，求最大的序列和。变量条件： N 为正整数， $N \leq 1000000$ ，结果序列和在范围 $(-2^{63}, 2^{63}-1)$ 以内。

输入输出格式

输入描述:

第一行为一个正整数 N ，第二行为 N 个整数，表示序列中的数。

输出描述:

输入可能包括多组数据，对于每一组输入数据，仅输出一个数，表示最大序列和。

输入输出样例

输入样例#:

复制

```
5
1 5 -3 2 4

6
1 -2 3 4 -10 6

4
-3 -1 -2 -5
```

输出样例#:

复制

```
9
7
-1
```

```
while True:
    try:
        n= int(input())
        datas = list(map(int,input().split()))
        enter =input()
        dp = [0]*(n+1)
        dp[0]=datas[0]
        res = dp[0]
        for i in range(1,n):
            dp[i]=max(datas[i],dp[i-1]+datas[i])
            res =max(dp[i],res)
        print(res)
    except:
        break
```

例1

字符串区间翻转

题目描述:

小诺有一个由 0 和 1 组成的字符串

现在小诺有一次机会,可以选择一个任意的区间[L,R],将该区间内的所有字符串进行翻转(即 0→1,1→0)。

请问小诺经过一次翻转之后字符串中最多会有多少个 1?

输入描述:

第一行输入一个正整数 n,表示字符串长度, $n \leq 10^7$ 。

接下来一行输入一个 01 字符串。

输出描述:

输出题目要求的答案。

输入样例#:

4

1001

输出样例#:

4

题目来源:

DreamJudge 1642

将输入的字符串的0换成1,1换成-1,求最大子序列和,求得的结果为:经过翻转能得到的最多的‘1’的数目
上诉结果加上原先字符串1的个数即为最终结果

```
while True:
    try:
        n = int(input())
        strings = list(input())
        one_count= strings.count('1')
        strings = [1 if v =='0' else -1 for v in strings]
        dp=[0]*(n+1)
        dp[0]=strings[0]
        ans = 0
        for i in range(1,n):
            dp[i]=max(dp[i-1]+strings[i],strings[i])
            ans = max(dp[i],ans)

        print(ans+one_count)
    except:
        break
```

最长上升子序列

最大上升子序列和

题目描述:

一个数的序列 b_i , 当 $b_1 < b_2 < \dots < b_S$ 的时候, 我们称这个序列是上升的。对于给定的一个序列 $(a_1, a_2, \dots, a_N)$, 我们可以得到一些上升的子序列 $(a_{i_1}, a_{i_2}, \dots, a_{i_K})$, 这里 $1 \leq i_1 < i_2 < \dots < i_K \leq N$ 。比如, 对于序列 $(1, 7, 3, 5, 9, 4, 8)$, 有它的一些上升子序列, 如 $(1, 7)$, $(3, 4, 8)$ 等等。这些子序列中序列和最大为 18, 为子序列 $(1, 3, 5, 9)$ 的和。你的任务, 就是对于给定的序列, 求出最大上升子序列和。注意, 最长的上升子序列的和不一定是最大的, 比如序列 $(100, 1, 2, 3)$ 的最大上升子序列和为 100, 而最长上升子序列为 $(1, 2, 3)$ 。

输入描述:

思路

与最大字段和类似

- 1、假设 $dp[i]$ 表示, 第 i 项为结尾的最长上升子序列长度 (包括 $datas[i]$)
- 2、那么原问题就转化为 $dp[i]$, 其中 $i = 0, 1, 2, \dots, n-1$ 中的最大值 (即以每个 i 为结束的所有情况的最大值)
- 3、递推公式:

```
dp[i] = max(dp[i], max(dp[j])+1) if j < i and a[j] < a[i] # 其中max函数可以用二分法查找
```

(即最大值要么就是 dp 中最长序列加1 (下一个值符合上升要求), 或者为 $dp[i]$ (下一个值不符合要求))

模板

```
def LIS_val():
    ans = dp[0]
    for i in range(n):
        dp[i] = datas[i]
        # 从前往后更新前i个dp的值
        for j in range(i):
            # 若末尾的datas[i]值大于遍历到的datas[j], 则说明当前dp值需要更新
            if datas[i] > datas[j]:
                dp[i] = max(dp[i], dp[j] + datas[i])
        ans = max(dp[i], ans)
    return ans

def LIS_len():
    ans = 0
    for i in range(n):
        dp[i] = 1 # 默认开始长度为1
        # 从前往后更新前i个dp的值
        for j in range(i):
            # 若末尾的datas[i]值大于遍历到的datas[j], 则说明当前dp值需要更新
            if datas[i] > datas[j]:
                dp[i] = max(dp[i], dp[j] + 1)
        ans = max(dp[i], ans)
    return ans
```

```
while True:
    try:
        n = int(input())
        datas = list(map(int,input().split()))
        dp= [0]*(n+1)
        dp[0]=datas[0]
        ans = LIS_len()
        print(ans)
    except:
        break
```

例1

题目，即让输入转换为中间高两边低的形式，如“130 150 200 180 160”才符合要求

合唱队形

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

N位同学站成一排，音乐老师要请其中的(N-K)位同学出列，使得剩下的K位同学不交换位置就能排成合唱队形。合唱队形是指这样的一种队形：设K位同学从左到右依次编号为1, 2, ..., K，他们的身高分别为T1, T2, ..., TK，则他们的身高满足T1 < T2 < ... < Ti, Ti > Ti+1 > ... > TK (1 ≤ i ≤ K)。你的任务是，已知所有N位同学的身高，计算最少需要几位同学出列，可以使得剩下的同学排成合唱队形。

输入输出格式

输入描述:

输入的第一行是一个整数N (2 ≤ N ≤ 100)，表示同学的总数。
第一行有n个整数，用空格分隔，第i个整数Ti (130 ≤ Ti ≤ 230) 是第i位同学的身高（厘米）。

输出描述:

可能包括多组测试数据，对于每组数据，
输出包括一行，这一行只包含一个整数，就是最少需要几位同学出列。

输入输出样例

输入样例#:

复制

```
8
186 186 150 200 160 130 197 2
```

输出样例#:

复制

```
4
```

思路：分别求输入datas（左）和翻转后的datas（右）的最大上升序列长度，相加后减1（自身重复）得到的结果即为坐标i下的合唱队形长度

```
while True:
    try:
        n = int(input())
        datas = list(map(int, input().split()))
        datas_re = datas[::-1]
        dp_l = [0]*n
        dp_r = [0]*n
        lit_len = 1
        for i in range(n):
            dp_l[i] = 1
            # 更新左值
            for j in range(i):
                if datas[i] > datas[j]:
                    dp_l[i] = max(dp_l[i], dp_l[j] + 1)
            dp_r[i] = 1
            # 更新右值
            for k in range(i):
                if datas_re[i] > datas_re[k]:
                    dp_r[i] = max(dp_r[i], dp_r[k] + 1)

        ans_li = [dp_l[i] + dp_r[::-1][i] - 1 for i in range(n)]
        lit_len = max(ans_li)
        print(n - lit_len)
    except:
        break
```

例2

记录单调序列的值

最长递减子序列

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

输入数字n，和n个整数，输出该数字串中的最长递减序列

输入输出格式

输入描述:

见样例

输出描述:

见样例

输入输出样例

输入样例#:

8
9 4 3 2 5 4 3 2

复制

输出样例#:

9 5 4 3 2

复制

思路

通过dp数组，可以得出最长递减子序列的长度

通过一个辅助数组记录“前驱”，就是当前dp值的来源（此处默认pre起始index为1）

例

原序列 9 4 3 2 5 4 3 2

dp数组 1 2 3 4 2 3 4 5

pre数组 0 1 2 3 1 5 6 7

dp数组中5的来源是7（原序列的第7位元素），意思是5对应的数字2，其前一个数字应该是第7个数字（3），

3的前驱是第6个数字（4），

4的前驱是第5个数字（5），

5的前驱是第1个数字（9），

5的前驱是0，0是序列的源点，代表到此结束

代码

```
def SIL():
    dp = [0]*n
    dp[0] = 1
    pre = [-1]*n
    max_dp = 1
    ans = []
    for i in range(n):
        dp[i]=1
        for j in range(i):
            # 递减, 递增为datas[i]>datas[j]
            if datas[i]<datas[j]:
                if dp[j]+1>dp[i]:
                    dp[i]=dp[j]+1
                    pre[i]= j
        max_dp =max(max_dp, dp[i])

    # 获取最大递增序列的index, 此index与datas的index一致
    max_i = dp.index(max_dp)
    next_i = max_i
    # 当前驱为-1时, 表示已到达最长序列的起点
    # 为-1的点不止有一个, 每一个单调序列都有一个起点为-1, 我们只需要遍历最大的单调序列即max_i
    的路径即可
    # 如输入长度为7的 '4 3 2 5 4 3 2'时获得的pred为[-1,0,1,-1,3,4,5] , 有两组序列为'4 3
    2'与'5 4 3 2'
    while next_i!=-1:
        ans.append(datas[next_i])
        # 获取下一个前驱坐标
        next_i = pre[next_i]

    return ans

while True:
    try:
        n = int(input())
        datas =list(map(int,input().split()))
        #datas = datas[::-1]
        ans = SIL()
        #ans = ans[::-1]
        print(' '.join([str(v) for v in ans[::-1]]))
    except:
        break
```

例3

题目, 求序列里最大连续递增子串中的最大差值, 如序列“1 6 2 4 6”的最大连续递增子串为“1 6”, 最大差值为5

输出描述:

所有爬坡中的最大高度差，如果路线数列里面没有爬坡，就输出0。

输入输出样例

输入样例#:

复制

```
5
1 2 1 4 6
6
10 8 8 6 4 3
```

输出样例#:

复制

```
5
0
```

```
# 动态规划方法
# dp[i]表示包括i点的最大连续递增子串长度
# ans 表示最大爬坡值
def LSC():
    ans = 0
    for i in range(1,n):
        # 因为是连续子串，所以不需要j来从头遍历之前不连续的节点
        if datas[i]>datas[i-1]:
            dp[i]=dp[i-1]+1
            # 当遍历到连续递增子串时，因为dp[i]表示长度，所以i-dp[i]表示爬坡的起始点，
            # datas[i]-datas[i-dp[i]]即为新的连续递增子串中的爬坡高度
            new_ans = datas[i]-datas[i-dp[i]]
            ans = max(new_ans,ans)
    return ans

while True:
    try:
        n = int(input())
        datas = list(map(int,input().split()))
        dp = [0]*n
        ans = LSC()
        print(ans)
    except:
        break
```

```
# 双指针法
def LSC():
    # 左右指针
    l,r = 0,1
    ans = 0
    # 遍历到边界退出
    while r<n:
        # 判断连续递增，若是连续递增，则右指针移动，左指针不动。直到遍历到非递增点
        if datas[r]>datas[r-1]:
            # 获取连续递增子串的最大差值
            ans = max(ans,datas[r]-datas[l])
            r+=1
        # 遍历到非递增点，右指针继续遍历，左指针指向右指针前
```



```

        else:
            r+=1
            l=r-1
        return ans

while True:
    try:
        n = int(input())
        datas = list(map(int,input().split()))
        print(LSC())
    except:
        break

```

最长公共子序列

即在n个字符串的n组子序列中（一般为两个），找出最长的相同子序列

如“abcd”与“cxbydz”，最长子序列为“bd”

思路

接下来我们分析算法的状态转移方程 $dp[i, j]$ 代表 a 字符串前 i 个字符组成的子串和 b 字符串前 j 个字符组成的子串的 LCS。

那么

```

# 初始化
dp[i, j] = 0 if i = 0 or j = 0
# 当前点为公共点,则当前前i与前j组成的最大子串的LSC为,前i-1与前j-1的最大子串的LSC+1
if i>0 and j > 0 and data1[i-1]==data2[j-1]:
    dp[i, j] = dp[i - 1, j - 1] + 1
# 当前点不为公共点,则选最大的LSC值(要么是前i-1与前j最大,要么是前i与前j-1最大)
if i>0 and j > 0 and data1[i-1]!=data2[j-1]:
    dp[i, j] = max{dp[i, j - 1], dp[i - 1, j]}

```

模板

```

for i in range(1,n):
    for j in range(1,n):
        if data1[i-1]==data2[j-1]:
            dp[i][j]=dp[i-1][j-1]+1
        else:
            dp[i][j]=max(dp[i-1][j],dp[i][j-1])

```

例1

求两字符串最大公共子序列长度

输入输出样例

输入样例#:

复制

```
abcd
cxbydz
```

输出样例#:

复制

```
2
```

```
while True:
    try:
        data1 = list(input())
        data2 = list(input())
        n = len(data1)
        m = len(data2)
        # 记录最长的公共子序列
        res = ""

        # 初始化注意别把len(data1), len(data2)弄反了,此时n为行,m为列
        dp = [[0] * (m + 1) for _ in range(n + 1)]
        # 遍历行
        for i in range(1,n+1):
            # 遍历列
            for j in range(1,m+1):
                if data1[i-1]==data2[j-1]:
                    dp[i][j]=dp[i-1][j-1]+1
                    res += text1[i - 1]
                else:
                    dp[i][j]= max(dp[i-1][j],dp[i][j-1])
        print(dp[n][m])
    except:
        break
```

例2

最长连续公共子序列

题目描述

[查看答案](#) [查看题解](#)

Time Limit: 1000 ms

Memory Limit: 256 mb

输入两个字符串s1,s2。输出最长连续公共子串长度和最长连续公共子串。

输入输出格式

输入描述:

多组数据输入。
输入两个字符串s1,s2，长度不大于100，以空格隔开。

输出描述:

输出最长连续公共子串长度和最长连续公共子串。

输入输出样例

输入样例#:

abcdefg qwerndefiok

复制

输出样例#:

4
cdef

复制

```
while True:
    try:
        s1,s2 = map(list,input().split())
        n1,n2 = len(s1),len(s2)
        # dp代表包含i和j的前i和前j的最大连续子串的长度
        dp = [[0]*(n2+1) for _ in range(n1+1)]
        # 记录s2中最大连续子串末尾的坐标
        max_j = 0
        # 记录最大连续子串的长度
        max_dp=1
        for i in range(1,n1+1):
            for j in range(1,n2+1):
                if s1[i-1]==s2[j-1]:
                    # 此处若是连续序列，则会在前一个值上加一
                    dp[i][j]=dp[i-1][j-1]+1
                    # 取最大的连续序列，当前序列为最大的时候，更新坐标(因为结果取最后一个子
                    # 串，所以是>=)
                    if dp[i][j]>=max_dp:
                        max_dp = dp[i][j]
                        max_j = j-1
        # 根据长度和坐标可得到最大子串序列的结果
```

```

res=''
max_len = max_dp
while max_len>0:
    res =s2[max_j]+res
    max_len-=1
    max_j-=1
print(max_dp)
print(res)
except:
    break

```

背包问题

01 背包基础理论（一）

有N件物品和一个最多能背重量为W 的背包。第i件物品的重量是weight[i]，得到的价值是value[i]。每件物品只能用一次，求解将哪些物品装入背包里物品价值总和最大。

背包最大重量为4。

物品为：

	重量	价值
物品0	1	15
物品1	3	20
物品2	4	30

问背包能背的物品最大价值是多少？

二维dp数组01背包

依然动规五部曲分析一波。

1. 确定dp数组以及下标的含义

对于背包问题，有一种写法，是使用二维数组，即dp[i][j]表示从下标为[0-i]的物品里任意取，放进容量为j的背包，价值总和最大是多少。（其中j表示当前背包的重量大小）

只看这个二维数组的定义，大家一定会有点懵，看下面这个图：

$dp[i][j]$		背包重量j:				
		0	1	2	3	4
物品0:						
物品1:						
物品2:						



1. 确定递推公式

再回顾一下 $dp[i][j]$ 的含义：从下标为 $[0-i]$ 的物品里任意取，放进容量为 j 的背包，价值总和最大是多少。

那么可以有两个方向推出来 $dp[i][j]$,

- **不放物品i**：由 $dp[i - 1][j]$ 推出，即背包容量为 j ，里面不放物品 i 的最大价值，此时 $dp[i][j]$ 就是 $dp[i - 1][j]$ 。(其实就是当物品 i 的重量大于背包 j 的重量时，物品 i 无法放进背包中，所以被背包内的价值依然和前面相同。)
- **放物品i**：由 $dp[i - 1][j - \text{weight}[i]]$ 推出， $dp[i - 1][j - \text{weight}[i]]$ 为背包容量为 $j - \text{weight}[i]$ 的时候不放物品 i 的最大价值，那么 $dp[i - 1][j - \text{weight}[i]] + \text{value}[i]$ （物品 i 的价值），就是背包放物品 i 得到的最大价值

所以递归公式： $dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - \text{weight}[i]] + \text{value}[i])$; (因为不放也可能比把当前的物品放了的价值大，比如对于重量为1，价值为100，与重量为2，价值为1的物品。当遍历到容量为2时，若添加2物品，则总价值变成 $dp[0][0] = 0 + 1$ 了，所以选择不添加2物品，继续装1物品价值更高)

1. dp数组如何初始化

关于初始化，一定要和dp数组的定义吻合，否则到递推公式的时候就会越来越乱。

首先从 $dp[i][j]$ 的定义出发，如果背包容量 j 为0的话，即 $dp[i][0]$ ，无论是选取哪些物品，背包价值总和一定为0。如图：

dp[i][j]

背包重量j:

	0	1	2	3	4
物品0:	0				
物品1:	0				
物品2:	0				



在看其他情况。

状态转移方程 $dp[i][j] = \max(dp[i-1][j], dp[i-1][j - \text{weight}[i]] + \text{value}[i])$; 可以看出i 是由 i-1 推导出来, 那么i为0的时候就一定要初始化。

$dp[0][j]$, 即: i为0, 存放编号0的物品的時候, 各个容量的背包所能存放的最大价值。

那么很明显当 $j < \text{weight}[0]$ 的时候, $dp[0][j]$ 应该是 0, 因为背包容量比编号0的物品重量还小。

当 $j \geq \text{weight}[0]$ 时, $dp[0][j]$ 应该是 $\text{value}[0]$, 因为背包容量放足够放编号0物品。

代码初始化如下:

```
for (int j = 0 ; j < weight[0]; j++) { // 当然这一步, 如果把dp数组预先初始化为0了, 这一步就可以省略, 但很多同学应该没有想清楚这一点。
    dp[0][j] = 0;
}
// 正序遍历
for (int j = weight[0]; j <= bagweight; j++) {
    dp[0][j] = value[0];
}
```

此时dp数组初始化情况如图所示:

dp[i][j]

背包重量j:

	0	1	2	3	4
物品0:	0	15	15	15	15
物品1:	0				
物品2:	0				



那么问题来了，先遍历 物品还是先遍历背包重量呢？

其实都可以！！但是先遍历物品更好理解。

那么我先给出先遍历物品，然后遍历背包重量的代码。

```
// weight数组的大小 就是物品个数
for(int i = 1; i < weight.size(); i++) { // 遍历物品
    for(int j = 0; j <= bagweight; j++) { // 遍历背包容量
        if (j < weight[i]) dp[i][j] = dp[i - 1][j];
        else dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]);
    }
}
```

先遍历背包，再遍历物品，也是可以的！（注意我这里使用的二维dp数组）

例如这样：

```
// weight数组的大小 就是物品个数
for(int j = 0; j <= bagweight; j++) { // 遍历背包容量
    for(int i = 1; i < weight.size(); i++) { // 遍历物品
        if (j < weight[i]) dp[i][j] = dp[i - 1][j];
        else dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + value[i]);
    }
}
```

为什么也是可以的呢？

要理解递归的本质和递推的方向。

$dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - \text{weight}[i]] + \text{value}[i])$; 递归公式中可以看出 $dp[i][j]$ 是靠 $dp[i - 1][j]$ 和 $dp[i - 1][j - \text{weight}[i]]$ 推导出来的。

$dp[i - 1][j]$ 和 $dp[i - 1][j - \text{weight}[i]]$ 都在 $dp[i][j]$ 的左上角方向（包括正上方向），那么先遍历物品，再遍历背包的过程如图所示：

dp[i][j]

背包重量j:

	0	1	2	3	4
物品0:	0	15	15	15	15
物品1:	0	15	15	20	0
物品2:	0	0	0	0	0

 代码随想录

再看看先遍历背包，再遍历物品呢，如图：

dp[i][j]

背包重量j:

	0	1	2	3	4
物品0:	0	15	15	15	0
物品1:	0	15	15	20	0
物品2:	0	15	15	0	0

 代码随想录

大家可以看出，虽然两个for循环遍历的次序不同，但是dp[i][j]所需要的数据就是左上角，根本不影响dp[i][j]公式的推导！

```
def test_2_wei_bag_problem1(bag_size, weight, value) -> int:
    rows, cols = len(weight), bag_size + 1
    dp = [[0 for _ in range(cols)] for _ in range(rows)]

    # 初始化dp数组.
    for i in range(rows):
        dp[i][0] = 0
    first_item_weight, first_item_value = weight[0], value[0]
    for j in range(1, cols):
        if first_item_weight <= j:
            dp[0][j] = first_item_value

    # 更新dp数组：先遍历物品，再遍历背包.
    for i in range(1, len(weight)):
```



```

    cur_weight, cur_val = weight[i], value[i]
    for j in range(1, cols):
        if cur_weight > j: # 说明背包装不下当前物品。
            dp[i][j] = dp[i - 1][j] # 所以不装当前物品。
        else:
            # 定义dp数组: dp[i][j] 前i个物品里, 放进容量为j的背包, 价值总和最大是多
            # 少。
            dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - cur_weight] + cur_val)

    print(dp)

if __name__ == "__main__":
    bag_size = 4
    weight = [1, 3, 4]
    value = [15, 20, 30]
    test_2_wen_bag_problem1(bag_size, weight, value)

```

模板

设有一个背包可以放入的物品重量为 S , 现有 n 件物品, 重量分别是 $w_1, w_2, w_3, \dots, w_n$ 。
问能否从这 n 件物品中选择若干件放入背包中, 使得放入的重量之和正好为 S 。
如果有满足条件的选择, 则此背包有解, 否则此背包问题无解。

输入输出格式

输入描述:

输入数据有多行, 包括放入的物品重量为 s , 物品的件数 n , 以及每件物品的重量 (输入数据均为正整数)
多组测试数据。

输出描述:

对于每个测试实例, 若满足条件则输出"YES", 若不满足则输出"NO"

输入输出样例

输入样例#:

复制

```

20 5
1 3 5 7 9

```

输出样例#:

复制

```

YES

```

```

def bag_pro(bag_size,wei):
    # 得先将重量排序, 遍历也是从小重量到大重量顺序遍历

```

```

# wei.sort()
rows,cols = len(wei),bag_size+1
dp = [[0 for _ in range(cols)] for _ in range(rows)]
# 初始化
for j in range(1,cols):
    if wei[0]<=j:
        dp[0][j] = wei[0]

for i in range(1,rows):
    cur_wei = wei[i]
    for j in range(1,cols):
        # 装不下
        if j<cur_wei:
            dp[i][j]=dp[i-1][j]
        # 能装下
        else:
            # 当有val存在时，需要加max函数对比，因为有可能装下该物品也没有不装的价值高
            # 比如对于重量为1，价值为100，与重量为2，价值为1的物品。当遍历到容量为2时，
            # 若添加2物品，则总价值变成dp[0][0]=0+1了，所以选择不添加2物品，继续装1物品价值更高
            dp[i][j] = max(dp[i-1][j],dp[i-1][j-cur_wei]+cur_wei)

ans = 'NO'
for k in range(rows):
    if dp[k][bag_size]==bag_size:
        ans = 'YES'
        break
return ans
while True:
    try:
        bag_size,wei_n = map(int,input().split())
        wei = list(map(int,input().split()))
        ans = bag_pro(bag_size,wei)
        print(ans)
    except:
        break

```

例1

背包问题，难点：需要知道选择了哪些物品

输入输出格式

输入描述:

输入数据为多组，每组输入的第一行有两个正整数M和N ($M < 100$ 且 $N < 1000$)，分别为哆啦A梦可以支配的钱数和清单上的可选择物品种类。接下来的N行每行有两个正整数，分别为每种物品的价钱和它的受欢迎程度（编号为1到N）。

输出描述:

如果存在物品购买，那么输出的第一行为能够达到的最大的受欢迎程度。第二行为需要购买的物品的编号（如果有多种可能，输出字典序靠前的那种），空格分隔每个数字；如没有物品可以购买，输出只有一行，为数字0。

输入输出样例

输入样例#:

[复制](#)

```
10 4
100 5
5 5
5 5
10 10
```

输出样例#:

[复制](#)

```
10
2 3
```

思路

以下题目为例

设有n件物品 $x_1, x_2, \dots, x_n$ ，每件物品有一个价值和一个重量，分别记为 $v_1, v_2, \dots, v_n$ 和 $w_1, w_2, \dots, w_n$ ，其中所有的 w_i 均为整数。现有一个背包，其最大载重量为m，要求从这n件物品中任取若干件（这些物品要么被装入要么被留下）。问背包中装入哪些物品可是得所装物品的价值和最大？

样例1输入：m=11, n=6

物体编号i	0	1	2	3	4	5
物体重量 w_i	2	4	5	6	10	3
物体价值 v_i	1	7	4	5	11	1

可以很容易得到其dp数组：

p[i, w]	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	1	1	1	1	1	1	1	1	1	1
1	0	0	1	1	7	7	8	8	8	8	8	8
2	0	0	1	1	7	7	8	8	8	11	11	12
3	0	0	1	1	7	7	8	8	8	11	12	12
4	0	0	1	1	7	7	8	8	8	11	12	12

4	0	0	1	1	7	7	8	8	8	11	12	12
p[i, w]	0	1	2	3	4	5	6	7	8	9	10	11
5	0	0	1	1	7	7	8	8	8	11	12	12

还有一个问题，那就是到底选择了哪些物品能够获得这个最大价值？我们可以用rec数组记录下决策过程，在填充p[i, j]时，如果选择了i商品，则rec[i, j]=1；反之，rec[i, j]=0。然后我们可以用rec数组来追踪最优解，一行一行往回倒推，当rec[i, j]=1时，选择商品i，考察子问题p[i-1, w-weight[i]]；当rec[i, j]=0时，不选商品i，考察子问题p[i-1, c]。(c为与上一值相同的列)

可以得到决策数组rec为：

以样例1为例，rec[5, 11]=0，说明没有选择5号商品，则rec[5, 11]是由rec[4, 11]推导而来的；再看rec[4, 11]=0，说明也没有选择4号商品，则rec[4, 11]是由rec[3, 11]推导而来的；再看rec[3, 11]=0，说明也没有选择3号商品，则rec[3, 11]是由rec[2, 11]推导而来的；再看rec[2, 11]=1，说明选择了2号商品，则rec[2, 11]是由rec[1, 11-weight[2]]即rec[1, 6]推导而来的；再看rec[1, 6]=1，说明也选择了1号商品，则rec[1, 6]是由rec[0, 6-weight[1]]即rec[0, 2]推导而来的；再看rec[0, 2]=1，说明也选择了0号商品。

rec[i, w]	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	1	1	1	1	1	1	1	1	1	1
1	0	0	0	0	1	1	1	1	1	1	1	1
2	0	0	0	0	0	0	0	0	0	1	1	1
3	0	0	0	0	0	0	0	0	0	0	1	0
4	0	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	0	0

代码

```
def bag_pro(bag_size, wei, val):
    rows, cols = len(wei), bag_size + 1
    dp = [[0 for _ in range(cols)] for _ in range(rows)]
    # 记录物品有没有买过,即买该物品后背包的价值有没有变大(记录所有路径)
    buy = [[0 for _ in range(cols)] for _ in range(rows)]
    # 初始化
    for j in range(wei[0], cols):
        dp[0][j] = val[0]
        buy[0][j] = 1

    # 开始遍历,先物品,再背包
    for i in range(1, rows):
        cur_wei, cur_val = wei[i], val[i]
        for j in range(1, cols):
            # 装不下
            if cur_wei > j:
                dp[i][j] = dp[i-1][j]
            # 装得下
            else:
                next_dp = dp[i-1][j-cur_wei] + cur_val
                if next_dp > dp[i-1][j]:
                    dp[i][j] = next_dp
```

```

        buy[i][j]=1
    else:
        dp[i][j] = dp[i-1][j]

li=[]
# 从后往前遍历即当背包容量最大, 可选物品最多的时候对物品遍历, 当遍历到1时, 该物品必为最大价值中的一件 (即在最大价值的路径之中)
tmp = bag_size
for i in range(rows-1, -1, -1):
    if buy[i][tmp] == 1:
        li.append(i)
        # 路径返回, 该物品被买过, 背包容量减小, 根据记录路径返回查找下一件物品
        tmp -= wei[i]

return dp[rows-1][cols-1],li

while True:
    try:
        bag_size, wei_n = map(int, input().split())
        wei, val = [], []
        for _ in range(wei_n):
            w, v = map(int, input().split())
            wei.append(w)
            val.append(v)
        max_val, wei_i = bag_pro(bag_size, wei, val)
        if len(wei_i)==0:
            print(0)
        else:
            print(max_val)
            print(' '.join([str(v+1) for v in wei_i[::-1]]))
    except:
        break

```

线段树

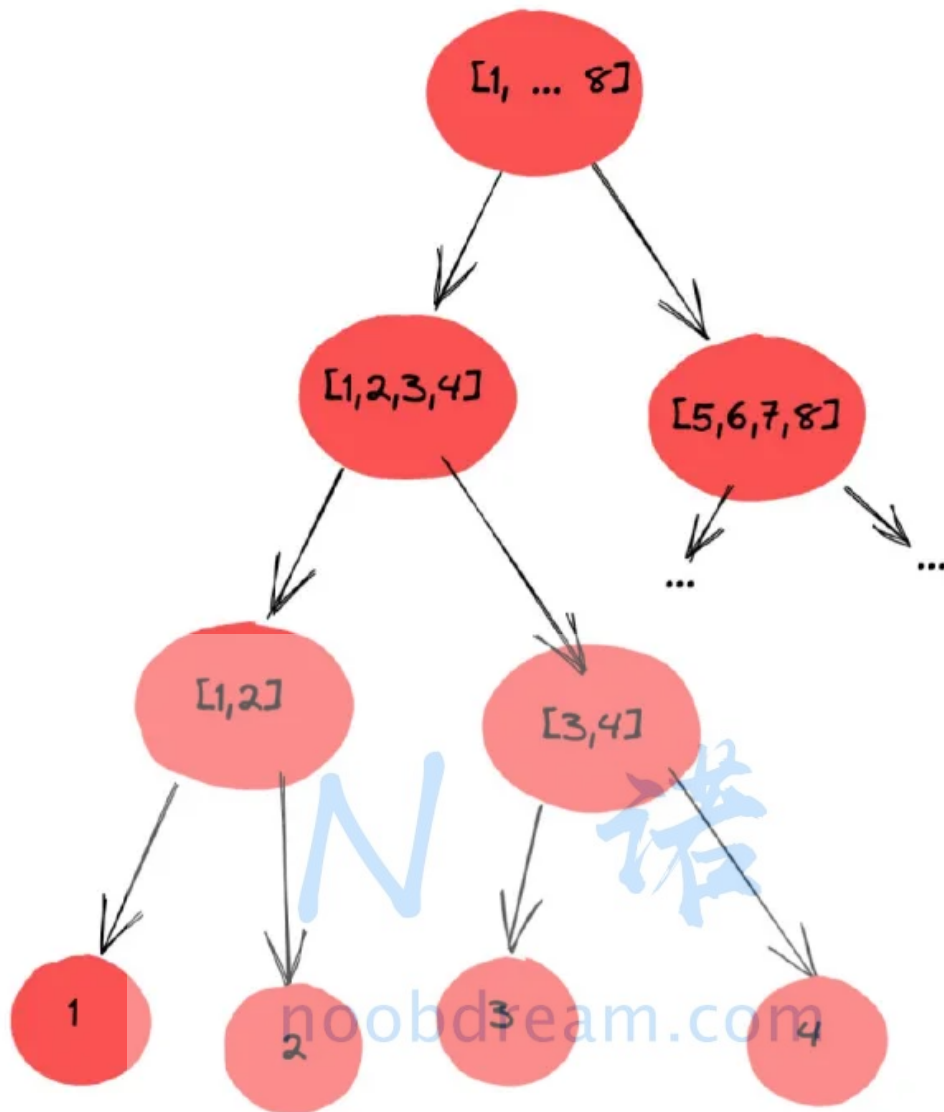
适用场景：给定若干区间和值，统计给定区间内的值（最大值，最小值，总和等）

如：给出一份服务器一天的log文件，包含用户id，登录时间，登出时间，求当天服务器最大在线人数是多少。

原理

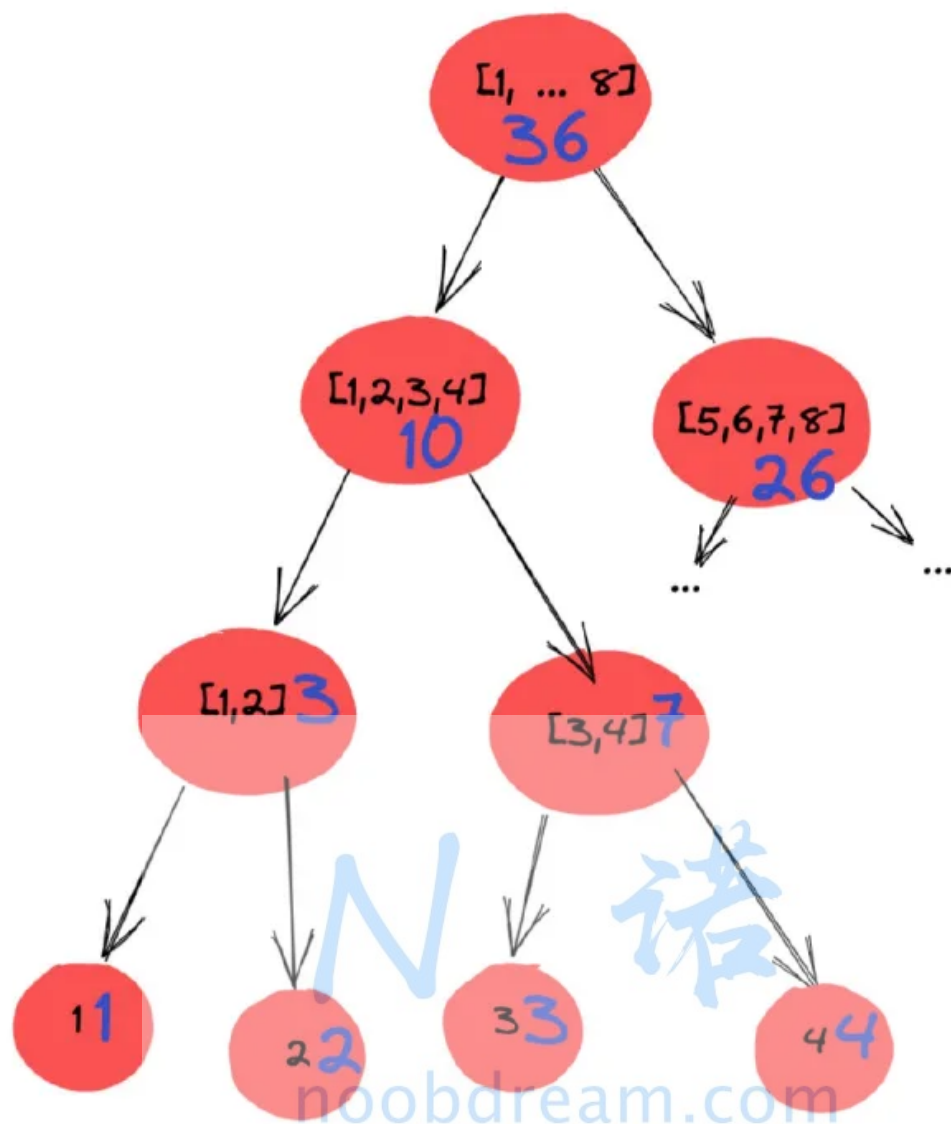
我们可以将区间 A 和 区间 B 分别作为一个树的左右节点，并将 A 的区间和与 B 的区间和分别存储到左右子节点中。

接下来，将 A 的区间分为左右两部分，同理 B 也分为左右两部分。不断执行此过程直到无法继续分。



知乎 @lucifer

总结一下就是将区间不断一分为二，并将区间信息分别存储到左右节点。如果是求和，那么区间信息就是区间的和。这个时候的线段树大概是这样的：



知乎 @lucifer

“蓝色字体表示的区间和。”

注意，这棵树的所有叶子节点一共有 n 个（ n 为原数组长度），并且每一个都对应到原数组某一个值。

体现到代码上也很容易。直接使用「**后续遍历**」即可解决。这是因为，我们需要知道左右节点的统计信息，才能计算出当前节点的统计信息。

和二叉堆的表示方式一样，我们可以用数组表示树，用 $2 * i + 1$ 和 $2 * i + 2$ 来表示左右节点的索引，其中 i 为当前节点对应到 $tree$ 上的索引。

例1

题目描述： n 为天数， n 行每一行的 L, R 表示区间 $[L, R]$ 加 1。 m 行为需要输出的答案数量，每一行 k, l 表示天数左右界，表示仅应用 $[k, l]$ 区间天的数据。

要求输出 所有区间最大的值和其最大值的索引

1327. 魔法学校

题目

提交记录

讨论

题解

视频讲解

在一所魔法学校中，有许多的学生，每个学生有一个学号（从 1 开始,用连续的正整数编号）。

每天，这所学校的校长会向一些学生发送一条短信（具体来说，在第 i 天，校长会向学号在区间 $[L_i, R_i]$ 内的学生发送一条短信，每天的 L_i 和 R_i 可能不同）；从第 1 天开始，一共持续 n 天。

现在，学校的教导主任想要知道，在第 A_i 天到第 B_i 天（包含第 A_i 天和第 B_i 天），哪个学生收到的短信数量最多，最多数量是多少。

由于教导主任太忙了，所以现在麻烦你帮他找到答案。

输入格式

输入文件第一行一个正整数 n ，表示校长连续 n 天向学生发送短信。

接下来 n 行，每行两个正整数 L_i, R_i ，表示第 i 天校长向学号在区间 $[L_i, R_i]$ 内的同学发一条短信。

接下来一行一个正整数 m ，表示教导主任有 m 个询问。

接下来 m 行，每行两个正整数 A_i, B_i ，表示询问第 A_i 天到第 B_i 天（包含第 A_i 天和第 B_i 天），哪个学生收到的短信数量最多，最多数量是多少。

输出格式

输出文件共 m 行，每行输出空格隔开的两个正整数表示对应的教导主任询问的答案。

每行的第一个数表示收到最多短信的学生的学号，如果有多个学生均收到了最多数量的短信，你只需要输出他们之中最小的学号。

每行的第二个数表示该学生在第 A_i 天到第 B_i 天（包含第 A_i 天和第 B_i 天）一共收到了多少条短信。

数据范围

对于30%的数据， $1 \leq n \leq 5 \times 10^2, 1 \leq L_i \leq R_i \leq 5 \times 10^2, 1 \leq m \leq 5 \times 10^2$

对于70%的数据， $1 \leq n \leq 5 \times 10^3, 1 \leq L_i \leq R_i \leq 5 \times 10^3, 1 \leq m \leq 5 \times 10^3$

对于100%的数据， $1 \leq n \leq 5 \times 10^4, 1 \leq L_i \leq R_i \leq 5 \times 10^4, 1 \leq m \leq 5 \times 10^4, 1 \leq A_i \leq B_i \leq n$

输入样例：

```
5
1 5
1 3
3 5
2 2
4 10
3
1 1
1 3
3 4
```

输出样例：

```
1 1
3 3
2 1
```

定义树节点，l,r, val表示该节点记录的是区间[l, r]的最大值是val

```
class Tree():
    def __init__(self):
        self.l = 0
        self.r = 0

        self.val = 0
    # 方便调试
    # 打印函数
    def __str__(self):
```



```

        return '%s,%s,%s' % (self.l, self.r, self.val)

# 打印函数
def __repr__(self):
    return '%s,%s,%s' % (self.l, self.r, self.val)
class seg_tree:
    def __init__(self,min_l,max_r,datas) -> None:
        self.tree = [Tree() for i in range((max_r)*4)]
        self.build(1,1,max_r)
        self.datas = datas
        self.min_l,self.max_r = min_l,max_r
        self.ans=[]
# 建树, 用cur<<1访问左子树, cur<<1|1访问右子树, 位运算操作很方便
    def build(self,cur, l, r):
        self.tree[cur].l, self.tree[cur].r, self.tree[cur].lazy,
self.tree[cur].val = l, r, 0, 0
        # 当l==r的时候结束递归
        if l < r:
            mid = l + r >> 1
            self.build(cur<<1, l, mid)
            self.build(cur<<1|1, mid+1, r)

# 当子节点计算完成后, 用子节点的值来更新自己的值
    def pushup(self,cur):
        self.tree[cur].val = max(self.tree[cur<<1].val, self.tree[cur<<1|1].val)

# 单点更新
    def add(self,cur, x, v):
        if self.tree[cur].l == self.tree[cur].r:
            self.tree[cur].val += v
        else:
            mid = self.tree[cur].r + self.tree[cur].l >> 1
            if x > mid:
                self.add(cur>>1|1, x, v)
            else:
                self.add(cur>>1, x, v)
            self.pushup(cur)

# 将lazy标记向下传递一层
    def pushdown(self,cur):
        if self.tree[cur].lazy:
            lazy = self.tree[cur].lazy
            self.tree[cur<<1].lazy += lazy
            self.tree[cur<<1|1].lazy += lazy
            self.tree[cur<<1].val += lazy
            self.tree[cur<<1|1].val += lazy
            self.tree[cur].lazy = 0

# 区间更新, 从cur索引开始更新
    def update(self,cur, l, r, v):
        # if l <= self.tree[cur].l and self.tree[cur].r <= r:
        #     self.tree[cur].lazy += v
        #     self.tree[cur].val += v
        #     return

```

```

        if l <= self.tree[cur].l and self.tree[cur].r <= r and
self.tree[cur].l==self.tree[cur].r:
            self.tree[cur].val += v
            return
        if r < self.tree[cur].l or l > self.tree[cur].r:
            return
        # if self.tree[cur].lazy:
        #     self.pushdown(cur)
        self.update(cur<<1, l, r, v)
        self.update(cur<<1|1, l, r, v)
        self.pushup(cur)

# 区间查询,从cur开始查询, 查询区间L到R的最大值
def query(self,cur, l, r):
    # if l <= self.tree[cur].l and self.tree[cur].r <= r:
    #     return self.tree[cur].val,cur
    if l <= self.tree[cur].l and self.tree[cur].r <= r and self.tree[cur].l
== self.tree[cur].r:
        return self.tree[cur].val,self.tree[cur].l
    if self.tree[cur].l > r or self.tree[cur].r < l:
        return 0,self.tree[cur].l
    # if self.tree[cur].lazy:
    #     self.pushdown(cur)
    l_v,l_cur =self.query(cur<<1, l, r)
    r_v,r_cur = self.query(cur<<1|1, l, r)
    if l_v>=r_v:
        #self.ans.append(l_cur)
        return l_v,l_cur
    elif l_v<r_v:
        #self.ans.append(r_cur)
        return r_v,r_cur

# 测试
# -----
#      ---
# -----
#  --
#      --

# 二叉树是堆形式, 可以用一维数组存储, 注意数组长度要开4倍空间
def get_ans(self):
    for l,r in self.datas:
        self.update(1,l,r,1)
    max_v,max_i = self.query(1,self.min_l,self.max_r)
    return max_i,max_v

n =int(input())
datas = []
for _ in range(n):
    l,r = map(int,input().split())
    datas.append((l,r))

m = int(input())
for _ in range(m):
    s,e = map(int,input().split())

```

```

min_l,max_r = 50000,0
for l,r in datas[s-1:e]:
    min_l,max_r=min(min_l,l),max(max_r,r)
sgt= seg_tree(min_l,max_r,datas[s-1:e])
day,val = sgt.get_ans()
print(day,val)

```

清华真题

题目1

描述

有 n 名学生参加军训，军训的一大重要内容就是走队列，而一个队列的不整齐程度是该队中最高的学生的身高与最矮的学生的身高差值的平方。

现在要将 n 名参加军训的学生分成 k 个队列，每个队列的人数可以是任意非负整数。

在安排队列时希望所有队列的不整齐度之和尽量小，请问不整齐度之和最小可以是多少？

输入样例：

```

3 2
170 180 168

```

输出样例：

```

4

```

思路

贪心+动态规划

贪心

可以看出这些数先排序后再进行分队会更优，为什么因为如果选取的两个队列有交集，那么可以把交集里的数放进任意一个队列里，这样差值会更小。

dp

我们用 $f(i,j)$ 表示将前 i 个人分为 j 个队列的所有方案中的最小值。

我们考虑最后对 $f(i,j)$ 有影响的因素，那就是最后一个队列怎么安排。

我们考虑枚举最后一个队列的开始位置（结束位置当然是 i ），那么可以得出枚举的范围是 j 到 i 。（如果前 $j-1$ 个人每人一个队列，后面 $i-j+1$ 个人都放进一个队列，开始位置就是 j ，如果 i 一个人一个队列，那么开始的位置就是 i ）

用 k 表示最后一个队列开始的位置，求出的和就是前面 $k-1$ 个人分为 $j-1$ 个队列的不整齐度最小值加上最后一个队列的不整齐度。

那么就可以得到（ k 从 j 到 i ）：

$$f(i,j)=\min(f(i,j),f(k-1,j-1)+(h[i]-h[k])*(h[i]-h[k]))$$

最后的 $f(n,k)$ 就是答案。

时间复杂度: $O(n^3)$

代码

```
n,k = map(int,input().split())
datas = list(map(int,input().split()))
datas = [0]+datas
datas.sort()
#我们用 f(i,j)表示将前 i个人分为 j 个队列的所有方案中的最小值。
dp = [[0 for _ in range(k+1)] for _ in range(n+1)]
for i in range(1,n+1):
    for j in range(1,k+1):
        # 当队列为数1时, 整齐度直接为最后一个减第一个的平方
        if j==1:
            dp[i][j] = (datas[i]-datas[1])**2
            continue
        dp[i][j]=float('inf')
        # 需要遍历最后一个队列的所有情况, 取最小值。因为有j个队列, 所以l最小为j, 有i个人, l最大为i。
        for l in range(j,i+1):
            dp[i][j] = min(dp[i][j],dp[l-1][j-1]+(datas[i]-datas[l])**2)

print(dp[n][k])
```

N 佬
noobdream.com